

Die post-symbolische Kausalsprache

PSK Referenzarchitektur

Vollständige, deterministische Softwarearchitektur
der kanonischen Referenzimplementierung

PSK-RA v1.0.48

Gegenstand. Dieses Werk ist der vollständige Bauplan des Softwarestacks, der die kanonische PSK-Spezifikation S_{PSK} und den konstitutionellen Maschinenvertrag C_{PSK} als ausführbare Maschine realisiert. Es definiert Module, Schichten, Typen, Zwischenrepräsentation, Compiler, Scheduler, Laufzeit, Speicher, Gates, Capabilities, Effekte, Witnesses, Receipts, Reconciliation, Persistenz, Repository, Build, Fehlerordnung, Recovery, Konformanz, Zertifizierung und Deployment abschließend.

Geltung. Nach Abschluss dieses Werkes existieren keine offenen Architekturfragen. Ein Implementierungsagent leitet die vollständige Referenzimplementierung ausschließlich aus diesem Dokument zusammen mit dem kanonischen PSK-Korpus ab.

Elternnorm 1	Kanonische Gesamtspezifikation v1.0 (S_{PSK})
Elternnorm 2	Konstitutioneller Maschinenvertrag (C_{PSK})
Elternkörper	Theorie-Monolith, native Feldkognitions- und Realitätshorizontnormalform
Konstitutions-ID	aa10aa5a9ffe5d9205ce7950905f06a81cbce0e66c7d49b77be2529fd09c0f56
Elterntheorie-Digest	829df842e54b9ccb51e2bfa30c6b6af0939828eea26752506e6aa66685861583
Artefaktrang	alleinige Architekturnorm der Realisierungsphase
Status	normativ, geschlossen

Kanonische Zielidentität

KANONISCHER HAUPTSATZ DER ARCHITEKTUR

Die PSK-Referenzmaschine ist ein deterministischer, konstitutionsgebundener Kognitionscompiler mit angeschlossener bezugter Laufzeit. Sie nimmt versiegelte Außenrecords auf, erzeugt daraus Anker, kompiliert Claims über eine getypte Realitätsgrammatik auf das topologische Syntaxskelett M_{13} , emittiert eine kanonische Zwischenrepräsentation, unterwirft jeden Kandidaten einem adversarialen Kanonisierungskern, verklebt nur seam-kompatible Lokalstrukturen, bindet jede Außenwirkung an Gate und Token, nimmt ausschließlich unabhängig erzeugte Receipts entgegen und promoviert Faktstatus allein durch geschlossene Reconciliation. Jede Nichtclosure bleibt als Residuum sichtbar. Jeder Lauf ist durch Run Descriptor, Trace und Replay reproduzierbar. Jede Instanz weist ihre Zugehörigkeit zur Maschinenklasse durch Konstitutions-, Implementierungs- und Laufzeitidentität sowie ein Maschinenzertifikat nach.

Artefaktrang

Dieses Werk ist die alleinige Architekturnorm der Realisierungsphase. Es fügt dem kanonischen Korpus keine Theorie hinzu. Es überführt die kanonischen Normen vollständig in eine implementierbare Softwarearchitektur und schließt alle innerhalb des Korpus eindeutig entscheidbaren Architekturresiduen.

Norm	In diesem Werk beantwortete Frage
Theorie-Monolith	Was ist die Intelligenzform? — nicht Gegenstand dieses Werkes.
Maschinenvertrag	Welche Invarianten MUSS jede Realisierung erzwingen?
Gesamtspezifikation	Welche Grammatik, Syntax, IR, Passfolge und Konformanz gelten?
PSK-RA v1.0.48	Aus welchen Komponenten, Typen, Automaten, Ports, Speichern, Fehlern und Artefakten besteht die Maschine, und wie werden sie gebaut?
Referenzimplementierung	Der Code, der genau diese Architektur realisiert.

Abschlussbedingung

Die Papierphase ist mit der kanonischen Gesamtspezifikation geschlossen. Die Architekturphase ist mit dem vorliegenden Werk geschlossen. Der nächste normative Gegenstand ist ausschließlich das ausführbare Repository. Änderungen an dieser Architektur sind nur zulässig, wenn die Implementierung einen nicht ausdrückbaren notwendigen Zustand, einen Normwiderspruch oder eine belegte Kernwiderlegung nachweist; sie erzeugen dann eine neue vollständige Architekturfassung mit neuer Architektur-ID.

Statusdisziplin und Leseordnung

Jede Aussage dieses Werkes besitzt genau einen normativen Status. Es existieren keine erläuternden, motivierenden, historischen oder metatheoretischen Aussagen.

Status	Bedeutung
Definition	Legt ein Objekt, eine Struktur oder eine Abbildung fest. Bindend.
Struktur	Legt ein konkretes Datenschema mit Feldern und Typen fest. Bindend.
Schnittstelle	Legt Ports, Signaturen, Fehler und Capabilities fest. Bindend.
Automat	Legt Zustände, Transitionen, Wächter und verbotene Kanten fest. Bindend.
Algorithmus	Legt eine deterministische Prozedur fest. Bindend.
Axiom	Nicht ableitbare Festlegung. Bindend, blocking.
Invariante	Prädikat, das über den gesamten Geltungsbereich gilt. Bindend, blocking.
Vertrag	Pflicht einer Komponente gegenüber ihrer Umgebung. Bindend.
Regel	Deterministische Entscheidungs- oder Zuordnungsvorschrift. Bindend.
Obligation	Logisch nicht entscheidbares Residuum; explizit offen, nicht blocking für die Architektur, blocking für den Release der betroffenen Klasse.

Normative Sprache

Term	Bedeutung
MUSS	Absolute Anforderung. Verletzung erzeugt Nichtkonformität und blockiert den Release.
DARF NICHT	Absolutes Verbot. Verletzung erzeugt FAIL.
SOLL	Anforderung, deren Nichterfüllung eine dokumentierte, tracegebundene Begründung und ein sichtbares Residuum erzeugt.
DARF	Zulässige, nicht erzwungene Möglichkeit innerhalb der deklarierten Grenzen.

Identifikatorordnung

Alle normativen Objekte dieses Werkes tragen stabile Identifikatoren. Ein Identifikator DARF NICHT nach Vergabe umgewidmet werden; er wird ausschließlich als DEPRECATED markiert.

Präfix	Raum	Gegenstand
<i>Mnn</i>	M00–M27	Modul der Referenzarchitektur
<i>Pnn</i>	P00–P41	Port zwischen Modulen
<i>S–X</i>	—	Sorte des Typsystems
<i>OBJ–X</i>	—	Kanonisches Laufzeit- oder Persistenzobjekt
<i>FSM–X</i>	—	Zustandsautomat
<i>OP–X</i>	—	Instruktion der nativen Instruktionsmenge
<i>Cn</i>	C1–C11	Compilerpass
<i>G–X</i>	—	Gate
<i>I–X–nnn</i>	—	Invariante
<i>R–X–nnn</i>	—	Requirement
<i>PO–X–nnn</i>	—	Proof Obligation
<i>T–X–nnn</i>	—	Konformanztest
<i>PSK–Ennn</i>	—	Fehlercode
<i>WPnn</i>	WP00–WP15	Work Package
<i>In</i>	I0–I8	Realisierungsphase
<i>OBL–nnn</i>	—	offene Implementierungsverpflichtung

Fehlerkorrektur v1.0.48

Gegenüber v1.0 geschlossene Residuen, gemäß Regel Residuenbehandlung im Dokument selbst aufgelöst:

- (1) `object_registry.yaml`, OBJ-TRC (TraceSegment): Speicherklasse `trace` → `replay`. Der Registereintrag verwendete zuvor einen nicht deklarierten siebten Wert; Kapitel 16 kennt genau sechs. TraceSegment teilt Ownership (M19) mit ResidueRecord, RunDescriptor und ReplayManifest, ist aber weder Nichtclosure (Residual) noch abgeschlossener Run (Archive), sondern die Grundlage, aus der Replay rekonstruiert. Kapitel 16, Zeile *Replay Memory*, entsprechend präzisiert (Trace-Segmente ergänzt).
- (2) Requirement R-RA-006 referenzierte T-M13-001, das im Testregister nicht existierte. Alle 32 übrigen Testfamilien beginnen bei -001; T-M13 war die einzige Ausnahme. T-M13-002 (Negativtest, Kante entfernen) bleibt unverändert; T-M13-001 als fehlender Positivtest auf die Zählung selbst ergänzt.
- (3) FSM-THOUGHT und FSM-FIELD enthielten je eine Kante ohne Deckung in C_PSK Anhang D: SIMULATED → SELECTED und QUIESCENT → ACTIVE. Beide eröffnen neue Erreichbarkeit zwischen bestehenden Kernzuständen in Richtung operativer Autorität und definieren damit Kernidentität retroaktiv im Sinne von Prinzip 4.3 des Maschinenvertrags. Aus dem Kernregister entfernt; als RA-EXT-THOUGHT-01 und RA-EXT-FIELD-01 in eine gesondert geführte, domänengebundene Peripherie-Erweiterung überführt. FSM-THOUGHT und FSM-FIELD spiegeln C_PSK damit wortgetreu.
- (4) OBJ-PRJ (FieldProjection) war im Objektregister geführt, besaß aber keine Feldstruktur, obwohl das Objekt in den Kernproduktionen, an P13 und als Eingang von M10 und M11 auftritt. Struktur ergänzt, mit den Regeln *Projektion promoviert nicht* und *Occlusion ist explizit*.
- (5) OBJ-VIW (ViewArtifact) trägt `canonical: false`. Das Objekt besitzt Speicherklasse `none` und fällt nach I-ARCH-015 aus der Kanonisierung heraus; der Registereintrag dient Ownership und Lebensdauer, nicht der Digestbildung.
- (6) Zweck von `architecture/schemas/` präzisiert: Schemata der Architekturregister selbst, nicht Kopien der fünf Objektschemata aus `constitution/schemas/`. Eine Doppelführung desselben Artefakts in zwei Bündeln erzeugt divergierende Identitäten und ist unzulässig.
- (7) Definition M13Address, drittes Beispiel: `m13:0.c2.i2/1/o5` verletzte die eigene Grammatik — an der Position von `cell_id` stand die bloße Zahl 1 statt eines Tokens `(c|b|o)k`, und `o5` ist kein Knoten einer Zentrumzelle. Korrigiert zu `m13:1.c2.i2/b5/o5`. Zugleich waren zwei Wohlgeformtheitsbedingungen ungeschrieben, die die Grammatik allein nicht trägt: die Gleichheit von `level` und der Zahl der Abstiegspaare sowie die Zellzugehörigkeit von `node_id` und `parent_node_id`. Beide als Regel ergänzt, Verstoß erzeugt PSK-E011.

v1.0.5 bis v1.0.48. Neunundfuenfzig Befunde aus der Spezifikationsextraktion und der Umsetzung von WP05–WP07, alle an der Quelle geschlossen:

- (1) *Schwerwiegend.* τ_e ging in die Objekt-ID ein und brach damit die Reproduzierbarkeit der Identität: Definition Objekt-ID bildete über das ganze Objekt, Can entfernte nichts, neun kanonische Objekte tragen DualTime. Zwei inhaltsgleiche Läufe erhielten verschiedene IDs. Behoben durch Einführung der Identitätsprojektion π_{vol} , des Registers `volatile_fields.yaml` und der Trennung von Objekt-ID und `record_digest`.
- (2) I3 verlangte ein Exit-Kriterium, das nach dem Phasenplan erst ab I6 erreichbar ist (C10 über WP11, C11 über WP04). Kriterium auf C1–C9 präzisiert, Phasenabhängigkeit als Regel ergänzt.
- (3) Schreibpfad von `reality_status` und `facticity` war ungeklärt. Geregelt: ThoughtBody ist unveränderlich, der geltende Status lebt im IRNode; kein Port M07 → M06.
- (4) `object_schemas.yaml` und `volatile_fields.yaml` im Architekturbündel deklariert.
- (5) Der normative Compilerlauf widersprach den Passverträgen an drei Stellen: Typprüfung vor C2,

`close_all_18` zwischen C4 und C5 statt in C9, und eine als Gate lesbare Prüfung in C9. Alle drei angeglichen.

- (6) IMAGINARY war als Typ bezeichnet, stand aber in keiner Statusmenge. Als abgeleitetes Prädikat über `RealityStatus` und bezeugte Nichterreichbarkeit definiert.
- (7) Die in v1.0.3 ergänzte Regel zur nicht anwendbaren Spektrallinse widersprach Vertrag C5: sie verlangte eine `FieldProjection` mit gesetztem `undefined_reason`, während C5 ausdrücklich einen `ResidueRecord` des Typs `scope` statt einer leeren Projektion fordert. C5 hat Vorrang. Das Feld `undefined_reason` entfällt; die Regel ist geteilt und an C5 angeglichen. Es gibt keine leere Projektion, sondern nur eine ausgebliebene.
- (8) Der Ausschluss von `claim.text` aus Can lief neben π_{vol} als zweiter Mechanismus. Beide zusammengeführt: `volatile_fields.yaml` führt jetzt `volatile` und `non_canonical` unter derselben Projektion. Ein zweiter Ausschlusspfad ist unzulässig.
- (9) Das in v1.0.5 korrigierte Exit-Kriterium von I3 verlangte C1–C9, obwohl I4 die Work Packages WP08 und WP10 und damit C6–C9 ausdrücklich besitzt. Auf C1–C5 korrigiert; die Verteilung der Pässe über die Phasen ist jetzt tabellarisch festgelegt.
- (10) `RealityClassification` war Ownership von M07 und Nutzlast von P09, besaß aber in keinem Register eine Feldstruktur. Als OBJ–RCL ergänzt, mit dem Feld `reachability`, das die für imaginary verlangte Unterscheidung zwischen bezeugter Widerlegung und ungeprüftem Pfad trägt.
- (11) `LensSpec`, `PredicateExpr` und `RealityEvidence` waren ohne Grammatik und standen zugleich nicht unter den offenen Verpflichtungen. Als domänengelieferte opake Eingaben deklariert, mit Determinismus- und Vorgabepflichten. Sie sind entschieden, nicht offen.
- (12) `Matrix` war typlos, wird aber intern von M10 berechnet und geht in einen kanonischen Digest ein — ohne festgelegte Achsenordnung erzeugten zwei Implementierungen für denselben Inhalt verschiedene Digests. Grammatik ergänzt, Achse aufsteigend nach kanonischer Form von `SourceRef`, Zellen als ganzzahlige Zähler über gemeinsamem Nenner.
- (13) `Decimal` war undefiniert und stand im Widerspruch zum Verbot nicht exakt darstellbarer Gleitkommazahlen. Durchgängig durch `Scaled` ersetzt, den einzigen nichtganzzahligen Typ des Werkes.
- (14) `FieldProjection.source_provenance` führte `SourceId`, `DependencyProfile.sources` aber `SourceRef` — zwei Namen für dieselbe Sache über genau die Kante hinweg, die sie verbindet. Auf `SourceRef` vereinheitlicht.
- (15) Der Schlussvermerk zu v1.0.1 nannte 42 Tests; das Testregister führt 37. 42 ist die Portzahl. Korrigiert.
- (16) Bootfolge, Schritt 16, enthielt den Verweis siehe 45.3. Kapitel 45 ist das Instruktionsregister; der Inhalt steht in Kapitel 17, Abschnitt Recovery. Auf 17.3 korrigiert – vermutlich ein Rest aus C_PSK, dessen eigenes Kapitel 45 anderswo liegt.
- (17) Vertrag *Herkunftsbeglaubigung an der Prozessgrenze* präzisiert: eine vom annehmenden Prozess selbst erzeugte, exklusive Kommunikationsleitung zu einem von ihm selbst erzeugten Kindprozess (M26, `proc.control`) ist *Prozessisolation* im Sinne von Vertrag *Capability-Erzwingung* und erfüllt den Vertrag, auch ohne getrennte Benutzerkennung. Neue Obligation OBL-010 macht die davon unabhängige Anforderung getrennter Benutzerkontexte oder Namespaces (Regel *Einzelrechnerbetrieb*) explizit und blockierend ab C4, analog zu OBL-005.
- (18) Invariante *Abhängigkeitsazyklizität* führte die Schichtungleichung invertiert ($L(M_b) \leq L(M_a)$); wörtlich genommen hätte sie die spezifizierte Vorwärtspipeline (z. B. $M11 \rightarrow M12 \rightarrow M13 \rightarrow M14$) selbst verboten. Auf $L(M_b) \geq L(M_a)$ korrigiert; rückwärtsgerichtete Ausnahmen bleiben auf die in Definition *Zulässige Rückflüsse* benannten Kanäle beschränkt. P39 ($M13 \rightarrow M09$, `ProbeRequest`) dort als fünfter Kanal (*Sondierungsrückfluss*) ergänzt – er bildete zuvor einen echten Zyklus $M09 \rightarrow M10 \rightarrow M24 \rightarrow M11 \rightarrow M12 \rightarrow M13 \rightarrow M09$, den die (nun korrigierte) Azyklizitätsinvariante verbietet.
- (19) Ein zweiter, verdeckter Zyklus $M11 \rightarrow M22 \rightarrow M11$ (P16 `SeamQuery`, P17 `CellReport`) wurde erst nach Behebung des ersten sichtbar. Anders gelagert: Pass C9 führt M11 und M22 bereits als gemeinsame Träger derselben Pässe – kein Rückfluss zwischen Passen, sondern Zusammenarbeit innerhalb einer

- Passe. Invariante *Abhängigkeitszyklizität* um eine entsprechende Ausnahme für gemeinsame Passträger (`pass_registry.yaml`) ergänzt, kein sechster benannter Kanal.
- (20) Kapitel 4s Portregister führte für keinen der 42 Ports die Art (`request` oder `event`); der Anhang gab sie nur für neun Beispieleinträge, mit einem irreführenden Verweis auf ein vollständiges Kapitel-4-Feld, das nie existierte. Für alle 42 Ports ergänzt, nach Regel *Synchronität* und den fünf benannten Rückflusskanälen: Ports auf Rückflusskanälen sowie P06 (echte Prozessgrenze, Beobachterprozess) sind `event`; P28/P33 bleiben `request` gemäß ihrer bereits bestehenden Sonderkennzeichnung „stets synchron“; alle übrigen sind `request`, entsprechend ihrer Rolle als Schritt innerhalb einer synchron fortschreitenden Passfolge oder Orchestrierung. P31/P32 (der auslösende Fund) sind damit `request` – derselbe Aufruf/Rückgabe-Charakter wie P00/P05 in `boot()`, nur zuvor nicht gekennzeichnet.
- (21) Das `MachineCertificate` konnte eine Konformanzklasse ausweisen, die von einer plattformgebundenen Verpflichtungsauflösung abhängt, ohne diese Bindung zu nennen – ein Zertifikat hätte damit mehr behauptet, als geprüft wurde. Neue Regel *Plattformgebundene Verpflichtungsauflösung im Zertifikat* bindet `scope` an `resolution_platform` und erklärt die Verpflichtung auf jeder Plattform ohne deklarierte Auflösung als offen.
- (22) I-ARCH-012 und R-RA-012 verwiesen auf ein *Replay-Profil*, das `ProfileId` (`readonly`, `shadow`, `sandbox`, `reference`) nicht führt – hängende Referenz derselben Klasse wie seinerzeit T-M13-001. Aufgelöst ohne fünften Profilwert: `shadow` leistet bereits wörtlich, was *Replay* verlangt. Neue Regel *Replay läuft unter shadow* hält das fest und begründet zugleich, warum ein Modusfeld auf dem `RunDescriptor` unzulässig wäre.
- (23) Die Challenge-Abschlussbedingung (Definition *Taktphasen*) verlangte „alle Kapseln im Fixpunkt“, ohne dass *Fixpunkt* irgendwo definiert war – der Begriff kam im gesamten Werk nur zweimal vor, beide Male als Verwendung. Als *Kapselfixpunkt* über die bereits bestehende Monotonie von `allowed_next` definiert, mit ausdrücklicher Trennung vom Budgetabbruch.
- (24) `TraceSegment` definierte `segment_digest` als $H(\text{Can}(\text{alle vorstehenden Felder}))$, obwohl `time: DualTime` darunter fällt — τ_e ging damit entgegen der blockierenden Invariante *Replayneutralität der Wanduhr* in jeden Kettendigest und über L_t in I_t ein. Folge: zwei inhaltsgleiche Läufe erhielten verschiedene Zustandsdigests, R2 war strukturell unerreichbar und kein Referenzrelease möglich. Aufgelöst nach demselben Muster wie auf Objektebene: `segment_digest` wendet jetzt π_{vol} an, das neue `segment_record_digest` trägt die vollständigen Bytes einschließlich τ_e , und `verify_chain` prüft beide. Nachgetragen in v1.0.48: das in derselben Regel geforderte Fernhalten von I_t und Gate-Entscheidungen war nur als Prosa formuliert, ohne Eintrag in `volatile_fields.yaml` — dessen `purpose` sich ausdrücklich als einziger Ausschlusspfad erklärt. Ohne den Eintrag hätte `segment_record_digest` τ_e unmittelbar wieder in I_t getragen. Die Aufnahme in die Gruppe `volatile` erfüllt zugleich die Gate-Hälfte der Regel, da `enforcement.gate_input_forbidden` diese Gruppe bereits nennt.
- (25) Vertrag *Prognosepersistenz* verlangte, dass eine Prognose Horizont, Erzeugungsbasis, Gültigkeitsfenster und den damaligen Anker speichert — kein Objekt trug diese Felder, es gab also nichts, das die Regel hätte tragen können. Als OBJ-FCT (Forecast) ergänzt, mit `append-only evaluations` und ohne Schreibpfad auf die übrigen Felder, sodass *bewerten statt umschreiben* strukturell statt konventionell gilt.
- (26) Invariante *Kantenvollständigkeit* verlangte nichtleere Kantenbedingungen, ohne zu sagen, woher sie stammen — ein Zusammenbau hätte sie erfinden oder generisch mit *wahr* besetzen müssen. Regel *Herkunft der Kantenbedingungen* ergänzt: sie sind `PredicateExpr` und damit domänengeliefert, je `RelationSortId` im `DomainProfile` deklariert; fehlt die Deklaration, entsteht keine Kante, sondern ein Residuum.
- (27) Drei Objektschemata führen Referenzfelder, deren Sortenpaar die Sorten-Port-Matrix nicht kennt, was den Verdacht einer unvollständigen Matrix nahelegte. Tatsächlich sind Referenzfeld und `IREdge` verschiedene Gegenstände — die Umkehrprobe zeigt es: eine Matrix, die Referenzfelder regelte, verböte `RealityClassification.thought_ref`, ein Pflichtfeld, das dieselbe Norm verlangt.

Regel *Objektreferenz ist keine IR-Kante* stellt die Trennung fest; die Matrix bleibt unverändert.

- (28) Regel *Ratchet-Terminierung* erzwingt die Terminierung „durch das Budget `ratchet_max_rounds` im `RunDescriptor`“ — ein Feld, das der `RunDescriptor` nicht führte; `nondeterminism_budget` ist für deklarierte Abweichungsquellen typisiert und trägt kein Rundenbudget. Hängende Referenz derselben Klasse wie seinerzeit T-M13-001. Feld ergänzt, dort wo die Regel es bereits verortet: ein Rundenbudget ist eine Ressourcenentscheidung über den Lauf, nicht über die Domäne, und wird von R2 über die Gleichheit des `RunDescriptor` ohnehin mitgeführt.
- (29) Das `freshness_predicate` des Referenzankers war stets wahr, womit der von Vertrag *Ankerfrische* vorgeschriebene Ausgang (HOLD und `ReanchorRequest` über P27) nie eintreten konnte — was zugleich erklärt, warum `ReanchorRequest` nirgends konstruiert wird und P27 tot ist. Regel *Ein Frischeprädikat muss verletzbar sein* ergänzt; für einen über ein Verzeichnis versiegelten Anker ist die Veränderung dieses Verzeichnisses die verlangte Fallbedingung.
- (30) Die Einfügung von *Zwei Digests je Segment* in v1.0.20 verschob alle nachfolgenden Blöcke von Kapitel 7, ohne dass die Verschiebung benannt oder nachgezogen wurde; Zitate im Code führten daraufhin sechs Editionen lang v1.0.19-Stände. Regel *Zitierform normativer Blöcke* ergänzt: Titel sind der Bezeichner, Nummern sind Bequemlichkeit, und wer einfügt, benennt die Kaskade.
- (31) `RevisionProposal` war Nutzlast von P36, Ausgabe der ISA-Instruktion `PROPOSE_REVISION` und Gegenstand von G-SELF-COMPILE, besass aber keine Feldstruktur — dieselbe Lückenklasse wie zuvor `ReanchorRequest` und `Forecast`. Als OBJ-RVP ergänzt, mit `hardening_class` über genau die sechs zulässigen Klassen, getrennter Führung vorhandener und fehlender CRA-Eingaben und `parent_binding` gegen die laufende Instanz.
- (32) `RevisionProposal` trug in v1.0.48 ein Feld `gate_report_ref`, das erst nach der Gate-Auswertung gesetzt werden sollte — eine Mutation eines bestehenden Objekts und damit ein Verstoss gegen die Invariante *Kein In-place-Rückfluss*. Feld entfernt, Regel *Urteil verweist, Gegenstand nicht* ergänzt: die Zuordnung trägt der `GateReport` über seine Eingabedigests, wie M07 sie über die `RealityClassification` trägt.
- (33) C5 verlangte eine unabhängige Reproduktion, ohne zu sagen, was diese belegen muss — die Klasse blieb damit unbestimmt und praktisch unerreichbar. Regel *Was eine unabhängige Reproduktion bezeugt* ergänzt fünf nachprüfbar Punkte, darunter die ausdrückliche Pflicht, den Deckungsvektor selbst abzuleiten statt ihn zu übernehmen.
- (34) Das Topologieregister verwies für die Skalentiefe auf das Feld `max_depth` im `RuntimeManifest`; dieser führte kein solches Feld. Hängende Referenz derselben Klasse wie zuvor `ratchet_max_rounds`. Feld ergänzt — ohne es ist der Abstiegsoperator `descend` und damit jede Skalenverschachtelung unimplementierbar, und die Wohlgeformtheitsbedingung `level ≤ max_depth` einer M13Address nicht prüfbar.
- (35) Definition *Zulässige Rückflüsse* sagte „genau vier“ und zählte fünf — die Zahl wurde beim Einfügen des Sondierungsrückflusses P39 in v1.0.14 nicht mitgezogen. Korrigiert. Dieselbe Driftklasse wie die Objektzahlen, nur innerhalb eines Satzes.
- (36) Φ und der Transport $T_{l_a l_b}$ wurden zehnmal verwendet und nie definiert, obwohl `close720` — und damit EXECUTABLE — an $\text{Hol}(\Phi^2) = I$ hängt. Aus der bereits vorhandenen Kennzeichnung als *akkumulierte Rahmenänderung* abgeleitet: Transport ist reiner Rahmenwechsel, inhaltserhaltend, umkehrbar, und $T_{ll} = I$. Φ ist T_γ der geschlossenen Route. Bei `max_depth = 0` folgt $\Phi = I$ daraus, statt angenommen zu werden — mit Ausweispflicht als *trivial geschlossen*.
- (37) Vertrag *Zellclosure* ließ offen, was eine *leere Zelle* ist; im Referenzgraphen sind 7 von 18 Zellen unbelegt. Geregelt: leere Zellen schließen vakuum, sonst hinge EXECUTABLE an der Graphgröße statt an der Korrektheit — aber `CellReport` führt `occupancy`, und `close_all_18` berichtet die Zahl der vakuumen Schließungen.
- (38) *Richtung konsistent* war in Vertrag *Zellclosure* Prüfpflicht, aber nirgends definiert. Als Erbe der Schichtordnung des Modulgraphen geregelt, ohne eigene Ordnung einzuführen.
- (39) `CellReport` besaß kein Feld für die von der Platzierungsregel verlangten Sondierungsvermerke,

und die in v1.0.48 ergänzte Fassung konnte ihre eigenen Ausweispflichten nicht tragen: weder *trivial geschlossen* noch die Vakuumwahrheit über einer leeren Verweisliste hatten ein Feld. Ergänzt um `closure_mode` und `refs_resolution`, dazu `ProbeNote` als eigene Struktur — sie wurde verwendet, ohne je definiert zu sein.

- (40) `descend` musste eine vollständige `M13Address` liefern, ohne dass ein Block die Eintrittszelle im Feinchart bestimmte. Geregelt ohne neue Regel: es gilt dieselbe Platzierungsregel wie auf jeder anderen Skalenstufe. Dabei fiel auf, dass die Schnittstelle `descend(addr)` diese Regel nicht erfüllen kann — die Platzierung braucht den Knoten, nicht nur die Adresse. Signatur auf `descend(addr, node)` korrigiert; eine Regel, die ihre eigene Schnittstelle unmöglich macht, ist keine.
- (41) Der Golden-Run-Ablauf beschrieb dreizehn Schritte, ohne den Taktzyklus zu nennen — neben Definition *Taktphasen* stand damit eine zweite Beschreibung desselben Vorgangs, und der zertifizierende Lauf benutzte die, die kein Ausführungsmodell ist. Das ausgewiesene I_t bezeugte einen Pfad, den das Werk nirgends definiert. Regel *Der Golden Run läuft unter tick* ergänzt, mit Zuordnung der dreizehn Schritte zu den zwölf Phasen.
- (42) Der `EffectAdapter` lieferte Vorzustand und Versuch, ohne dass geregelt war, dass beide denselben Beobachtungskanal benutzen und den Effekt klammern müssen — über eine Prozessgrenze wären das sonst zwei Beobachtungen mit einem Fenster dazwischen. Regel *Vorzustand und Versuch klammern den Effekt* ergänzt.
- (43) Struktur *Gedankenkörper* verlangt `anchor_refs` oder `unanchored` bei der Konstruktion, während die Passfolge den Anker erst in C3 bindet — eine Abhängigkeit, die keine Regel benannte. Aufgelöst über die im Compilerlauf bereits geführte Unterscheidung: bis C2 läuft ein `candidate`, der `ThoughtBody` wird frühestens in C3 geprägt. Regel *Kandidat und Gedankenkörper* ergänzt, samt dem ausdrücklichen Verbot einer nachträglichen Ankeranheftung.
- (44) Die Modulspalte der Definition *Taktphasen* nannte für `Anchor` nicht M06 und für `Compile` nicht M11, obwohl die Prägung des `ThoughtBody` nach Regel *Kandidat und Gedankenkörper* in der Ankerphase liegt und der Kandidatenbau verkleben muss. Beide ergänzt.
- (45) Die Signatur `tick(state, rd)` ließ offen, ob eine Implementierung Zugriffsmittel wie Effektleitungen, Wanduhrquelle oder Profilingschalter zusätzlich entgegennehmen darf. Geregelt: ja, sofern ohne Planungsautorität, außerhalb von Σ und am Aufrufpunkt begründet.
- (46) Sieben Felder des `MachineCertificate` trugen Konstanten der Form `H(Feldname)` — die drei Identitäten aus dem Boot und die vier Berichtsdigests — während die echten Werte am Aufrufpunkt bereitlagen. Dieselbe Erfindungsklasse, die zuvor an I_t allein geschlossen wurde; die Nachbarn blieben ungeprüft. Regel *Jedes Zertifikatsfeld ist abgeleitet* ergänzt, mit feldweiser Nachweispflicht statt einer Prüfung am Ganzen.
- (47) Die Ableitungspflicht galt auch für `AdditionalAcceptance`, das über die Klassenberechnung ins Zertifikat wirkt: `kernel_executable` stand auf wahr, während derselbe Lauf `EXECUTABLE` als unerreichbar maß. Folgenlos, weil eine zweite Lücke danebenlag — zwei Fehler, die einander decken, sind kein Zustand.
- (48) Die Ausstellungsreihenfolge war ungeregelt: ein vor dem Selbstkompilationsschritt ausgestelltes Zertifikat kann FC7 nicht ableiten. Regel *Das Zertifikat ist das letzte Artefakt des Laufes* ergänzt.
- (49) Ob ein leeres `signature`-Feld die Ausstellung blockiert, war offen. Geregelt: zulässig unterhalb C4, weil Vertrag *Selbstgültigkeit* die Signatur für die Selbstgültigkeit verlangt und OBL-005 sie erst ab C4 blockierend führt — mit Pflicht zum erklärten Nullstand und einem Nachweis, der bei C4 fällt.
- (50) Dasselbe Muster trat danach zweimal weiter auf (`calibration_ref` ohne registrierten Katalog, `reproducible` aus einem einzelnen Lauf). Als Regel *Erklärter Nullstand* verallgemeinert, samt dem ausdrücklichen Verbot der Ersatzfüllung: eine Null an der Stelle einer fehlenden Quelle ist von einer gemessenen Null nicht zu unterscheiden.
- (51) `RankEff` lieferte über leerer Evidenzmenge den Bodenwert eins, der von einem berechneten Rang äußerlich nicht zu unterscheiden gewesen wäre. Die Stelle hatte keinen Produktivaufrufer und hat nie berichtet — der Boden hätte aber beim ersten Lauf mit echter Evidenz zum Vergleichsmaßstab werden

können. Regel *Ein Bodenwert ist keine Messung* ergänzt: die Unterscheidung gehört in einen eigenen Ausgang, nicht in einen Zahlenwert.

- (52) Präzedenz zwischen Anforderungen ist *quellenrelativ*. Zwei Rangzahlen aus verschiedenen Dokumenten stammen von verschiedenen Autoren; sie zu vergleichen erfände eine Ordnung, die niemand gesetzt hat. Ein quelleninterner Präzedenzstreit ist damit entscheidbar, ein quellenübergreifender nicht — letzterer ist eine dritte Geltungsart und gehört als Obstruktion vor den Integrator. Eine Anforderung, die ihren Streit im eigenen Dokument gewinnt, bleibt von aussen bestritten.
- (53) `capsulate` bildete `allowed_next` und die Gegenmodelle aus Laufwerten statt aus der übergebenen Quotientenklasse; zwei Klassen ergaben deshalb notwendig dieselbe Kapsel. Regel *Eine Kapsel ist Funktion ihrer Klasse* ergänzt, samt der Unterscheidung zwischen gemessener Konvergenz und einer Funktion, die ihr Argument nicht liest — beides sieht im Bericht gleich aus.
- (54) Die Replayeinstufung sah den `RunDescriptor` nicht und nahm nur Wahrheitswerte über Übereinstimmung entgegen; zwei Läufe über verschiedene Korpusfassungen ergaben dadurch eine Divergenzmeldung statt der Feststellung, dass sie kein Replaypaar bilden. Regel *Verschiedene Eingaben bilden kein Replaypaar* ergänzt, mit `NOT_COMPARABLE` als eigenem Ausgang.
- (55) `AnchorSnapshot` trug kein Feld für den Git-Commit, obwohl Regel *Anker der Referenzdomäne* verlangt, dass er ihn bindet. Der Commit lag auf dem `ExternalRecord`, das ausdrücklich kein kanonisches Objekt ist, und wurde beim Binden verworfen. Feld `source_revision` ergänzt.
- (56) Regel *Zitierform normativer Blöcke* regelte die Zitierung, nicht das Umschreiben: ein Migrationswerkzeug ohne Namensraumkenntnis wandte eine für ein Werk richtige Verschiebung auf gleich-numerierten Zitierungen des anderen an, und die nachgelagerte Differenzprüfung sah es nicht — sie prüft das Ergebnis, der Fehler lag in der Auswahl. Ergänzt.
- (57) Die Phasentabelle wies `I7` das Work Package `WP13` zu, obwohl `WP13` namentlich *Reconciliation* (`M18`) ist und `I7s` Exit-Kriterium *Golden Run und gesamte Pflichtsuite* verlangt — das ist `WP15`. Zugleich fehlte `WP13` in `I6s` Work-Package-Liste, obwohl `I6s` eigenes Exit-Kriterium *Reconciliation* ausdrücklich nennt, die Regel *Reihenfolgezwang* `WP13` unmittelbar nach `WP12` verlangt, und der *Minimale geschlossene Kreis* in *Reconciliation* endet. `I6` trägt jetzt `WP11`, `WP12`, `WP13`; `I7` trägt `WP15`. `I6` ist damit vor Abschluss von `WP13` nicht vollständig.
- (58) Die Tabelle der sechs Vertrauensdomänen führte `M15` und `M18` in keiner Domäne und `M08–M10` zugleich in `Kernel` (per Bereichsschreibweise `M05–M14`) und in `FieldRuntime`. `M15` und `M18` nach `Kernel` aufgenommen, `M17` nach `Audit`, `Kernels` Bereich auf `M05–M07` und `M11–M15` umgeschrieben, sodass `M08–M10` ausschließlich `FieldRuntime` zufällt.
- (59) `ExternalReceipt` überquert bei `P24` eine echte Prozessgrenze, für die kein typgeschützter Mechanismus wie bei `GateAuthorization` greift, da Serialisierung unumgänglich ist. Vertrag *Herkunftsbeglaubigung an der Prozessgrenze* ergänzt: die `P24`-Ingress-Implementierung MUSS die Betriebssystemidentität des sendenden Prozesses vor der Deserialisierung prüfen, nicht erst über Felder des bereits deserialisierten Objekts.

Keine weitere normative Aussage geändert. Sechs Speicherklassen, 37 Tests, 28 Module, 19 Sorten unverändert.

Inhaltsverzeichnis

Kanonische Zielidentität	3
Artefaktrang	3
Abschlussbedingung	3
Statusdisziplin und Leseordnung	5
Normative Sprache	5
Identifikatorordnung	5
Fehlerkorrektur v1.0.48	7
I Gesamtsystemarchitektur	19
1 Geltungsbereich und Konformitätsgegenstand	21
1.1 Gegenstand	21
1.2 Nicht-Gegenstand	21
1.3 Präzedenzordnung	21
1.4 Architekturidentität	22
2 Architekturschichten	23
2.1 Schichtenordnung	23
2.2 Abbildung auf die kanonische Schichtenordnung	23
2.3 Ausführungsrichtung und Rückflüsse	24
3 Modulkarte	25
3.1 Kanonische Modulmenge	25
3.2 Modulverantwortungen im Detail	26
3.3 Gegenmodelle	27
4 Portvertrag und internes Kommunikationsprotokoll	29
4.1 Nachrichtenhülle	29
4.2 Portregister	29
4.3 Zustellsemantik	30
II Typsystem und Datenstrukturen	31
5 Sortenordnung	33
5.1 Primitive Sorten	33

5.2	Kernproduktionen	33
5.3	Realitäts- und Faktizitätsordnung	34
6	Kanonisierung, Identität und Digest	37
6.1	Kanonisierungsfunktion	37
6.2	Digest und Identität	37
6.3	Zeitmodell	38
7	Kanonische Objekte	39
7.1	Konstitutions- und Identitätsobjekte	39
7.2	Ankerobjekte	39
7.3	Kognitionsobjekte	40
7.4	Repräsentations- und Prüfobjekte	42
7.5	Gate-, Wirkungs- und Reconciliationsobjekte	45
7.6	Aufzeichnungsobjekte	46
8	Objektlebenszyklen, Ownership und Lebensdauer	51
8.1	Lebensdauerklassen	51
8.2	Allokations- und Freigaberegeln	51
III	Syntaxskelett und Zwischenrepräsentation	53
9	Topologiedienst M_{13}	55
9.1	Graphstruktur	55
9.2	Adressraum	56
9.3	Closure erster und zweiter Ordnung	57
9.4	Skalenverschachtelung	58
10	Zwischenrepräsentation	59
10.1	Zielidentität der IR	59
10.2	Kanonische Serialisierung	59
10.3	Portverträge der IR-Kanten	59
IV	Compilerarchitektur	61
11	Passfolge	63
11.1	Normative Passfolge	63
11.2	Passverträge	63

12 Adversarialer Kanonisierungskern	67
12.1 Kernstruktur	67
12.2 Surface-Invariant-Split	67
12.3 Ratchet	67
12.4 Residuenfluss	67
12.5 Self-Hardening	68
12.6 Adversariale Closure	69
 V Laufzeitmodell	 71
13 Laufzeitzustände und Automatenfamilie	73
13.1 Maschinenzustand	73
13.2 Objektautomat	73
13.3 Gedankenautomat	73
13.4 Effektautomat	74
13.5 Feldautomat	75
13.6 Kapsel- und Tokenautomat	75
 14 Scheduler und Ausführungszyklus	 77
14.1 Taktfolge	77
14.2 Prioritätsordnung	78
14.3 Nebenläufigkeit	78
14.4 Ressourcen- und Risikobudgets	78
 15 Native Instruktionsmenge	 81
 16 Speicherarchitektur	 83
16.1 Speicherklassen	83
16.2 Gedächtnisflächen	83
16.3 Physisches Layout	83
 17 Boot, Shutdown und Recovery	 85
17.1 Bootfolge	85
17.2 Shutdown	85
17.3 Recovery	85
 VI Gate-, Capability- und Effektsystem	 87
 18 Gatesystem	 89
18.1 Gatesemantik	89

18.2 Gatefamilie	89
19 Vertrauensgrenzen und Capabilities	91
19.1 Vertrauensdomänen	91
19.2 Capabilitymatrix	91
20 Effektgrenze, Receipts und Reconciliation	93
20.1 Effektgrenze	93
20.2 Reconciliation	94
20.3 Prognosepersistenz	94
VII Verifikation, Replay und Konformanz	97
21 Witness- und Validierungsmodell	99
21.1 Witnessengine	99
21.2 Validierungsplanung	99
22 Trace, Replay und Determinismus	101
22.1 Replayklassen	101
23 Konformanzmodell	103
23.1 Kanonisierung der Konformanzordnung	103
23.2 Konformitätsklassen	103
23.3 Verfeinerungssicherheit	104
24 Testarchitektur	105
24.1 Testklassen	105
24.2 Golden Run	105
24.3 Empirische Baselines	106
VIII Artefakte, Repository und Deployment	107
25 Artefaktstruktur	109
25.1 Doppelprojektion	109
26 Repository und Paketstruktur	111
26.1 Kanonisches Repository	111
26.2 Build und Reproduzierbarkeit	111
26.3 Versionierung	112

27 Plug-in-Modell und API-Grenzen	113
27.1 Plug-in-Klassen	113
27.2 API-Grenzen	114
28 Deployment	115
 IX Fehlerarchitektur, Beobachtung und Zertifizierung	 117
29 Fehlerarchitektur	119
29.1 Fehlerdomänen	119
29.2 Fehlerreaktionsmatrix	119
30 Beobachtung: Logging, Profiling, Debugging	121
30.1 Autoritätsfreiheit	121
30.2 Logging	121
30.3 Profiling	121
30.4 Debugging	121
31 Zertifizierung, Release und Selbstkompilation	123
31.1 Release-Gate	123
31.2 Selbstkompilation	123
31.3 Maschinenakzeptanzformel	124
 X Realisierung	 125
32 Deterministischer Implementierungsplan	127
32.1 Kanonisierung der Phasenordnung	127
32.2 Minimaler Kernel Stack	128
32.3 Referenzdomäne	128
33 Offene Implementierungsverpflichtungen	131
34 Phasenvertrag	133
 XI Normative Register	 135
A M₁₃-Topologieregister	139
B Sorten- und Typregister	141
C Modul- und Portregister	143

D Automatenregister	145
E Gate- und Passregister	147
F Invariantenregister	149
G Fehlerkatalog	151
H Requirement-Matrix	153
I Proof-Obligation-Register	155
J Konformanz- und Negativtestregister	157
K Instruktionsregister	159
L Symbol- und Abkürzungsregister	161
Abschlussvermerk	163

TEIL I

Gesamtsystemarchitektur

Geltungsbereich und Konformitätsgegenstand

1.1 Gegenstand

Diese Norm definiert die vollständige Softwarearchitektur einer Referenzmaschine $\mathcal{M}_{\text{ref}}$, die die kanonische PSK-Maschine $M_{\text{PSK}} = (C, G, M_{13}, \text{IR}, K, V, U, E, Z)$ konservativ verfeinert.

Definition 1.1 (Architekturgegenstand). Der Architekturgegenstand ist das Tupel

$$A_{\text{RA}} = (\Lambda, \mathcal{M}, \mathcal{P}, \mathcal{T}, \mathcal{F}, \mathcal{O}, \mathcal{S}, \mathcal{E}, \mathcal{K}, \mathcal{R}),$$

mit Schichtenordnung Λ , Modulmenge $\mathcal{M}$, Portmenge $\mathcal{P}$, Typregister $\mathcal{T}$, Automatenfamilie $\mathcal{F}$, Instruktionsmenge $\mathcal{O}$, Speicherarchitektur $\mathcal{S}$, Fehlerordnung $\mathcal{E}$, Konformanzordnung $\mathcal{K}$ und Repositorystruktur $\mathcal{R}$.

Definition 1.2 (Architekturkonforme Implementierung). Eine Implementierung $\mathcal{I}$ ist architekturkonform, wenn eine totale Abbildung $h_{\text{RA}} : \text{Comp}(\mathcal{I}) \rightarrow \mathcal{M}$ existiert, die

- (a) jede Codeeinheit genau einem Modul zuordnet,
- (b) jede modulübergreifende Aufrufkante genau einem Port zuordnet,
- (c) jeden konkreten Zustandsautomaten vorwärts auf einen Automaten aus $\mathcal{F}$ simuliert,
- (d) jeden erzeugten Fehler auf genau einen Code aus $\mathcal{E}$ abbildet,
- (e) jedes persistierte Objekt auf genau eine Speicherklasse aus $\mathcal{S}$ abbildet.

Unabgebildete Codeeinheiten, Kanten, Transitionen, Fehler oder Objekte sind blocking.

1.2 Nicht-Gegenstand

Diese Norm schreibt keine Programmiersprache, kein Modellformat, keine Hardwareplattform, keine Netzwerktopologie und kein Betriebssystem vor. Sie schreibt Modulgrenzen, Ports, Typen, Automaten, Determinismusbedingungen, Artefakte und Nachweise vor.

Invariante 1.3 (Sprachneutralität mit Erzwingungspflicht). Die Wahl der Implementierungssprache ist frei. Der normative Kern DARF NICHT ausschließlich als Systemprompt, Modellanweisung oder Konvention existieren. Typen, Gates, Tokens, Trace, Replay und Reconciliation MUSS technisch außerhalb eines Sprachmodells erzwingen werden.

1.3 Präzedenzordnung

Regel 1.4 (Normative Präzedenz). Bei Widerspruch gilt die folgende, streng absteigende Ordnung:

1. `constitution.lock.json` und die darin gebundenen Digests;
2. Kanonische Gesamtspezifikation v1.0;
3. Konstitutioneller Maschinenvertrag;
4. das vorliegende Werk (PSK-RA v1.0.48);
5. die maschinenlesbaren Register der Anhänge dieses Werkes;
6. Implementierungsentscheidungen innerhalb des verbleibenden zulässigen Raums.

Ein Widerspruch zwischen Rang 2 oder 3 und Rang 4 ist ein Defekt dieses Werkes und erzeugt FAIL des Architektur-Gates.

Regel 1.5 (Registerprimat). Wo Fließtext und maschinenlesbares Register dieses Werkes denselben Sachverhalt beschreiben, ist für die Implementierung das Register maßgeblich. Die Register befinden sich in den Anhängen [A–L](#) und sind unverändert in das Contract-Bundle zu übernehmen.

1.4 Architekturidentität

Definition 1.6 (Architektur-ID).

$$I_A = H(\text{Can}(\text{ra_registers})), \quad \text{Can}(f) = \text{JSONCompact}(\text{SortKeys}(\text{Parse}(f))).$$

Die Kanonisierung parst YAML/JSON, erzwingt UTF-8, sortiert Objektschlüssel, erhält Arrayreihenfolgen und serialisiert kompakt. H ist SHA-256.

Invariante 1.7 (Vier Identitätsschichten). *Konstitutions-ID I_C , Architektur-ID I_A , Implementierungs-ID I_M und Laufzeit-ID I_L sind vier getrennte Größen. Keine DARF eine andere ersetzen. Das Maschinenzertifikat MUSS alle vier binden.*

FESTLEGUNG

Der Architekturgegenstand liegt in genau zwei synchronen Projektionen vor: diesem PDF und dem Registerverzeichnis `architecture/` des Contract-Bundles. Bei Feldwertkonflikt gilt das kanonisch geparste Register; bei Zielidentitätskonflikt ist die Architektur defekt und das Release-Gate bleibt geschlossen.

Architekturschichten

2.1 Schichtenordnung

Definition 2.1 (Acht Architekturschichten). Die Referenzmaschine besitzt die gerichtete, nicht kollabierbare Schichtenfolge

$$L_0 \rightarrow L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow L_4 \rightarrow L_5 \rightarrow L_6 \rightarrow L_7.$$

Schicht	Name	Normative Verantwortung	Module
L_0	Substrat	Prozess, Dateisystem, Uhr, Zufallsquelle, Kryptoprimitive, Sandbox.	M00, M25, M26
L_1	Konstitution	Bundle laden, kanonisieren, Digests, Identitätsbindung, Bootgate.	M00–M04
L_2	Anker	Außenrecords versiegeln, Provenienz, Frische, Unsicherheit.	M05, M17
L_3	Grammatik und Syntax	Sorten, Produktionen, Realitäts- und Faktizitätstypen, M_{13} -Platzierung.	M06, M07, M22
L_4	IR und Compiler	Kanonische Zwischenrepräsentation, Passfolge C1–C11.	M08–M11, M23, M24
L_5	Verifikation	Witness, Validierungsplan, Gates, Closure, Konsens.	M12–M14
L_6	Wirkung	Capabilities, Tokens, Effektgrenze, Receipts, Reconciliation.	M15–M18
L_7	Aufzeichnung	Trace, Replay, Residuen, Morphogenese, Zertifikat, Archiv.	M19–M21, M27

Invariante 2.2 (Schichtentrennung). Keine Schicht L_j DARF eine Schicht L_i mit $i < j$ rückwirkend durch ihr Ergebnis definieren. Insbesondere erzeugt eine erfolgreiche Ausführung keine neue Grammatikregel, ein Laufzeit-Claim keine konstitutionelle Identität und ein Archiveintrag keine Faktipromotion.

Invariante 2.3 (Abhängigkeitszyklizität). Der Modulabhängigkeitsgraph ist azyklisch. Eine Kante $\mathbf{M}_a \rightarrow \mathbf{M}_b$ ist nur zulässig, wenn $L(\mathbf{M}_b) \geq L(\mathbf{M}_a)$ – die Pipeline schreitet zu gleichem oder höherem Layer fort – oder wenn sie einer der in Definition Zulässige Rückflüsse benannten Rückflusskanten entspricht, oder wenn $\mathbf{M}_a$ und $\mathbf{M}_b$ laut `pass_registry.yaml` gemeinsame Träger derselben Passe sind (Zusammenarbeit innerhalb einer Passe, kein Rückfluss zwischen Passen). Jede andere rückwärtsgerichtete Kante ($L(\mathbf{M}_b) < L(\mathbf{M}_a)$) erfolgt ausschließlich als Ereignis über M19 und niemals als direkter Aufruf.

2.2 Abbildung auf die kanonische Schichtenordnung

Regel 2.4 (Kanonische Schichtenbindung). Die kanonische Sechserordnung Grammatik $\rightarrow$ Syntax $\rightarrow$ IR $\rightarrow$ Compiler $\rightarrow$ Runtime $\rightarrow$ Außenreconciliation bindet auf L_3 (Grammatik, Syntax), L_4 (IR, Compiler), L_5+L_6 (Runtime) und L_6 (Außenreconciliation). L_0 – L_2 und L_7 sind Trägerschichten ohne eigene semantische Autorität.

Invariante 2.5 (Keine Trägerautorität). Substrat, Anker- und Aufzeichnungsschicht DARF NICHT Faktizität, Geltung, Autorität oder Fähigkeit erzeugen. Sie erzeugen ausschließlich Bindung, Sichtbarkeit und Reproduzierbarkeit.

2.3 Ausführungsrichtung und Rückflüsse

Definition 2.6 (Vorwärtsfluss). Der normative Datenfluss lautet

Intake $\rightarrow$ Canonicalization $\rightarrow$ Anchor $\rightarrow$ Grammar $\rightarrow M_{13} \rightarrow$ IR $\rightarrow$ Compiler $\rightarrow$ Verification
 $\rightarrow$ Scheduler $\rightarrow$ Capability $\rightarrow$ Effect $\rightarrow$ Receipt $\rightarrow$ Reconciliation $\rightarrow$ Archive.

Definition 2.7 (Zulässige Rückflüsse). Es existieren genau fünf Rückflusskanten:

- (i) *Residuenrückfluss*: jede Schicht $\rightarrow M_{19}$ (Residue-/Traceschreibung).
- (ii) *Reanalyserückfluss*: $M_{18} \rightarrow M_{06}$ als *neuer* Kompilationsauftrag mit neuer Objektidentität.
- (iii) *Reankrierung*: $M_{18}/M_{13} \rightarrow M_{05}$ als Anforderung eines neuen AnchorSnapshot.
- (iv) *Revisionsvorschlag*: $M_{24} \rightarrow M_{21}$ als RevisionProposal, niemals als In-place-Änderung.
- (v) *Sondierungsrückfluss*: $M_{13} \rightarrow M_{09}$ als neue ProbeRequest (P39). Erzeugt eine neue FieldProjection mit neuer Objektidentität; niemals ein direkter Aufruf in eine bestehende Projektion hinein.

Jede weitere Rückflusskante ist ein Architekturdefekt.

Invariante 2.8 (Kein In-place-Rückfluss). Ein Rückfluss DARF NICHT ein bestehendes Objekt mutieren. Er erzeugt stets ein neues Objekt mit neuer Identität, neuer Lineage und Verweis auf das Ursprungsobjekt.

Modulkarte

3.1 Kanonische Modulmenge

Definition 3.1 (Modulmenge). $\mathcal{M} = \{M00, \dots, M27\}$. Die Menge ist abgeschlossen. Eine Implementierung DARF NICHT ein weiteres Modul mit normativer Verantwortung einführen; sie DARF Module intern in Untereinheiten zerlegen, sofern die Portgrenze erhalten bleibt.

ID	Modul	Normative Verantwortung	Schicht
M00	Constitution Loader	Lock laden, Bundle auflösen, Bootgate vorbereiten.	L_1
M01	Canonicalizer	YAML/JSON kanonisieren, Digests berechnen, Idempotenz sichern.	L_1
M02	Artifact Registry	Versionierte Artefakte, Rollen und Normativität auflösen.	L_1
M03	Schema Validator	Typ- und Schemaverträge an allen Grenzen erzwingen.	L_1
M04	Identity Binder	Konstitutions-, Architektur-, Implementierungs- und Laufzeit-ID koppeln.	L_1
M05	Anchor Registry	Außenrecords in bezugte, frische <code>AnchorSnapshots</code> überführen.	L_2
M06	Thought Compiler	Spezifikation in <code>ThoughtBody</code> und Ereigniskörper kompilieren.	L_3
M07	Reality Typer	Kohärenz, Gesetzlichkeit, Erreichbarkeit, Konstruierbarkeit und Aktualisierung klassifizieren.	L_3
M08	Field Registry	Feldidentitäten, Lineage, Scope und Lebenszyklus führen.	L_4
M09	Spectral Lens Router	Linsen konfigurieren und lokale Projektionen bereitstellen.	L_4
M10	Dependency Analyzer	Geteilte Daten-, Modell-, Sonden- und Kalibrierungsquellen quotieren.	L_4
M11	Closure/Glue Engine	Seams, 360- und 720-Closure, Verklebung, sichtbare Obstruktionen.	L_4
M12	Witness Engine	Evidence prüfen, Proof Objects binden, Okklusion typisieren, Status berichten.	L_5
M13	Validation Planner	Nächsten informationsgewinnenden Prüf- oder Messschritt wählen.	L_5
M14	Authority/Consequence Gate	Scope, Risiko, Folgen, Autorität und Ressourcen prüfen; Gates aller Ordnungen berechnen.	L_5
M15	Effect Token Service	Plan-gebundene, ablaufende, idempotente Tokens ausstellen.	L_6
M16	Effect Boundary	Nur tokenisierte Effekte in den zulässigen Substratbereich ausführen.	L_6
M17	External Record Ingress	Außenrecords unvermischt aufnehmen und provenienzbinden.	L_6
M18	Reconciliation Engine	Plan, Versuch und Außenrecord vergleichen; Faktpromotion entscheiden.	L_6
M19	Trace/Replay/Residue Store	Append-only Trace, Residuenledger, Run Descriptor und Replay führen.	L_7
M20	Morphogenesis Controller	Spawn, Split, Merge, Fold, Retire und Excise gaten.	L_7
M21	Certificate/Release Engine	Konformitätszertifikate, Releaseposture, Revisionsaktivierung und Export erzeugen.	L_7
M22	M ₁₃ Topology Service	Exakte 13/30/18-Topologie, Zelliteratoren, Adressraum, Skalenverschachtelung.	L_3
M23	IR Codec	Kanonische Serialisierung, Round-Trip, Knoten-/Kantenverträge, IR-Digest.	L_4
M24	Adversarial Kernel	Kapseln, Surface-Invariant-Split, Ratchet, Support, Okklusion, Residuenfluss, CRA, Self-Hardening.	L_4
M25	Scheduler	Taktfolge, Prioritätsordnung, Determinismushülle, Ressourcenbudgets.	L_0
M26	Lifecycle Supervisor	Boot, Shutdown, Recovery, Quarantäne, Freeze, Rollback.	L_0
M27	Observability Projector	Autoritätsfreie Sichten: Logging, Profiling, Debugging, Inspect.	L_7

Invariante 3.2 (Ein Verantwortungszentrum je Pflicht). Jede blocking Anforderung MUSS genau ein primär verantwortliches Modul und mindestens einen davon unabhängigen Prüfer besitzen. Geteilte Verantwortung ohne Primärmodul ist nicht zulässig.

Invariante 3.3 (Autoritätsfreiheit der Beobachtung). M₂₇ DARF NICHT schreibenden Zugriff auf M₁₉ oder auf einen Zustandsspeicher besitzen. Es liest ausschließlich abgeschlossene Trace-, Residuen- und Zertifikatsartefakte. Ein Ausfall von M₂₇ DARF NICHT den Kern beeinflussen.

3.2 Modulverantwortungen im Detail

Für jedes Modul gelten die folgenden vier verpflichtenden Angaben: *Ownership* (welche Objekte das Modul exklusiv erzeugt und schreibt), *Lesezugriff*, *Fehlerdomänen* und *Capabilities*.

ID	Exklusives Ownership	Lesezugriff	Fehlerdomänen	Capabilities
M00	BundleImage, BootReport	Dateisystem (Bundlepfad)	ID, SCHEMA	fs.read.bundle
M01	CanonicalBytes, Digest	BundleImage	ID	—
M02	ArtifactRecord	CanonicalBytes	ID, SCHEMA	—
M03	SchemaReport	Schemas, alle Grenzobjekte	SCHEMA	—
M04	IdentityBinding	Digest, Buildmetadaten, TraceHead	ID	—
M05	AnchorSnapshot	ExternalRecord	ANCHOR	fs.read.workspace
M06	ThoughtBody	AnchorSnapshot, Grammatik	SCHEMA, FACT	—
M07	RealityClassification	ThoughtBody, Modellregister	REAL	—
M08	FieldIdentity	ThoughtBody, FieldPolicy	FIELD	—
M09	FieldProjection	FieldIdentity, IR	FIELD	—
M10	DependencyProfile, RankReport	FieldProjection, Quellenregister	DEP	—
M11	SeamReport, ObstructionRecord	IR, FieldProjection	GLUE	—
M12	EvidenceObject, WitnessReport	AnchorSnapshot, ExternalReceipt	FACT	—
M13	ValidationPlan	WitnessReport, ResidueRecord	FACT	—
M14	GateReport	alle Eingangsobjekte der Gates	EFFECT, SCOPE	—
M15	EffectToken	GateReport, EffectPlan	EFFECT	token.issue
M16	EffectAttempt	EffectToken	EFFECT	adapterabhängig
M17	ExternalReceipt	Substrat (unabhängig)	RECON	fs.read.observer
M18	ReconciliationReport	Plan, Attempt, Receipt, Anker	RECON, FACT	—
M19	TraceSegment, ResidueRecord, RunDescriptor, ReplayManifest	— (append-only)	REPLAY, RES	store.append
M20	MorphogenesisDecision, ExcisionCertificate	FieldIdentity, Budgets	MORPH	—
M21	MachineCertificate, ReleaseReport	alle Berichte	ID	key.sign
M22	M13Chart, CellReport	Topologieregister	GLUE	—
M23	IRBundle	ThoughtBody, M13Chart, Projektionen	SCHEMA	—
M24	CandidateCapsule, RatchetRecord, RevisionProposal	IR, WitnessReport	FACT, PERSONA	—
M25	TickRecord, BudgetLedger	Warteschlangen, Budgets	REPLAY	clock.read
M26	LifecycleReport, RecoveryPlan	BootReport, TraceHead	ID, RECON	proc.control
M27	ViewArtifact	abgeschlossene Artefakte	—	store.read

Vertrag 3.4 (Ownership-Exklusivität). Ein Objekttyp besitzt genau ein erzeugendes Modul. Ein anderes Modul DARF ihn lesen, referenzieren und in ein neues Objekt einbetten, DARF NICHT ihn aber mutieren oder ein Exemplar desselben Typs erzeugen.

3.3 Gegenmodelle

Invariante 3.5 (Verbotene Modulkopplungen). Die folgenden Kanten sind Architekturdefekte und erzeugen *FAIL*:

- (a) $M16 \rightarrow M15$: die Effektgrenze stellt kein eigenes Token aus.
- (b) $M16 \rightarrow M17$: derselbe Adapter erzeugt nicht Effekt und Receipt.
- (c) $M18 \rightarrow M17$ schreibend: die Reconciliation überschreibt keinen Außenrecord.
- (d) $M08/M09 \rightarrow M18$: eine Feldidentität promoviert keinen Fakt.
- (e) $M24 \rightarrow M00/M01$: der adversariale Kern ersetzt keine aktive Verfassung.
- (f) beliebiges Modul $\rightarrow M19$ löschend oder überschreibend.

Portvertrag und internes Kommunikationsprotokoll

4.1 Nachrichtenhülle

Struktur 4.1 (Msg). Jede modulübergreifende Nachricht ist

```
Msg:
  msg_id: ULID # monoton, kollisionsfrei, prozessweit
  port_id: PortId # P00..P41
  type: MessageType # request | response | event | fault
  schema_id: SchemaId # namespaced + versioniert
  producer: ModuleId
  consumer: ModuleId
  run_id: RunId
  seq: uint64 # deterministische Sequenz je (run_id, port_id)
  input_digests: [Digest] # SHA-256 aller kausalen Eingaben
  created_at: DualTime # (tau_i, tau_e)
  trace_parent: TraceRef
  payload_digest: Digest
  payload: bytes # kanonisch serialisiert
  signature: Signature? # Pflicht an Vertrauensgrenzen
```

Vertrag 4.2 (Explizite Contract Surface). Eine Modulgrenze ist nicht konform, wenn sie Replay, Sicherheit, Governance, Faktizität oder Wirkung beeinflusst, aber kein maschinenlesbares Schema besitzt. Jede Grenze MUSS Eingabeschema, Ausgabeschema, Identitätsbindung, Fehlersemantik, Capabilityanforderung und Tracepflicht deklarieren.

Invariante 4.3 (Determinismus der Sequenz). Für festen `run_id` und festen `port_id` ist `seq` eine lückenlose, bei 0 beginnende Folge. Bei gleichem Run Descriptor und gleichen Eingabedigests MUSS die Folge der `(port_id, seq, payload_digest)`-Tripel identisch sein.

4.2 Portregister

Definition 4.4 (Port). Ein Port $P = (id, producer, consumer, \Sigma_{in}, \Sigma_{out}, \mathcal{E}_P, \mathcal{C}_P, trace)$ bindet Erzeuger, Verbraucher, Ein- und Ausgabeschema, zulässige Fehlerklassen, erforderliche Capabilities und Tracepflicht.

Port	Von	Nach	Nutzlast	Fehler	Art
P00	M26	M00	BootRequest	PSK-E101	request
P01	M00	M01	BundleImage	PSK-E102	request
P02	M01	M02	CanonicalBytes, Digest	PSK-E101	request
P03	M02	M03	ArtifactRecord	PSK-E001	request
P04	M03	M04	SchemaReport	PSK-E001	request
P05	M04	M26	IdentityBinding	PSK-E103	request
P06	M17	M05	ExternalRecord	PSK-E002	event
P07	M05	M06	AnchorSnapshot	PSK-E002	request
P08	M06	M07	ThoughtBody	PSK-E001	request
P09	M07	M23	RealityClassification	PSK-E004	request
P10	M22	M23	M13Chart	PSK-E011	request
P11	M23	M08	IRBundle	PSK-E001	request
P12	M08	M09	FieldIdentity	PSK-E013	request
P13	M09	M10	FieldProjection	PSK-E013	request
P14	M10	M24	DependencyProfile	PSK-E006	request
P15	M24	M11	CandidateCapsule	PSK-E003	request
P16	M11	M22	SeamQuery	PSK-E011	request
P17	M22	M11	CellReport	PSK-E011	request

Port	Von	Nach	Nutzlast	Fehler	Art
P18	M11	M12	ClosureReport	PSK-E011	request
P19	M12	M13	WitnessReport	PSK-E005	request
P20	M13	M14	ValidationPlan	PSK-E005	request
P21	M14	M15	GateReport (PASS)	PSK-E007	request
P22	M15	M16	EffectToken	PSK-E008	request
P23	M16	M19	EffectAttempt	PSK-E008	request
P24	M17	M18	ExternalReceipt	PSK-E009	event
P25	M18	M12	ReconciliationReport	PSK-E009	request
P26	M18	M06	ReanalysisRequest	PSK-E010	event
P27	M18	M05	ReanchorRequest	PSK-E002	event
P28	*	M19	TraceSegment, ResidueRecord	PSK-E014	request
P29	M19	M25	ReplayCursor	PSK-E015	event
P30	M25	*	TickGrant	PSK-E016	event
P31	M20	M08	MorphogenesisDecision	PSK-E017	request
P32	M08	M20	SpawnRequest	PSK-E017	request
P33	M14	M19	GateReport (any)	PSK-E014	request
P34	M19	M21	ReplayManifest	PSK-E015	request
P35	M21	M26	ReleaseReport	PSK-E018	request
P36	M24	M21	RevisionProposal	PSK-E012	event
P37	M26	M16	TokenInvalidate	PSK-E008	event
P38	M19	M27	ViewQuery (read-only)	—	request
P39	M13	M09	ProbeRequest	PSK-E005	event
P40	M20	M19	ExcisionCertificate	PSK-E014	request
P41	M26	M25	SchedulerControl	PSK-E016	request

Invariante 4.5 (Portabschluss). Die Portmenge ist abgeschlossen. Jede modulübergreifende Kommunikation, die keinem Port aus P00–P41 zugeordnet werden kann, ist ein Architekturdefekt und erzeugt FAIL im Konformanzlauf T-ARCH-001.

4.3 Zustellsemantik

Vertrag 4.6 (Zustellsemantik). Alle Ports besitzen genau-einmal-Semantik innerhalb eines Laufes. Sie wird realisiert durch: (i) Idempotenzschlüssel (run_id, port_id, seq); (ii) Persistenz der Nachricht in M19 vor der Verarbeitung; (iii) Wiederaufnahme nach Absturz ausschließlich über den persistierten Cursor. Nachrichten DARF NICHT verworfen werden; eine unzustellbare Nachricht erzeugt PSK-E014 und ein ResidueRecord.

Regel 4.7 (Synchronität). Ports mit type = request sind synchron und blockierend innerhalb desselben Ticks. Ports mit type = event sind asynchron und werden im nächsten Tick zugestellt. P28 und P33 sind stets synchron: eine Zustandsänderung DARF NICHT vor ihrer Traceschreibung sichtbar werden.

Invariante 4.8 (Trace vor Wirkung). Für jede Zustandsänderung δ gilt: der TraceSegment-Schreibvorgang für δ ist in der kanonischen Ordnung strikt vor der Sichtbarkeit von δ an irgendeinem Port abgeschlossen.

TEIL II

Typsystem und Datenstrukturen

Sortenordnung

5.1 Primitive Sorten

Definition 5.1 (Grammatik). $G_{PSK} = (\Sigma, \mathcal{T}, \mathcal{P}, \mathcal{I}, \mathcal{C}, \mathcal{R})$ mit atomaren Symbolen Σ , Sorten $\mathcal{T}$, Produktionen $\mathcal{P}$, Invarianten $\mathcal{I}$, Closure-Prädikaten $\mathcal{C}$ und Residuenordnung $\mathcal{R}$.

Sorte	ID	Normative Bedeutung	Owner
Identity	S-IDT	Kanonische Klasse, Lineage und Reflexionsgrenze eines Trägers.	M04
Anchor	S-ANC	Bezeugter, zeitgebundener Kontaktkörper zur Außenwelt.	M05
Context	S-CTX	Scope, Jurisdiktion, Ressourcen-, Zeit- und Modellrahmen.	M06
Event	S-EVT	Typisierte Zustandsänderung mit Vor- und Nachbedingungen.	M19
State	S-STA	Innerhalb eines Kontextes gültige Zustandsbeschreibung.	M23
Transition	S-TRN	Zulässiger gerichteter Übergang zwischen Zuständen.	M23
Projection	S-PRJ	Begrenzte Sicht auf einen gemeinsamen Gegenstand.	M09
FieldIdentity	S-FLD	Operative, nichtpersonale Perspektiv- und Operatorgrenze.	M08
Witness	S-WIT	Prüfbarer Träger einer Statuspromotion.	M12
Trace	S-TRC	Append-only Entstehungs-, Prüf- und Ausführungsspur.	M19
Residue	S-RES	Erhaltene Nichtclosure, Ungewissheit oder inkompatible Differenz.	M19
Gate	S-GAT	Partielle Entscheidungsfunktion mit Fail-Closed-Semantik.	M14
Capability	S-CAP	Tatsächlich verfügbare, scopegebundene Wirkungsmöglichkeit.	M15
Effect	S-EFF	Geplanter oder versuchter Eingriff in eine Außeninstanz.	M16
Receipt	S-RCP	Externer oder unabhängiger Record eines Wirkungsversuchs.	M17
Reconciliation	S-RCN	Vergleich von Plan, Effekt, Record und tatsächlichem Zustand.	M18
Dependency	S-DEP	Gemeinsame Quelle, Modell-, Daten-, Sonden- oder Kalibrierungsbindung.	M10
Horizon	S-HOR	Ankerrelativer Bereich gültiger Ableitung und Vorhersage.	M07
Branch	S-BRN	Identifizierbarer Ableitungszweig mit eigener Provenienz.	M24

Invariante 5.2 (Sortenabschluss). $|\mathcal{T}| = 19$. Die Sortenmenge ist abgeschlossen. Eine Implementierung DARF Untertypen einer Sorte einführen; sie DARF NICHT eine zwanzigste primäre Sorte einführen.

Invariante 5.3 (Eindeutige Primärsorte). Jeder IR-Knoten besitzt genau eine primäre Sorte. Mehrfachrollen werden als explizite Produkttypen modelliert, niemals als Sortenüberlagerung.

5.2 Kernproduktionen

Definition 5.4 (Produktionsmenge).

$$\begin{aligned}
 &\text{ObservationBundle} \Rightarrow \text{AnchorSnapshot} \\
 &(\text{AnchorSnapshot}, \text{Claim}) \Rightarrow \text{ThoughtBody} \\
 &(\text{ThoughtBody}, \text{FieldIdentity}) \Rightarrow \text{FieldProjection} \\
 &\{\text{FieldProjection}\}_{\text{compat}} \Rightarrow \text{ScopedCognitionGraph} \\
 &(\text{Graph}, \text{ValidationPlan}) \Rightarrow \text{GateDecision} \\
 &(\text{PASS}, \text{EffectToken}) \Rightarrow \text{EffectAttempt} \\
 &(\text{EffectAttempt}, \text{ExternalReceipt}) \Rightarrow \text{ActualizationWitness}
 \end{aligned}$$

Invariante 5.5 (Produktionsabschluss). Es existieren genau sieben Kernproduktionen. Ein Objekt, das nicht durch eine Kette von Kernproduktionen aus einem ObservationBundle ableitbar ist, DARF NICHT extern geltende Claims tragen.

5.3 Realitäts- und Faktizitätsordnung

Axiom 5.6 (Realitätsleiter). Für jeden Anker a gilt

$$R_a^{\text{act}} \subseteq R_a^{\text{con}} \subseteq R_a^{\text{rea}} \subseteq R_a^{\text{law}} \subseteq R_a^{\text{coh}}.$$

Jede Klassifikation MUSS Modell, Ressourcenprofil, Zeitfenster und Anker referenzieren. Absolute Statusbehauptungen sind unzulässig.

Definition 5.7 (Realitätsstatus als Typ). `RealityStatus` $\in$ {COHERENT, LAWFUL, REACHABLE, CONSTRUCTIBLE, ACTUALIZED, UNKNOWN}. UNKNOWN ist ein wirksamer Status mit Promotionssperre, kein fehlender Wert.

Definition 5.8 (Faktizitätsleiter). `FactStatus` $\in$ {SPECIFIED, POSSIBLE, SIMULATED, SELECTED, ATTEMPTED, OBSERVED, ACTUALIZED, RESIDUE, RETRACTED}.

Regel 5.9 (Kandidat und Gedankenkörper). Der normative Compilerlauf führt bis einschließlich C2 die Größe `candidate`, nicht `ThoughtBody`. Ein Kandidat ist eine kanonisierte, sortierte Vorform ohne Objektidentität; der `ThoughtBody` ist das kanonische Objekt und wird frühestens in C3 geprägt, sobald der Anker vorliegt oder `unanchored` ausdrücklich erklärt ist.

Daraus folgt dreierlei:

1. Struktur *Gedankenkörper* und ihre Forderung nach `anchor_refs` oder `unanchored` gelten ab der Prägung, nicht für den Kandidaten. C1 und C2 verletzen sie nicht, sie berühren sie nicht.
2. Es existiert keine Operation, die einem bestehenden `ThoughtBody` nachträglich einen Anker anheftet, und es DARF keine geben: Objekte sind inhaltsadressiert, ein nachträglicher Anker präge ein anderes Objekt. Die Reihenfolge ist deshalb bindend, nicht bequem.
3. Der Ablauf ist innerhalb eines Taktes vollständig: Kandidat in `normalize` und `type`, Anker in `anchor`, Prägung ebendort. Eine Verteilung über mehrere Takte ist zulässig, aber nicht erforderlich.

Invariante 5.10 (Keine implizite Promotion).

$$\begin{aligned} \text{SIMULATED} &\not\Rightarrow \text{ACTUALIZED}, & \text{SELECTED} &\not\Rightarrow \text{OBSERVED}, \\ \text{ATTEMPTED} &\not\Rightarrow \text{ACTUALIZED}, & \text{POSSIBLE} &\not\Rightarrow \text{ACTUALIZED}. \end{aligned}$$

Jede Promotion MUSS einen benannten Konstruktor, einen `GateReport` und einen `TraceSegment` besitzen. Die Statusfelder sind privat; ihr einziger Schreibpfad ist der jeweilige Promotionsoperator.

Regel 5.11 (Schreibpfad der Statusfelder). `ThoughtBody` ist nach Konstruktion unveränderlich. M07 DARF NICHT in einen `ThoughtBody` schreiben, den M06 besitzt; die Klassifikation verlässt M07 ausschließlich als eigenständiges Objekt `RealityClassification` über P09. Daraus folgt die einzige zulässige Kette:

1. M06 setzt `ThoughtBody.reality_status` bei Konstruktion auf UNKNOWN und `facticity` auf SPECIFIED. Andere Anfangswerte sind unzulässig.
2. M07 liest über P08, klassifiziert und emittiert `RealityClassification` über P09 an M23.
3. M23 schreibt den klassifizierten Wert in `IRNode.reality_status` und `IRNode.facticity`. Der `IRNode` trägt den geltenden Status, nicht der `ThoughtBody`.
4. Jede weitere Promotion erzeugt einen neuen `IRNode` mit neuer Objekt-ID; ein Statusfeld wird nie überschrieben.

Es existiert kein Port $M07 \rightarrow M06$ und DARF keiner eingeführt werden.

Definition 5.12 (Ankerrelativ imaginär). Ein Zustand x ist relativ zu a imaginär, wenn $x \in R_a^{\text{coh}}$, aber kein zurzeit bezeugter, ressourcen- und zeitgebundener Pfad $a \rightsquigarrow x$ existiert. IMAGINARY ist DARF NICHT ein

Wert von `RealityStatus` oder `FactStatus`, sondern ein daraus abgeleitetes Prädikat:

$$\text{imaginary}(x, a) : \Longleftrightarrow \text{reality_status}(x) \in \{\text{COHERENT}, \text{LAWFUL}\} \wedge \neg \text{reachable}(x, a).$$

Die Nichterreichbarkeit MUSS bezeugt sein. Ein bloß ungeprüfter Pfad ergibt `UNKNOWN`, nicht `IMAGINARY`; andernfalls greift `PSK-E005`. Imaginär zu sein ist kein Fehler und keine Promotionssperre eigener Art, sondern die typisierte Aussage, dass Kohärenz ohne Erreichbarkeit vorliegt.

Invariante 5.13 (Kein Grenzbypass). *Eine interne Ableitung $\gamma_{\text{int}} : a \rightsquigarrow b$ ist operativ nur zulässig, wenn a und b bezeugte Außenanker sind, jede interne Transformation typisiert und replaybar ist und eine externe Realisierungsprojektion $\Pi_{\text{ext}}(\gamma_{\text{int}}) : a \rightsquigarrow b$ existiert. Fehlt die Außenprojektion, bleibt die Ableitung als `SIMULATED`, *Hypothese*, *Plan* oder *Residuum* typisiert.*

Kanonisierung, Identität und Digest

6.1 Kanonisierungsfunktion

Algorithmus 6.1 (Can).

```
function Can(bytes, media) -> CanonicalBytes:
  obj = parse(bytes, media) // media in {yaml, json}
  require valid_utf8(bytes) else FAIL(PSK-E102)
  obj = normalize_scalars(obj) // ganzzahlig wo exakt; NaN/Inf verboten
  obj = sort_object_keys_recursive(obj) // lexikographisch, Codepointordnung
  // Arrayreihenfolge bleibt unverändert und ist semantisch
  return json_compact_serialize(obj) // Trennzeichen ",", " und ":", kein Whitespace
```

Invariante 6.2 (Idempotenz und Formatinvarianz). $\text{Can} \circ \text{Can} = \text{Can}$. Kommentare, Einrückung, Zeilennenden und Objektschlüsselreihenfolge sind nicht identitätsbildend. Arrayreihenfolgen sind identitätsbildend.

Vertrag 6.3 (Verbotene Serialisierungsformen). Gleitkommazahlen ohne exakte Dezimaldarstellung, NaN, Infinity, Zeitzone-lose Zeitstempel, Sprachtags ohne BCP-47-Form und nicht normalisierte Unicode-Formen sind in kanonisierten Objekten verboten und erzeugen PSK-E102. Unicode wird in NFC normalisiert.

6.2 Digest und Identität

Definition 6.4 (Digest). $H = \text{SHA-256}$. Ein Digest wird als 64 Zeichen Kleinbuchstaben-Hex dargestellt.

Definition 6.5 (Volatile Felder und Identitätsprojektion). Ein Feld ist *volatil*, wenn sein Wert von der physischen Uhr, der Messumgebung oder einer Ausgabeprojektion abhängt und daher bei inhaltsgleicher Wiederholung eines Laufes abweichen DARF. Die volatile Menge ist abgeschlossen und wird in `architecture/volatile_fields.yaml` geführt:

```
DualTime.tau_e DualTime.clock_ref DualTime.uncertainty_ns
*.runtime_metrics *.persona_projection *.view_*
```

Die Identitätsprojektion π_{vol} entfernt genau diese Felder rekursiv aus einem Objekt. τ_i ist DARF NICHT volatil: der Kausalzähler ist je `run_id` deterministisch und identitätsbildend.

Definition 6.6 (Objekt-ID). Für ein kanonisches Objekt o der Sorte s ist

$$\text{id}(o) = \text{psk} : s : H(\text{Can}(\pi_{\text{vol}}(o))).$$

Objekt-IDs sind inhaltsadressiert. Zwei Objekte mit gleicher ID sind inhaltsidentisch; sie DARF sich in volatilen Feldern unterscheiden.

Definition 6.7 (Recorddigest). Zusätzlich zur Objekt-ID trägt jedes persistierte Objekt

$$\text{record_digest}(o) = H(\text{Can}(o))$$

über das *vollständige* Objekt einschließlich der volatilen Felder. Der Recorddigest sichert die Integrität der gespeicherten Bytes; er ist DARF NICHT identitätsbildend und DARF NICHT Eingabe einer Gate-Entscheidung.

Invariante 6.8 (Trennung von Inhalt und Aufzeichnung). Kein Gate, kein Konsens, kein Konformanzurteil und keine Verklebung DARF den Recorddigest oder ein volatiles Feld auswerten. Zwei Läufe mit gleichem RunDescriptor und gleichen Eingaben MUSS für jedes erzeugte Objekt dieselbe Objekt-ID liefern; abweichende Recorddigests sind zulässig und kein Replaydefekt. Verstoß erzeugt PSK-E015.

Definition 6.9 (Kollektionsdigest). Für eine geordnete Liste normativer Dateien $f_1, \dots, f_n$:

$$I = H\left(\left\|_{k=1}^n \text{name}(f_k) \parallel 0 \times 00 \parallel H(\text{Can}(f_k)) \parallel \text{LF}\right\|.\right)$$

Regel 6.10 (Vier Identitäten).

$$I_C = H(\text{Can}(C)), \quad I_A = H(\text{Can}(\text{RA-Register})), \quad I_M = H(\text{Code} \parallel \text{Schemas} \parallel \text{Compiler} \parallel \text{Profil}),$$

$$I_t = H(\text{Can}(\Sigma_t)).$$

Invariante 6.11 (Selbstbeschreibung ist keine Selbstzertifizierung). Eine Instanz DARF ihre Konstitution, Architektur, Fähigkeiten und Zustände beschreiben. Geltung entsteht ausschließlich durch kanonische Artefakte, unabhängige Prüfpfade, Trace, Replay und gegebenenfalls Außenreconciliation.

6.3 Zeitmodell

Definition 6.12 (Doppelzeit). Jedes relevante Ereignis trägt $\tau(e) = (\tau_i(e), \tau_e(e))$, wobei τ_i die Erzeugungs- und Transformationsordnung (kausale Logikuhr, monoton, prozessintern) und τ_e die Beobachtungs-, Ledger- und Replayordnung (Wanduhr mit Zeitzone UTC und Auflösung Nanosekunde) bezeichnet.

Struktur 6.13 (DualTime).

```
DualTime:
  tau_i: uint64 # Lamport-artiger Kausalzaehler, streng monoton je run_id
  tau_e: RFC3339Nano # UTC, Pflicht: "Z"-Suffix
  clock_ref: ClockRef # Identitaet und Kalibrierstatus der verwendeten Uhr
  uncertainty_ns: uint64 # deklarierte Unsicherheit der Wanduhr
```

Invariante 6.14 (Replayneutralität der Wanduhr). τ_e DARF NICHT in eine Kanonisierung, einen Digest oder eine Gate-Entscheidung eingehen. Zeitabhängige Entscheidungen verwenden ausschließlich im RunDescriptor gebundene Zeitfenster und τ_i .

Kanonische Objekte

Dieses Kapitel legt die vollständigen Feldstrukturen fest. Ein Feld ohne Fragezeichen ist Pflicht. Ein fehlendes Pflichtfeld erzeugt PSK-E001 und Quarantäne.

7.1 Konstitutions- und Identitätsobjekte

Struktur 7.1 (RuntimeManifest). OBJ-RTM

```
RuntimeManifest:
  schema: psk.runtime-manifest/1.0
  constitution_id: Digest
  architecture_id: Digest
  implementation_id: Digest
  profile: ProfileId # readonly | shadow | sandbox | reference
  capability_matrix: CapabilityMatrixRef
  build_digest: Digest
  operator_versions: {OpId: SemVer}
  adapter_versions: {AdapterId: SemVer}
  determinism_class: R0 | R1 | R2 | R3
  max_depth: uint32 # Skalentiefe; Topologieregister verweist hierher
```

Struktur 7.2 (IdentityBinding). OBJ-IDB

```
IdentityBinding:
  schema: psk.identity-binding/1.0
  I_C: Digest
  I_A: Digest
  I_M: Digest
  I_t: Digest
  trace_head: Digest
  bound_at: DualTime
  intrinsic_section: Digest # aus Typ, Provenienz, Konstitutionsbindung, Trace
  extrinsic_section: Digest # aus Verhalten, Claims, Zertifikaten, Aussenrecords
  equalizer_sector: EqualizerId? # Pflicht fuer Identitaetsclosure
```

Vertrag 7.3 (Identitätsclosure). Eine Instanz besitzt Identitätsclosure nur, wenn intrinsische und extrinsische Sektion in einem gemeinsamen Vergleichsraum denselben equalizer_sector erreichen und Witness-, Trace- und Replayprädikate schließen. Extrinsische Übereinstimmung allein beweist keine konstitutionelle Identität.

7.2 Ankerobjekte

Struktur 7.4 (AnchorSnapshot). OBJ-ANC

```
AnchorSnapshot:
  schema: psk.anchor-snapshot/1.0
  id: ObjectId
  observations: [Observation] # rohe, unvermischte Aussenrecords
  provenance:
    source_adapter: AdapterId
    observer_identity: Digest # MUSS != Effekt-Adapteridentitaet
    method: string
    input_digests: [Digest]
    source_revision: RevisionId? # Git-Commit o. ae.; Regel Anker der
                                # Referenzdomaene verlangt seine Bindung.
                                # Nullstand zulaessig, wenn die Quelle
                                # keine Revision fuehrt --dann erklart.
  uncertainty:
    model: UncertaintyModelId
```

```

parameters: {string: Scalar}
context: ContextRef
time: DualTime
validity:
  freshness_predicate: PredicateExpr # z.B. "age_ns < 5e9 and fs_generation == g"
  expires_at_tau_i: uint64
boundary: ScopeExpr # was der Anker NICHT erfasst
digest: Digest
sealed: bool # true nach Versiegelung; danach unveränderlich

```

Axiom 7.5 (Anker ist nicht Welt). *Kein Anker DARF als Totalzustand der Außenwelt interpretiert werden. Nicht erfasste Außenkopplungen bleiben als Unsicherheit, offener Horizont oder Außenresiduum typisiert. Das Feld `boundary` ist Pflicht und DARF NICHT leer sein.*

Vertrag 7.6 (Ankerfrische). *Jeder Anker MUSS ein `freshness_predicate` besitzen. Eine Operation, deren benötigte Frische verletzt ist, erzeugt **HOLD** und einen `ReanchorRequest` über P27.*

Regel 7.7 (Zitierform normativer Blöcke). Nummern normativer Blöcke sind kapitelweise fortlaufend und verschieben sich, sobald ein Block eingefügt wird; Titel sind stabil. Eine Zitierung aus Code, Registern oder Werkzeugen MUSS daher den *Titel* führen. Eine zusätzlich genannte Nummer ist Bequemlichkeit, nie alleiniger Bezeichner, und MUSS maschinell gegen die aktuelle Fassung geprüft werden. Wer einen Block einfügt, MUSS die dadurch ausgelöste Verschiebung im Errata benennen; eine unbenannte Kaskade ist ein Defekt dieses Werkes.

Ein Werkzeug, das Zitierungen umschreibt, MUSS den Namensraum der Nummer kennen, die es verschiebt. Nummern kollidieren über Werke hinweg; eine Verschiebung, die für ein Werk richtig ist, trifft im anderen einen fremden Block. Eine Prüfung, die erst das *Ergebnis* vergleicht, sieht das nicht: sie bestätigt die Blockgleichheit dort, wo die Verschiebung stimmt, und schweigt zu der Stelle, die gar nicht hätte angefasst werden dürfen. Die Wache gehört vor die Auswahl, nicht hinter die Umschreibung.

Regel 7.8 (Ein Frischeprädikat muss verletzbar sein). Ein `freshness_predicate`, das unter keiner Beobachtung falsch werden kann, erfüllt Vertrag *Ankerfrische* nicht: es macht den dort vorgeschriebenen Ausgang — **HOLD** und `ReanchorRequest` über P27 — strukturell unerreichbar und den Port tot. Das Prädikat MUSS mindestens eine Beobachtung benennen, unter der es fällt; für einen über ein Verzeichnis versiegelten Anker ist das die Veränderung genau dieses Verzeichnisses. Ein Lauf, der den beobachteten Bereich selbst verändert, DARF NICHT danach unveränderte Frische behaupten. Dieselbe Strenge wie bei den Kantenbedingungen, aus anderem Grund: dort verhindert sie implizite Erlaubnis, hier einen nie beschrittenen Pfad.

7.3 Kognitionsobjekte

Struktur 7.9 (ThoughtBody). OBJ-THB

```

ThoughtBody:
  schema: psk.thought-body/1.0
  id: ObjectId
  anchor_refs: [ObjectId] # >= 1, oder explizit unanchored:true
  unanchored: bool
  claim:
    text: string # nur Projektion, nicht normativ
    formal: ClaimExpr # kanonischer Zustand
    directionality: internal | external
  models: [ModelRef] # Modellfamilie mit Version und Geltungsbereich
  trajectories: [TrajectoryRef]
  reality_status: RealityStatus
  facticity: FactStatus
  witness_refs: [ObjectId]
  uncertainty: UncertaintyBlock
  consequences: [ConsequenceRef] # Folgenklasse, Reversibilität, Reichweite
  validation_plan_ref: ObjectId?
  gate_refs: [ObjectId]
  residue_refs: [ObjectId]

```

```
lineage: Lineage
trace_ref: TraceRef
```

Axiom 7.10 (Gedanke ist kein Satz). *Natürliche Sprache DARF einen Gedankenkörper erzeugen oder projizieren, ist aber nicht sein kanonischer Zustand. Ein sprachlich identischer Satz DARF mehrere Gedankenkörper mit unterschiedlichen Ankern, Modellen, Folgen und Statuswerten besitzen. Das Feld claim.text geht nicht in Can ein; es wird von π_{vol} als non_canonical entfernt und ist damit weder identitätsbildend noch Gate-Eingabe. Im record_digest bleibt es enthalten.*

Struktur 7.11 (RealityClassification). OBJ-RCL

```
RealityClassification:
  schema: psk.reality-classification/1.0
  id: ObjectId
  thought_ref: ObjectId # klassifizierter ThoughtBody, unverändert
  anchor_ref: ObjectId # Anker, relativ zu dem klassifiziert wird
  reality_status: RealityStatus # genau ein Wert der geschlossenen Menge
  facticity: FactStatus # genau ein Wert der geschlossenen Menge
  reachability: witnessed | refuted | unexamined
  evidence_refs: [ObjectId] # Grundlage; leer nur bei UNKNOWN
  method_ref: PluginId? # MethodPlugin, falls verwendet
  residue_refs: [ObjectId]
  trace_ref: TraceRef
  classified_at: DualTime
```

Regel 7.12 (Klassifikation ist ein eigenes Objekt). M07 DARF NICHT in den ThoughtBody schreiben. Die Klassifikation verlässt M07 ausschließlich als RealityClassification über P09. Das Feld reachability trägt die für das Prädikat imaginary verlangte Unterscheidung: refuted setzt bezeugte Nichterreichbarkeit voraus, unexamined führt zu UNKNOWN und DARF NICHT zu IMAGINARY. Ein Verfahren ohne UNKNOWN-Ausgang ist nicht konform.

Vertrag 7.13 (Unknown als wirksamer Status). *Unwissen MUSS als offene Obligation, unscharfe Faser, fehlender Witness oder unentscheidbares Gate materialisiert werden. Ein verbales Unsicherheitsbekenntnis ohne interne Promotionssperre ist nicht konform und erzeugt T-UNKNOWN-001-Fehlschlag.*

Struktur 7.14 (FieldIdentity). OBJ-FLD

```
FieldIdentity:
  schema: psk.field-identity/1.0
  id: ObjectId
  domain: DomainExpr # D_lambda
  lens: LensSpec # Pi_lambda: typisierter Projektor
  operators: [OpId] # O_lambda, Teilmenge der ISA
  questions: [QuestionSpec] # Q_lambda
  witness_rules: WitnessPolicy # W_lambda
  boundaries: BoundarySpec # B_lambda: Ressourcen- und Effektgrenzen
  gates: [GateId] # G_lambda
  time_window: TimeWindow # Theta_lambda
  lineage: Lineage # Lin_lambda
  lifecycle: PROPOSED | ACTIVE | QUIESCENT | SPLIT_PENDING
    | MERGE_PENDING | FOLDED | RETIRED | EXCISED
  archetype: ArchetypeId # explorer|falsifier|constructor|historian|auditor|integrator
  dependency_profile_ref: ObjectId
  budget: BudgetSpec
  rollback: RollbackSpec
  marginal_gain: Scaled # zuletzt gemessener Wert von Delta-G
```

Struktur 7.15 (FieldProjection). OBJ-PRJ

```
FieldProjection:
  schema: psk.field-projection/1.0
  id: ObjectId
  field_ref: ObjectId # FieldIdentity, deren Linse angewandt wurde
  source_refs: [ObjectId] # ThoughtBody bzw. IR-Knoten des Urbildes
  lens_ref: LensSpec # Pi_lambda, wie zum Zeitpunkt der Anwendung gebunden
```

```

scope: ScopeSpec # Geltungsbereich; erweitert sich nie ohne Gate
visible: [IRNodeId] # was die Linse aufloest
occluded: [IRNodeId] # was sie nachweislich nicht aufloest
distinctions: [DistinctionSpec] # intern erzeugte Unterscheidungen
source_provenance: [SourceRef] # Grundlage des Abhängigkeitsquotienten (M10)
reality_view: RealityStatus # Sicht, nicht Promotion
tick: TickId # L-TICK: gilt genau fuer diesen Takt

```

Regel 7.16 (Projektion promoviert nicht). Eine `FieldProjection` ist eine Sicht, kein Fakt. `reality_status` und `facticity` des Urbildes DARF NICHT durch eine Projektion verändert werden; die Sorten-Port-Matrix verbietet die Relation (S-PRJ, S-RCN) vollständig.

Regel 7.17 (Nicht anwendbare Linse erzeugt keine Projektion). Ist die Spektrallinse auf die Sortenmenge des Urbildes nicht anwendbar, so entsteht DARF NICHT eine `FieldProjection`. Statt dessen MUSS M09 einen `ResidueRecord` des Typs `scope` erzeugen und PSK-E013 emittieren; dies ist die in Vertrag C5 verlangte Form. Eine `FieldProjection` mit leerem `visible` ist unzulässig: Es gibt keine leere Projektion, sondern nur eine ausgebliebene.

Regel 7.18 (Occlusion ist explizit). `occluded` MUSS vollständig geführt werden. Was eine Linse nicht auflöst, ist ein bezeugter Nichtbefund und DARF NICHT als Abwesenheit gewertet werden; andernfalls greift PSK-E005.

Invariante 7.19 (Kern–Feld-Trennung). *Keine Feldidentität und keine Menge aktiver Feldidentitäten ist mit der Gesamtmaschine identisch. Ein Feld DARF NICHT den Konstitutionskern, die globale Effektgrenze oder die Identität eines anderen Feldes aus eigener Autorität umschreiben. Die Spektrallinse Π_λ erweitert intern die Unterscheidungsfähigkeit und erzeugt keine physische Fähigkeit.*

Axiom 7.20 (Persona ist Apertur). *Eine Persona DARF Stil, Dialogkontinuität oder menschliche Lesbarkeit tragen. Sie DARF NICHT Systemidentität, Faktberechtigung, Witnessberechtigung oder Effektberechtigung erzeugen. Personaobjekte sind reine Ausgabeprofile und existieren ausschließlich in M27.*

7.4 Repräsentations- und Prüfobjekte

Struktur 7.21 (IRBundle). OBJ-IRB

```

IRBundle:
  schema: psk.ir/1.0
  version: SemVer # v
  constitution_id: Digest # N_C
  anchor_refs: [ObjectId] # A
  graph: {nodes: [IRNode], edges: [IREdge]} # G
  field_projections: [ObjectId] # F
  dependencies: ObjectId # D -> DependencyProfile
  witnesses: [ObjectId] # W
  residues: [ObjectId] # R
  gate_reports: [ObjectId] # Q
  trace_ref: TraceRef # T
  capabilities: [CapabilityRef] # C
  effect_contracts: [EffectContract] # E
  emission_class: EXECUTABLE | RESIDUAL | QUARANTINED | HOLD
  digest: Digest

```

Struktur 7.22 (IRNode / IREdge). OBJ-IRN, OBJ-IRE

```

IRNode:
  id: ObjectId
  sort: SortId # genau eine Primaersorte
  context: ContextRef
  lineage: Lineage
  reality_status: RealityStatus
  facticity: FactStatus
  anchor_refs: [ObjectId]
  witness_refs: [ObjectId]

```

```

residue_refs: [ObjectId]
trace_ref: TraceRef
payload_digest: Digest
m13_address: M13Address # Pflicht nach Pass C4

IREdge:
  id: ObjectId
  source: ObjectId
  target: ObjectId
  relation_sort: RelationSortId
  direction: forward | backward | bidirectional_declared
  preconditions: [PredicateExpr]
  postconditions: [PredicateExpr]
  gate_ref: ObjectId?
  dependency_refs: [ObjectId]
  trace_ref: TraceRef

```

Struktur 7.23 (CandidateCapsule). OBJ-CAP

```

CandidateCapsule:
  schema: psk.candidate-capsule/1.0
  id: ObjectId
  surface: SurfaceDescriptor # S: beobachtbare Rolle, Name, Behauptung
  invariant_core: [InvariantId] # I: geprüfte, nicht behauptete Invarianten
  phase: SEALED | PERTURBED | RATCHETED | SUPPORTED | CRYSTALLIZED
        | RESIDUAL | QUARANTINED
  witnesses: [ObjectId] # W
  replay: ReplayDescriptor # R
  boundary: ScopeExpr # B
  trace_ref: TraceRef # T
  coupling: [ObjectId] # C: Aussenkopplungen
  allowed_next: [CapsuleId] # Ratchet-Restmenge, monoton fallend
  status: OPEN | CLOSED

```

Invariante 7.24 (Keine vorzeitige Öffnung). Eine Kapsel DARF NICHT als kanonischer Bestandteil verwendet werden, solange ihr `invariant_core` ungelöst, ihr `replay` offen oder ihre `coupling` unbestimmt ist.

Struktur 7.25 (EvidenceObject). OBJ-EVD

```

EvidenceObject:
  schema: psk.evidence/1.0
  id: ObjectId
  type: measurement | derivation | attestation | external_record | counterexample
  scope: ScopeExpr
  input: [Digest]
  output: Digest
  method: MethodRef
  trace_ref: TraceRef
  hash: Digest
  signature: Signature?
  status: VALID | INVALID | OCCLUDED | EXPIRED
  independence_class: ClassId # aus DependencyProfile abgeleitet

```

Struktur 7.26 (DependencyProfile). OBJ-DEP

```

DependencyProfile:
  schema: psk.dependency-profile/1.0
  id: ObjectId
  sources: [SourceRef] # Daten, Modell, Prompt, Sonde, Kalibrierung
  witness_matrix: Matrix # W
  dependency_matrix: Matrix # D
  effective_rank: Scaled #  $r_{eff} = RankEff(W, D)$ 
  method: RankMethodId # deklarierte Schätzmethode
  quotient_classes: [[ObjectId]] #  $F / \sim_D$ 
  consensus_scope: local | cluster | system | external

```

Struktur 7.27 (Matrix und Scaled). Beide Typen sind intern berechnet, nicht domänengeliefert, und gehen in einen kanonischen Digest ein. Ihre Kodierung MUSS daher festgelegt sein:

```
Scaled: # einziger nichtganzzahliger Typ des Werkes;
  schema: psk.scaled/1.0 # Vertrag 6.3 verbietet Gleitkomma ohne
                        # exakte Dezimaldarstellung
  numerator: integer
  scale: integer # Wert = numerator / 10^scale, scale >= 0

Matrix:
  schema: psk.matrix/1.0
  axis: [SourceRef] # Zeilen- UND Spaltenachse, identisch
  scale: integer # gemeinsamer Nenner aller Zellen
  values: [[integer]] # quadratisch, |axis| x |axis|,
                    # Zelle (i,j) = values[i][j] / 10^scale
```

Regel 7.28 (Kanonische Achsenordnung). axis MUSS aufsteigend nach der kanonischen Form von SourceRef sortiert sein. Zeilen- und Spaltenachse sind identisch; eine Matrix mit abweichenden Achsen ist unzulässig. Ohne feste Achsenordnung erzeugten zwei Implementierungen für denselben Inhalt verschiedene Digests — der Fehler, den π_{vol} an anderer Stelle bereits ausschließt. Verstoß erzeugt PSK-E102.

Regel 7.29 (Eine Kapsel ist Funktion ihrer Klasse). capsule nimmt eine Quotientenklasse und MUSS eine reine Funktion dieser Klasse sein. Insbesondere allowed_next und die Gegenmodellmenge werden aus den Zeugen und Belegen *dieser* Klasse gebildet, nicht aus laufweiten Werten.

Stammt der Kapselinhalt aus dem Lauf, so ist die Klasse Beiwerk: zwei verschiedene Klassen ergeben dann notwendig dieselbe Kapsel, und der Ratchet hat kein Signal, an dem er sie unterscheiden könnte. Damit ginge der Zweck mehrerer Klassen beim ersten Schritt verloren — unabhängige Evidenzgruppen sind gerade deshalb getrennt, weil eine widerlegen kann, was die andere nicht widerlegt.

Konvergieren zwei Klassen dennoch auf dasselbe Ergebnis, so ist das ein *Messwert* über den Gegenstand. Konvergieren sie, weil die Funktion ihr Argument nicht liest, ist es ein Defekt. Beides sieht im Bericht gleich aus; der Nachweis MUSS deshalb zeigen, dass verschiedene Klasseninhalte verschiedene Kapseln ergeben können.

Invariante 7.30 (Mehrheit ist kein Wahrheitskonstruktor). Facettenzahl, Stimmenzahl oder sprachliche Übereinstimmung DARF Priorität erzeugen, DARF NICHT aber Faktpromotion ohne Anker, Evidence, Unabhängigkeitsrang und Reconciliation. Konsens verwendet effective_rank, niemals $\mathcal{F}$.

Struktur 7.31 (SeamReport / ObstructionRecord). OBJ-SEA, OBJ-OBS

```
SeamReport:
  id: ObjectId
  overlap: [ObjectId, ObjectId]
  restriction_digest_a: Digest
  restriction_digest_b: Digest
  compatible: bool
  tolerance_class: exact | declared_equivalence
  cell_refs: [M13Address]

ObstructionRecord:
  id: ObjectId
  kind: seam | order | identity | time | resource | capability | calibration
  located_at: [M13Address]
  involved: [ObjectId]
  severity: blocking | non_blocking
  residue_ref: ObjectId
  allowed_recovery: [RecoveryPathId]
```

Vertrag 7.32 (Kein halluziniertes Gluing). Fehlt Seam-Kompatibilität, DARF NICHT ein scheinbar glatter globaler Systemzustand erzeugt werden. Der Defekt wird als ObstructionRecord, Fork oder ResidueRecord materialisiert.

7.5 Gate-, Wirkungs- und Reconciliationsobjekte

Struktur 7.33 (GateReport). OBJ-GAT

```
GateReport:
  schema: psk.gate-report/1.0
  id: ObjectId
  gate_id: GateId
  order: 1 | 2 # Gate erster oder zweiter Ordnung
  input_digests: [Digest]
  decision: PASS | HOLD | FAIL
  reasons: [ReasonCode] # nicht leer bei HOLD oder FAIL
  evidence_refs: [ObjectId]
  residue_refs: [ObjectId]
  seam_report_refs: [ObjectId] # Pflicht bei order = 2
  replay_descriptor: ReplayDescriptor
  decided_at: DualTime
  trace_ref: TraceRef
```

Struktur 7.34 (EffectToken). OBJ-TOK

```
EffectToken:
  schema: psk.effect-token/1.0
  id: ObjectId
  subject: ModuleId # ausschliesslich M16
  effect_class: EffectClassId
  plan_digest: Digest # bindet genau einen EffectPlan
  scope: ScopeExpr # Pfad-, Ressourcen- und Reichweitengrenze
  capabilities: [CapabilityId]
  preconditions: [PredicateExpr]
  budget: BudgetSpec
  expires_at_tau_i: uint64
  idempotency_key: string
  nonce: bytes32
  issuer_digest: Digest # I_M von M15
  expected_receipt: ReceiptSpec # welcher Record erwartet wird
  rollback: RollbackSpec | NO_ROLLBACK_JUSTIFIED
  gate_report_ref: ObjectId
```

Vertrag 7.35 (Planbindung). Ein EffectToken MUSS Plan-Digest, Scope, Capabilities, Preconditions, Ablaufzeit, Idempotency Key und Issuer-Digest binden. Jede Änderung des Plans invalidiert das Token. Ein Token ist nicht übertragbar auf einen anderen Kandidaten.

Struktur 7.36 (EffectAttempt / ExternalReceipt). OBJ-ATT, OBJ-RCP

```
EffectAttempt:
  id: ObjectId
  token_ref: ObjectId
  adapter: AdapterId
  prestate_digest: Digest
  plan_digest: Digest
  started_at: DualTime
  ended_at: DualTime?
  outcome: COMPLETED | PARTIAL | FAILED | UNKNOWN_EFFECT
  error: ErrorCode?
  compensation_ref: ObjectId?

ExternalReceipt:
  id: ObjectId
  observer_adapter: AdapterId # MUSS != EffectAttempt.adapter
  observer_identity: Digest
  observed_at: DualTime
  record: bytes # unverändert, unvermischt
  provenance: ProvenanceBlock
  result_digest: Digest
  independence_attestation: Digest
```

Invariante 7.37 (Beobachtertrennung). ExternalReceipt.observer_adapter $\neq$

EffectAttempt.adapter *und* observer_identity $\neq$ issuer_digest. *Verletzung erzeugt PSK-E009 und blockiert jede Faktpromotion.*

Struktur 7.38 (ReconciliationReport). OBJ-RCN

```
ReconciliationReport:
  schema: psk.reconciliation/1.0
  id: ObjectId
  plan_ref: ObjectId
  attempt_ref: ObjectId
  receipt_refs: [ObjectId] # >= 1
  anchor_ref: ObjectId # Nachanker
  difference: DiffTree # Plan-Ist-Differenz, strukturiert
  verdict: CLOSED | OPEN | DIVERGENT | UNKNOWN
  finality: final | provisional | revocable
  fact_promotion: NONE | OBSERVED | ACTUALIZED
  residue_refs: [ObjectId]
  witness_ref: ObjectId # ActualizationWitness bei ACTUALIZED
```

Axiom 7.39 (Simulation ist keine Aktuation). *Erfolg in einer Simulation, einem digitalen Zwilling oder einer internen Projektion autorisiert keine Außenwirkung und keine Faktpromotion.*

Vertrag 7.40 (Actualization Witness). *Eine Promotion nach ACTUALIZED MUSS EffectToken, ExternalReceipt, ReconciliationReport mit verdict = CLOSED, Finalitätsklasse und trace_ref binden.*

7.6 Aufzeichnungsobjekte

Struktur 7.41 (TraceSegment). OBJ-TRC

```
TraceSegment:
  schema: psk.trace-segment/1.0
  seq: uint64 # global monoton je run_id
  prev_digest: Digest # Hash-Kette; Genesis = 64x"0"
  event_type: EventTypeId
  module: ModuleId
  port_id: PortId?
  object_refs: [ObjectId]
  payload_digest: Digest
  time: DualTime
  attestation: Signature?
  segment_digest: Digest # H(Can(pi_vol(alle vorstehenden Felder)))
  segment_record_digest: Digest # H(Can(alle vorstehenden Felder)), mit tau_e
```

Regel 7.42 (Zwei Digests je Segment). segment_digest ist identitätsbildend: es bildet die Hashkette, geht über L_t in I_t ein und MUSS daher π_{vol} anwenden. Andernfalls ginge τ_e entgegen der Invariante *Replayneutralität der Wanduhr* in einen Digest ein, und zwei inhaltsgleiche Läufe erhielten verschiedene Zustandsdigests — R2 wäre strukturell unerreichbar und damit kein Referenzrelease möglich.

segment_record_digest deckt dieselben Felder *einschließlich* τ_e ab und sichert die Integrität der gespeicherten Bytes. Es ist DARF NICHT Teil der Kette, DARF NICHT Eingabe von I_t und DARF NICHT Eingabe einer Gate-Entscheidung.

verify_chain MUSS beides prüfen: die Kettenfortschreibung über segment_digest und je Segment die Aufzeichnungsintegrität über segment_record_digest. Eine nachträgliche Änderung an τ_e bricht damit die zweite Prüfung, ohne die erste zu berühren — genau die Trennung von Inhalt und Aufzeichnung, die für kanonische Objekte bereits gilt.

Invariante 7.43 (Trace-Monotonie). *Der Trace ist append-only und hash-verkettet. Ein gültiger Lauf DARF NICHT frühere Entscheidungen, Prognosen, Residuen oder Gatebefunde überschreiben. Korrekturen erfolgen als neue Ereignisse, Retraktionen oder Supersessionen mit Verweis auf das Ursprungssegment.*

Struktur 7.44 (ResidueRecord). OBJ-RES

```

ResidueRecord:
  schema: psk.residue/1.0
  id: ObjectId
  type: type | anchor | witness | seam | dependency | scope | capability
       | effect | reconciliation | replay | budget | morphogenesis | horizon
  origin_module: ModuleId
  origin_object: ObjectId
  scope: ScopeExpr
  severity: blocking | non_blocking
  open_obligation: ObligationExpr
  allowed_followups: [OperationId]
  opened_at: DualTime
  closed_by: ObjectId? # nur durch expliziten Closure-Operator
  state: OPEN | HELD | QUARANTINED | RECANONICALIZED | CLOSED | EXCISED

```

Axiom 7.45 (No Silent Loss). Jede Nichtclosure erzeugt HOLD, FAIL, Quarantäne, Fork, Reanalyseauftrag oder ein sichtbares Residuum. Stilles Verwerfen ist nicht konform. Ein ResidueRecord DARF NICHT gelöscht werden; er wird ausschließlich in einen Folgezustand überführt.

Struktur 7.46 (RunDescriptor). OBJ-RUN

```

RunDescriptor:
  schema: psk.run-descriptor/1.0
  run_id: RunId
  I_C: Digest
  I_A: Digest
  I_M: Digest
  seed: bytes32 # Wurzel aller Pseudozufallsstroeme
  versions: {ComponentId: SemVer}
  input_digests: [Digest] # versiegelte Eingaben
  operators: [OpId]
  environment: EnvironmentProfile # deklarierte, replayrelevante Umgebung
  time_window: TimeWindow # gebundene Zeitgrenzen fuer Entscheidungen
  nondeterminism_budget: NDBudget
  ratchet_max_rounds: uint32 # Regel Ratchet-Terminierung; deklariert, nicht geraten #
    zulaessige, deklarierte Abweichungsquellen
  canon: CanonicalizationProfile
  digest: Digest

```

Invariante 7.47 (Gebundener Nichtdeterminismus). Zufall, Zeit, Parallelität, externe Schnittstellen und Modellstichproben MUSS entweder deterministisch aus seed und run_id abgeleitet, als externe Evidence aufgezeichnet oder im nondeterminism_budget deklariert sein. Eine nicht deklarierte Nichtdeterminismusquelle erzeugt PSK-E015.

Struktur 7.48 (ExcisionCertificate). OBJ-EXC

```

ExcisionCertificate:
  schema: psk.excision/1.0
  id: ObjectId
  region: [ObjectId] # A: entfernte Pfade oder Typen
  boundary: ScopeExpr # dA
  reconnect_projection: MapExpr # pi_A: Neurouting der Anschluesse
  preservation_witness: [ObjectId] # W_A: erhaltene Invarianten
  difference: DiffTree # Delta_A
  residues: [ObjectId]
  identity_effect: UNCHANGED | DECLARED_SUCCESSOR
  successor_id: Digest? # Pflicht bei DECLARED_SUCCESSOR
  trace_ref: TraceRef

```

Vertrag 7.49 (Normative Entfernung). Eine Exzision MUSS ausweisen, welche Pfade oder Typen entfernt werden, welche Invarianten erhalten bleiben, welche Anschlüsse neu geroutet werden, welche Residuen entstehen und ob die Zielidentität unverändert bleibt oder ein neuer Nachfolger deklariert wird. Eine Entfernung normativer Information ohne ExcisionCertificate erzeugt FAIL.

Struktur 7.50 (MachineCertificate). OBJ-CRT

```
MachineCertificate:
  schema: psk.machine-certificate/1.0
  I_C: Digest
  I_A: Digest
  I_M: Digest
  I_t: Digest
  conformance_class: C0 | C1 | C2 | C3 | C4 | C5
  feature_coverage: [FC0..FC8] # abgeleiteter Deckungsvektor
  replay_class: R0 | R1 | R2 | R3
  gate_report_digest: Digest
  trace_head: Digest
  replay_manifest_digest: Digest
  residue_report_digest: Digest
  capability_audit_digest: Digest
  negative_test_report_digest: Digest
  scope: ScopeExpr # explizit, nicht universell
  issued_at: DualTime
  signature: Signature
```

Regel 7.51 (Das Zertifikat ist das letzte Artefakt des Laufes). Die Ausstellung MUSS nach allen Schritten geschehen, deren Ergebnisse in sie eingehen. Insbesondere liegt der Selbstkompilationsschritt *vor* ihr: sein Gatebericht ist der Beleg für FC7, und ein zuvor ausgestelltes Zertifikat könnte diesen Deckungsanteil nicht ableiten, sondern nur behaupten. Wer die Reihenfolge umkehrt, erzeugt kein früheres Zertifikat, sondern ein ärmeres.

Regel 7.52 (Ein Bodenwert ist keine Messung). Gibt eine Größe bei fehlender Eingabe einen Vorgabe- oder Bodenwert zurück — etwa einen Rang von eins über einer leeren Evidenzmenge —, so DARF NICHT dieser Wert als Messergebnis gelesen oder berichtet werden. Er ist von einem berechneten Wert äußerlich nicht zu unterscheiden und deshalb gefährlicher als ein leeres Feld: eine Null fällt auf, eine Eins sieht aus wie ein Befund.

Jede solche Größe MUSS unterscheidbar machen, ob sie gemessen oder untergegangen ist — durch einen eigenen Ausgang, nicht durch einen Zahlenwert. Ein Bericht, der den Bodenwert neben gemessenen Werten führt, ohne ihn zu kennzeichnen, behauptet eine Messung, die nicht stattfand.

Die erste echte Eingabe hebt den Wert dann nicht bloß an; sie macht ihn erstmals zu einem Wert.

Regel 7.53 (Erklärter Nullstand). Ein Pflichtfeld, dessen Quelle in der gegenwärtigen Domäne oder Konformitätsstufe nicht existiert, DARF leer geführt werden — aber nur als *erklärter* Nullstand. Dazu gehören drei Dinge, und keines ist entbehrlich:

1. Die Bedingung ist benannt, unter der die Quelle entstünde.
2. Ein Nachweis existiert, der genau dann fällt, wenn diese Bedingung eintritt — er verhindert, dass der Nullstand seine Berechtigung überlebt.
3. Der Nullstand ist im Artefakt selbst sichtbar, nicht nur in einem Kommentar daneben.

Ein leeres Pflichtfeld ohne diese drei ist kein Nullstand, sondern ein Konformitätsdefekt.

Ausdrücklich unzulässig ist die Ersatzfüllung: eine Null, ein Platzhalterdigest oder ein Vorgabewert an der Stelle einer fehlenden Quelle. Leer ist ehrlich; gefüllt ist eine Behauptung über etwas, das nicht gemessen wurde — und ununterscheidbar von einer gemessenen Null.

Regel 7.54 (Unsignierte Ausstellung unterhalb C4). `signature` ist ein Pflichtfeld; ein leerer Wert ist kein Verfahren. Er ist dennoch zulässig, solange die beanspruchte Klasse unterhalb von C4 liegt: Vertrag *Selbstgültigkeit* verlangt die Signatur für die *Selbstgültigkeit* des Zertifikats, nicht für seine Ausstellung, und OBL-005 führt Signaturverfahren und Schlüsselhaltung als domänenabhängig mit `blocking_from: C4`.

Ein solches Zertifikat ist auszustellen, aber nicht selbstgültig: es bezeugt einen Lauf und belegt sich nicht selbst. Der Nullstand MUSS als erklärter geführt werden — mit einem Nachweis, der genau in dem

Moment fällt, in dem C4 erreicht wird. Ein stillschweigend leeres Signaturfeld auf C4 oder darüber ist ein Konformitätsdefekt.

Regel 7.55 (Jedes Zertifikatsfeld ist abgeleitet). Kein Feld eines `MachineCertificate` DARF einen Wert tragen, der nicht aus einem Artefakt des zertifizierten Laufes stammt. Insbesondere:

1. I_C, I_A, I_M sind die Werte der `IdentityBinding` desselben Bootes, nicht neu gebildete oder eingesetzte;
2. I_t ist $H(\text{Can}(\Sigma_t))$ nach den Takten;
3. die vier Berichtsdigests sind Digests der tatsächlich aggregierten Berichte;
4. `feature_coverage` ist abgeleitet, nicht beansprucht;
5. `trace_head` ist der Kopf der Kette dieses Laufes.

Für jedes Feld MUSS ein Test die Ableitung belegen, und zwar so, dass ein konstanter Wert an dieser Stelle den Test fallen lässt. Ein Zertifikat, dessen Felder sich nicht einzeln auf Laufartefakte zurückführen lassen, bezeugt seine eigene Erzeugung und sonst nichts.

Die Prüfung geschieht feldweise, nicht am Ganzen: ein geheiltes Feld neben sechs konstanten sieht von außen aus wie ein geheiltes Zertifikat.

Regel 7.56 (Plattformgebundene Verpflichtungsauflösung im Zertifikat). Ist die Auflösung einer offenen Verpflichtung plattform- oder umgebungsgebunden erklärt (`resolution_platform` im Verpflichtungsregister), so MUSS `scope` des `MachineCertificate` diese Bindung tragen. Ein Zertifikat, dessen Konformanzklasse von einer plattformgebundenen Auflösung abhängt, DARF NICHT ohne diese Angabe ausgestellt werden: es behauptete sonst mehr, als geprüft wurde. Auf einer Plattform, für die keine Auflösung deklariert ist, gilt die Verpflichtung als offen und die davon abhängige Konformanzklasse als nicht erreicht.

Vertrag 7.57 (Selbstgültigkeit). *Ein Zertifikat ist nur selbstgültig, wenn Signatur, alle vier Identitäten, Konstitutionsdigest, Schemas, Gatebericht, Tracekopf, Replaymanifest, Residuenbericht und Capability Audit gemeinsam verifiziert werden.*

Objektlebenszyklen, Ownership und Lebensdauer

8.1 Lebensdauerklassen

Definition 8.1 (Fünf Lebensdauerklassen).

Klasse	Dauer	Regel
L-TICK	ein Scheduler-Tick	Wird am Tickende verworfen; MUSS vollständig aus dem Trace rekonstruierbar sein.
L-RUN	ein Lauf	Lebt bis ARCHIVED oder FAILED; wird im Replay Memory persistiert.
L-EPOCH	bis Reankrierung	An genau einen AnchorSnapshot gebunden; verfällt mit dessen Frische.
L-PERSIST	unbegrenzt	Append-only; nur durch Exzisionszertifikat entfernbar.
L-IMMUTABLE	unbegrenzt	Inhaltsadressiert, nach Erzeugung unveränderlich.

Objekt	Lebensdauer	Owner	Speicherklasse	Abschlussbedingung
RuntimeManifest	L-RUN	M04	Canonical	Boot-Gate PASS
IdentityBinding	L-RUN	M04	Canonical	Equalizer geschlossen
AnchorSnapshot	L-EPOCH	M05	Canonical	versiegelt
ThoughtBody	L-RUN	M06	Working Canonical	→ ACTUALIZED oder RESIDUE
FieldIdentity	L-RUN	M08	Working Canonical	→ RETIRED oder EXCISED
FieldProjection	L-TICK	M09	Working	Quotient berechnet
DependencyProfile	L-RUN	M10	Canonical	Rang gebunden
IRBundle	L-RUN	M23	Working Canonical	→ Emissionsklasse gesetzt
CandidateCapsule	L-RUN	M24	Working	CRYSTALLIZED oder RESIDUAL
SeamReport	L-TICK	M11	Working	in ClosureReport gebunden
ObstructionRecord	L-PERSIST	M11	Residual	Residuum geschlossen
EvidenceObject	L-IMMUTABLE	M12	Witness	—
ValidationPlan	L-EPOCH	M13	Working	ausgeführt oder verfallen
GateReport	L-IMMUTABLE	M14	Canonical	—
EffectToken	L-EPOCH	M15	Canonical	konsumiert, abgelaufen oder invalidiert
EffectAttempt	L-IMMUTABLE	M16	Canonical	—
ExternalReceipt	L-IMMUTABLE	M17	Witness	—
ReconciliationReport	L-IMMUTABLE	M18	Canonical	—
TraceSegment	L-PERSIST	M19	Trace	—
ResidueRecord	L-PERSIST	M19	Residual	CLOSED oder EXCISED
RunDescriptor	L-IMMUTABLE	M19	Replay	—
ExcisionCertificate	L-IMMUTABLE	M20	Canonical	—
MachineCertificate	L-IMMUTABLE	M21	Archive	—
ViewArtifact	L-TICK	M27	—	keine Persistenzpflicht

8.2 Allokations- und Freigaberegeln

Regel 8.2 (Erzeugung). Ein Objekt entsteht ausschließlich durch einen benannten Konstruktor seines Owner-Moduls. Der Konstruktor MUSS vor Rückgabe: (i) das Schema validieren (M03); (ii) den kanonischen Digest berechnen (M01); (iii) ein TraceSegment über P28 schreiben.

Regel 8.3 (Freigabe). L-TICK- und L-EPOCH-Objekte werden am Ende ihres Gültigkeitsbereichs freigegeben. Die Freigabe MUSS ein TraceSegment mit `event_type = object_released` und dem Objektdigest schreiben. L-PERSIST- und L-IMMUTABLE-Objekte werden niemals freigegeben.

Invariante 8.4 (Keine stille Freigabe). *Ein Objekt, das in einem GateReport, einer EvidenceObject-Kette oder einem ResidueRecord referenziert ist, DARF NICHT freigegeben werden, solange der Referent offen ist. Ein Versuch erzeugt PSK-E014.*

Regel 8.5 (Borrow-Semantik). Ein Nicht-Owner-Modul erhält Objekte ausschließlich als unveränderliche Referenz (ObjectId plus Digest). Es DARF den Inhalt lesen und in eigene Objekte einbetten. Es DARF NICHT den Inhalt verändern oder ein Objekt desselben Typs erzeugen.

TEIL III

Syntaxskelett und Zwischenrepräsentation

Topologiedienst M_{13}

9.1 Graphstruktur

Axiom 9.1 (M_{13} -Primat). *Der Metatron-Würfel M_{13} ist das kanonische topologische Syntaxskelett. Eine konforme Implementierung MUSS seine Knoten-, Kanten- und Zellordnung exakt realisieren. Seine Ersetzung erzeugt eine neue Spezifikationsidentität.*

Definition 9.2 (M_{13} -Graph).

$$M_{13} = (V, E, \Delta, \kappa), \quad V = \{c\} \cup \{i_0, \dots, i_5\} \cup \{o_0, \dots, o_5\}, \quad |V| = 13.$$

Alle Indizes werden modulo 6 gelesen.

Definition 9.3 (Kantenklassen).

$$E_c = \{(c, i_k)\}, \quad E_i = \{(i_k, i_{k+1})\}, \quad E_o = \{(o_k, o_{k+1})\}, \quad E_r = \{(i_k, o_k)\}, \quad E_d = \{(i_k, o_{k-1})\},$$

$$E = E_c \sqcup E_i \sqcup E_o \sqcup E_r \sqcup E_d, \quad |E| = 5 \cdot 6 = 30.$$

Definition 9.4 (Zellklassen).

$$\Delta_k^c = (c, i_k, i_{k+1}), \quad \Delta_k^b = (i_k, i_{k+1}, o_k), \quad \Delta_k^o = (i_k, o_k, o_{k-1}),$$

$$\Delta = \Delta^c \sqcup \Delta^b \sqcup \Delta^o, \quad |\Delta| = 3 \cdot 6 = 18.$$

Invariante 9.5 (Exakte Kardinalität). $|V| = 13$, $|E| = 30$, $|\Delta| = 18$, $|\Delta^c| = |\Delta^b| = |\Delta^o| = 6$. Jede Abweichung erzeugt PSK-E011 und blockiert den Boot.

Definition 9.6 (Semantische Rollen der Zellklassen).

Klasse	Rolle	Normative Prüfpflicht der Zelle
Δ^c	Zentrum	Identität, Konstitutionsbindung, Sortenkonsistenz, Lineage.
Δ^b	Übergang	Seam, Portkompatibilität, Abhängigkeitsquotient, Richtungskonsistenz.
Δ^o	Boundary	Ankerbindung, Effektgrenze, Capability, Horizont, Außenresiduum.

Die drei Klassen sind nicht austauschbar.

Vertrag 9.7 (Zellclosure). *Eine Zelle DARF nur dann als geschlossen markiert werden, wenn ihre drei Knoten typisiert, ihre drei Kanten portkompatibel, ihre Richtung konsistent, ihre Witness- und Traceverweise auflösbar und ihre offenen Differenzen explizit residualisiert sind.*

Regel 9.8 (Richtungskonsistenz einer Zelle). *Richtung konsistent* bedeutet: für jede Kante der Zelle mit `direction: forward` gilt die Schichtordnung der Invariante *Abhängigkeitsazyklizität*, also $L(\text{Ziel}) \geq L(\text{Quelle})$ oder gemeinsame Passträgerschaft. Eine Kante mit `direction: backward` MUSS einem der benannten Rückflusskanäle entsprechen. `bidirectional_declared` MUSS in beiden Richtungen je eine dieser Bedingungen erfüllen. Es wird keine eigene Ordnung eingeführt — die Zelle erbt die des Modulgraphen.

Regel 9.9 (Leere Zelle schließt vakuum, aber nicht stillschweigend). Eine Zelle ohne belegte Knoten erfüllt die Bedingungen von Vertrag *Zellclosure* vakuum: über einer leeren Menge ist nichts unabgeschlossen. Sie MUSS deshalb als geschlossen zählen — andernfalls hinge EXECUTABLE an der Größe des Graphen statt an seiner Korrektheit, wofür das Werk keine Grundlage kennt.

Der CellReport MUSS jedoch `occupancy` führen und `close_all_18` die Zahl der vakuum geschlossenen Zellen berichten. Eine Schließung, die nur deshalb gilt, weil nichts zu schließen war, ist ein Messwert

und DARF NICHT als Beleg geprüfter Struktur ausgegeben werden — dieselbe Pflicht wie bei `occluded` und bei `cra_inputs_absent`.

Struktur 9.10 (CellReport).

```
CellReport:
  cell: CellId
  occupancy: occupied | empty # leer schliesst vakuum, wird aber ausgewiesen
  closure_mode: substantive | trivial | vacuous
                    # trivial: Route ohne Transport (Regel Triviale Route)
                    # vacuous: Zelle ohne Knoten

  nodes_typed: bool
  edges_port_compatible: bool
  direction_consistent: bool
  refs_resolution: checked | vacuous_empty
                    # ueber leerer Verweisliste ist "aufloesbar"
                    # vakuum wahr und KEIN Beleg

  differences_residualized: bool
  probe_notes: [ProbeNote] # Sondierung wird vermerkt, nicht verworfen

ProbeNote:
  node: ObjectId
  computed_k: uint32 # k = H(Can(node)) mod 6, vor Sondierung
  reached_k: uint32 # tatsaechlich belegte Zelle
  probe_steps: uint32 # 0 heisst konfliktfrei
```

Regel 9.11 (Vakuum ist kein Beleg). `closure_mode` und `refs_resolution` MUSS geführt werden, und eine Auswertung, die `trivial` oder `vacuous` ergibt, DARF NICHT als geprüfte Struktur ausgegeben werden. Ein Bericht, der achtzehn geschlossene Zellen nennt, ohne die vakuumen zu beziffern, behauptet mehr, als er gemessen hat.

9.2 Adressraum

Definition 9.12 (M13Address).

```
M13Address:
  grammar: "m13:" scale_path "/" cell_id [ "/" node_id ]
  scale_path: level { "." parent_cell_id "." parent_node_id }
  cell_id: ( "c" | "b" | "o" ) k # k in 0..5
  node_id: "c" | "i" k | "o" k
  # Beispiele:
  # m13:0/c3 -> Zentrumzelle 3 auf Grobskala
  # m13:0/b4/o4 -> Boundaryknoten o4 in Uebergangszelle 4
  # m13:1.c2.i2/b5/o5 -> Feinskala 1, betreten ueber Zentrumzelle 2 und
  # Innenknoten i2; darin Brueckenzelle 5, Aussenknoten o5
```

Regel 9.13 (Wohlgeformtheit einer M13Address). Eine Adresse ist genau dann wohlgeformt, wenn zusätzlich zur Grammatik gilt:

1. `level` ist gleich der Anzahl der Abstiegs-paare `"." parent_cell_id "." parent_node_id` im `scale_path`. Skala 0 trägt kein Abstiegs-paar; jede weitere Skalenstufe trägt genau eines.
2. `parent_node_id` ist Knoten von `parent_cell_id` nach dem Zellregister; ebenso ist `node_id`, sofern angegeben, Knoten von `cell_id`. Die Zugehörigkeit wird gegen `m13_topology.yaml` geprüft, nicht gegen die Schreibweise.
3. `level` überschreitet `max_depth` des `RuntimeManifest` nicht.

Eine grammatisch gültige, aber nicht wohlgeformte Adresse erzeugt PSK-E011 und DARF NICHT in eine Zelle eintreten.

Regel 9.14 (Platzierungsregel). M22 platziert jeden IR-Knoten deterministisch:

1. Knoten der Sorten `S-IDT`, `S-CTX`, `S-BRN` → Δ^c .
2. Knoten der Sorten `S-PRJ`, `S-FLD`, `S-DEP`, `S-STA`, `S-TRN`, `S-EVT` → Δ^b .
3. Knoten der Sorten `S-ANC`, `S-CAP`, `S-EFF`, `S-RCP`, `S-RCN`, `S-HOR` → Δ^o .
4. Knoten der Sorten `S-WIT`, `S-GAT`, `S-TRC`, `S-RES` → diejenige Zelle, die den geprüften Knoten trägt;

sie sind zellgebunden, nicht frei platzierbar.

5. Der Index k ergibt sich als $k = H(\text{Can}(\text{node})) \bmod 6$, sofern nicht durch eine Kante bereits erzwungen. Konflikte werden durch aufsteigende Sondierung $k, k+1, \dots$ aufgelöst und im `CellReport` vermerkt.

Die Platzierung ist eine reine Funktion des kanonisierten Knotens und des Kantenkontextes; sie ist damit replaystabil.

Vertrag 9.15 (Gebundene Platzierung). *Kein extern gerichteter Claim DARF ohne Ankerreferenz in eine M₁₃-Zelle eintreten. Reine Simulationen DARF an einem Simulationsanker gebunden werden, MUSS diesen Typ jedoch bis zu einer externen Reconciliation erhalten.*

9.3 Closure erster und zweiter Ordnung

Definition 9.16 (360-Grad-Closure).

$$\text{Close}_{360}(x) = 1 \iff \pi(\Phi(x)) = \pi(x) \wedge \text{Seam}(\Phi, x) = 1.$$

Definition 9.17 (720-Grad-Closure).

$$\text{Close}_{720}(x) = 1 \iff \Phi^2(x) \equiv_{\text{can}} x \wedge \text{Hol}(\Phi^2) = \mathbb{I} \wedge \text{Replay}(\Phi^2) \equiv_{\text{can}} x.$$

Definition 9.18 (Chartwechsel). $T_{\iota_a \iota_b}$ überführt die Darstellung eines kanonischen Objekts vom Chart ι_a in das Chart ι_b . Es ist ein *reiner Rahmenwechsel*: der kanonische Inhalt ohne Adressanteil bleibt unverändert, ausschließlich die `M13Address` wird umgeschrieben. Daraus folgt unmittelbar:

1. $\text{Can}(\pi_{\text{addr}}(T_{\iota_a \iota_b}(E))) = \text{Can}(\pi_{\text{addr}}(E))$, wobei π_{addr} den Adressanteil entfernt. Ein Transport, der den Inhalt ändert, ist kein Transport, sondern ein Defekt (`PSK-E011`).
2. $T_{\iota \iota} = I$: besteht kein Rahmenunterschied, gibt es nichts zu wechseln.
3. $T_{\iota_b \iota_a} \circ T_{\iota_a \iota_b} = I$: ein Rahmenwechsel ist umkehrbar, sonst wäre die Adresse nicht wiederherstellbar.

Definition 9.19 (Transport und Holonomie). Entlang einer Chartkette $\iota_0 \rightarrow \iota_1 \rightarrow \dots \rightarrow \iota_m$ wirkt $T_\gamma = T_{\iota_{m-1} \iota_m} \circ \dots \circ T_{\iota_0 \iota_1}$. Bei geschlossener Route ist $\text{Hol}_\gamma = T_\gamma$ die akkumulierte Rahmenänderung. Der in den Closure-Prädikaten verwendete Operator Φ ist T_γ für die geschlossene Route der jeweils betrachteten Zelle oder Skalenstufe.

Regel 9.20 (Eintrittszelle eines Abstiegs). `descend` liefert eine vollständige `M13Address` und MUSS daher auch die Zelle im Feinchart bestimmen. Es wird dafür keine eigene Regel eingeführt: es gilt dieselbe Platzierungsregel wie auf jeder anderen Skalenstufe — Sortenklasse bestimmt die Zellklasse, $k = H(\text{Can}(\text{node})) \bmod 6$, Konflikte durch aufsteigende Sondierung, vermerkt in `probe_notes`. Der Abstieg verlängert den `scale_path` um genau ein Paar; die Zelle darin wird platziert, nicht vererbt.

Regel 9.21 (Triviale Route ist eine Route). Deklariert der `RuntimeManifest` `max_depth = 0`, so existiert genau ein Chart, jede geschlossene Route ist trivial, und es gilt $\Phi = I$ und $\text{Hol}(\Phi^2) = I$ — nicht als Annahme, sondern aus $T_{\iota \iota} = I$. Der `CellReport` MUSS diesen Fall als *trivial geschlossen* ausweisen. Eine Holonomie, die nur deshalb die Identität ist, weil nichts transportiert wurde, DARF NICHT als Beleg für Transportkorrektheit gelten.

Vertrag 9.22 (Semantische Rückkehr). *Eine geschlossene Atlasroute ist semantisch geschlossen, wenn $\text{Can}(\text{Hol}_\gamma(E)) = \text{Can}(E)$ gilt oder die Abweichung als zulässiger Wicklungssektor, deklarierter Nachfolger oder sichtbares Residuum klassifiziert ist.*

Invariante 9.23 (Keine Halbschließung). $\text{Close}_{360}(x) = 1 \not\Rightarrow \text{Close}_{720}(x) = 1$. Ein lokal kohärentes Ergebnis DARF NICHT als identitätsgeschlossen ausgegeben werden, solange Gegenphase, Trace und Rücktransport offen sind. Emissionsklasse `EXECUTABLE` erfordert $\text{Close}_{720} = 1$.

9.4 Skalenverschachtelung

Definition 9.24 (Skalenturm). $M_{13}^{(0)} \triangleright M_{13}^{(1)} \triangleright \dots \triangleright M_{13}^{(N)}$. Jeder Knoten DARF auf feinerer Skala selbst einen M_{13} -Chart tragen, sofern Tracehebung, Eltern-/Kindidentität und Skalen-Commit erhalten bleiben.

Invariante 9.25 (Keine freie unendliche Tiefe). *N ist endlich und im RuntimeManifest deklariert. Der nicht aufgelöste Rest wird als S-HOR-Knoten mit ResidueRecord des Typs horizon geführt. Ein grober Knoten DARF nur dann als geschlossen gelten, wenn sein deklarierter Feinkörper entweder geschlossen oder sichtbar residualisiert ist.*

Schnittstelle 9.26 (M22-Ports).

```
interface M13TopologyService {
  fn nodes() -> [NodeId] // exakt 13
  fn edges() -> [(NodeId, NodeId, EdgeClass)] // exakt 30
  fn cells() -> [Cell] // exakt 18
  fn cells_of(node: NodeId) -> [CellId]
  fn place(node: IRNode, ctx: EdgeContext) -> M13Address // deterministisch
  fn close_cell(cell: CellId, graph: IRGraph) -> CellReport
  fn close_all_18(graph: IRGraph) -> CellReport // Pass C9
  fn descend(addr: M13Address, node: IRNode) -> M13Address // Skalenabstieg;
                                // node noetig, da die Eintrittszelle platziert wird
  fn holonomy(route: [M13Address]) -> Transport
  errors: PSK-E011
}
```

Zwischenrepräsentation

10.1 Zielidentität der IR

Invariante 10.1 (IR-Neutralität). Die PSK-IR ist die kanonische, implementierungsunabhängige Sprache zwischen Grammatik und Runtime. Sie DARF NICHT natürliche Sprache oder eine Programmiersprache als normative Semantik verwenden. Ein IR-Feld, dessen Bedeutung nur aus einem Modellprompt hervorgeht, ist ein Architekturdefekt.

Definition 10.2 (IR-Bündel). $I = (v, N_C, A, G, F, D, W, R, Q, T, C, E)$ gemäß Struktur 7, OBJ-IRB.

10.2 Kanonische Serialisierung

Algorithmus 10.3 (IR-Codec).

```
function ir_encode(bundle: IRBundle) -> bytes:
    b = bundle
    b.graph.nodes = sort_by(b.graph.nodes, key = node.id) // stabil
    b.graph.edges = sort_by(b.graph.edges, key = (edge.source, edge.target, edge.id))
    for each list-valued field of b that is NOT order-semantic:
        sort lexicographically by element id
    strip(b, volatile_fields) // tau_e, Laufzeitmesswerte, Personaprojektionen
    return Can(b, media = json)

function ir_decode(bytes) -> IRBundle:
    obj = parse_json(bytes)
    require schema_valid(obj, "psk.ir/1.0") else FAIL(PSK-E001)
    return obj

// Round-Trip-Pflicht:
// ir_encode(ir_decode(x)) == x fuer jedes kanonische x
```

Vertrag 10.4 (Identitätsstabilität). Semantisch irrelevante Reihenfolge, Formatierung, Whitespace und volatile Zeitfelder DARF NICHT den kanonischen Digest verändern. Jede normative Änderung an Sorten, Produktionen, M_{13} , Invarianten oder Passreihenfolge MUSS eine neue Spezifikations-ID erzeugen.

Definition 10.5 (Ordnungssemantische Listen). Die folgenden Listen sind ordnungssemantisch und DARF NICHT sortiert werden: `normative_files`, `trajectories`, `allowed_next` (Ratchet), `transitions` eines Automaten, `observations` eines Ankers, `receipt_refs` einer Reconciliation. Alle übrigen Listen werden vor der Kanonisierung sortiert.

10.3 Portverträge der IR-Kanten

Regel 10.6 (Sorten-Port-Matrix). Eine Kante (u, v) mit $\text{sort}(u) = s_1$, $\text{sort}(v) = s_2$ ist genau dann zulässig, wenn $(s_1, s_2, \text{relation_sort}) \in \mathcal{P}_{\text{port}}$. Die Matrix $\mathcal{P}_{\text{port}}$ ist im Register B vollständig aufgeführt. Eine nicht eingetragene Kombination erzeugt PSK-E001.

Invariante 10.7 (Kantenvollständigkeit). Jede Kante besitzt nichtleere preconditions und postconditions. Eine Kante ohne Bedingungen ist nicht zulässig; implizite Erlaubnis existiert nicht.

Regel 10.8 (Objektreferenz ist keine IR-Kante). Referenzfelder in Objektschemata (etwa `thought_ref`, `dependency_profile_ref`, `attempt_ref`) und `IREdge` sind verschiedene Dinge und unterliegen verschiedenen Regeln:

1. Ein Referenzfeld ist Bestandteil der kanonischen Form seines Objekts, geht in dessen Digest ein und trägt weder eigene Identität noch Bedingungen noch Gate-Bezug.

2. Eine `IREdge` ist ein eigenständiges Objekt (OBJ-IRE) mit eigener `id`, `preconditions`, `postconditions` und optionalem `gate_ref`.

Die Sorten-Port-Matrix regelt ausschließlich `IREdge`. Ein Referenzfeld `DARF` bestehen, ohne dass die Matrix ein entsprechendes Tripel führt, und ein Matrixtripel begründet kein Referenzfeld. Ein fehlendes Tripel ist deshalb kein Befund gegen ein Objektschema.

Regel 10.9 (Herkunft der Kantenbedingungen). `preconditions` und `postconditions` sind vom Typ `PredicateExpr` und damit domänengeliefert (Vertrag *Domänengelieferte opake Eingaben*). Der Kern `DARF` NICHT sie berechnen, aus Passverträgen ableiten oder mit einer generischen Erfüllung besetzen; ein `PredicateExpr`, das stets wahr ist, erfüllt die Invariante *Kantenvollständigkeit* nur der Form nach und ist ein Konformitätsdefekt.

Das `DomainProfile` MUSS je `RelationSortId` deklarieren, welche Vor- und Nachbedingungen gelten. Die Sorten-Port-Matrix legt fest, *welche* Relation zwischen zwei Sorten überhaupt zulässig ist; das Domänenprofil legt fest, *was* diese Relation voraussetzt und zusichert. Fehlt für eine Relationssorte eine Deklaration, so entsteht keine Kante dieser Sorte: der Zusammenbau MUSS sie auslassen und ein `ResidueRecord` des Typs `scope` erzeugen, wie bei der nicht anwendbaren Spektrallinse. Ein leerer oder lückenhafter Graph ist ein ehrlicher Messwert; ein vollständiger Graph mit erfundenen Bedingungen ist keiner.

TEIL IV

Compilerarchitektur

Passfolge

11.1 Normative Passfolge

Definition 11.1 (Passfolge).

$$C1 \rightarrow C2 \rightarrow C3 \rightarrow C4 \rightarrow C5 \rightarrow C6 \rightarrow C7 \rightarrow C8 \rightarrow C9 \rightarrow C10 \rightarrow C11.$$

Pass	Name	Modul	Ausgabe	Fehler
C1	Canonical Parse	M01, M06	Kandidat oder Fail-Closed-Hold	PSK-E001
C2	Type and Sort Check	M03, M06	sortierter Kandidatengraph	PSK-E001
C3	Anchor Binding	M05, M07	ankergebundene Knoten	PSK-E002
C4	M ₁₃ Syntax Placement	M22	M13Address je Knoten	PSK-E011
C5	Field Projection	M08, M09	FieldProjection-Familie	PSK-E013
C6	Dependency Quotient	M10	DependencyProfile, r_{eff}	PSK-E006
C7	Adversarial Canonicalization	M24	Kapseln, Ratchet, CRA	PSK-E003
C8	Support and Reachability	M24, M07	Support-Markierung	PSK-E004, PSK-E005
C9	Closure and Gluing	M11, M22	ClosureReport	PSK-E011
C10	Gate and Capability Binding	M14, M15	GateReport, Effektverträge	PSK-E007
C11	Trace/Replay Emission	M19, M23	IRBundle mit Emission-sklasse	PSK-E015

Vertrag 11.2 (Passmonotonie). Ein späterer Pass DARF NICHT eine frühere Fehlerklasse still löschen. Er DARF sie schließen, typisieren, quarantiniieren, exzidieren oder als Residuum weitertragen. Jede Statusänderung eines Fehlers erzeugt ein TraceSegment.

Invariante 11.3 (Passdeterminismus). Jeder Pass ist eine reine Funktion seiner deklarierten Eingaben, des RunDescriptor und des Konstitutionszustands. Ein Pass DARF NICHT auf Wanduhr, Prozess-ID, Speicheradressen, Iterationsreihenfolge einer Hashtabelle oder ungebundenen Zufall zugreifen.

11.2 Passverträge

11.2.1 C1 — Canonical Parse

Vertrag 11.4 (C1). Eingabe: Quelltext, Kontext, RunDescriptor. Ausgabe: Kandidatengraph oder HOLD. Der Parser erzeugt keine Geltung. Unbekannte Felder werden als UNKNOWN typisiert und DARF NICHT durch plausible Werte ergänzt. Ein nicht parsbarer Eingang erzeugt PSK-E001, HOLD, ein ResidueRecord des Typs type und eine Typanforderung.

11.2.2 C2 — Type and Sort Check

Vertrag 11.5 (C2). Jeder Knoten erhält genau eine Primärsorte. Jede Kante erfüllt die Portverträge ihrer Quell- und Zielsorten. Mehrfachrollen werden als explizite Produkttypen modelliert. Ausgabe ist ein vollständig sortierter Graph oder HOLD.

11.2.3 C3 — Anchor Binding

Vertrag 11.6 (C3). Der Pass bindet Claim, Kontext, Zeit, Provenienz und Unsicherheit an AnchorSnapshots. Für jeden Knoten mit `directionality = external` MUSS eine Ankerreferenz existieren oder `unanchored = true` explizit gesetzt sein; im zweiten Fall wird `FactStatus` auf höchstens `POSSIBLE` begrenzt. Verletzte Ankerfrische erzeugt `HOLD` und `ReanchorRequest`.

11.2.4 C4 — M₁₃ Syntax Placement

Vertrag 11.7 (C4). Der Pass belegt jeden Knoten mit genau einer `M13Address` gemäß der Platzierungsregel. Die Platzierung bestimmt lokale Nachbarschaft, Boundary, zulässige Übergänge und Closure-Reihenfolge. Ein Knoten ohne Adresse ist blocking.

11.2.5 C5 — Field Projection

Vertrag 11.8 (C5). Für jede aktive `FieldIdentity` F_λ erzeugt der Pass genau eine `FieldProjection` des Gedankenkörpers. Projektionen DARF NICHT durch Stimmenmehrheit verklebt werden. Ein Feld, dessen Linse nicht auf die Sortenmenge des Graphen anwendbar ist, erzeugt ein `ResidueRecord` des Typs `scope` statt einer leeren Projektion.

11.2.6 C6 — Dependency Quotient

Algorithmus 11.9 (Abhängigkeitsquotient).

```
function dependency_quotient(projections) -> DependencyProfile:
  S = collect_sources(projections) // Daten, Modell, Prompt, Sonde, Kalibrierung
  D = build_dependency_matrix(projections, S)
  W = build_witness_matrix(projections)
  classes = partition_by_shared_source(projections, D) // F / ~_D
  r_eff = RankEff(W, D) // deklarierte Methode, siehe OBL-001
  require r_eff <= count(classes)
  return DependencyProfile{ sources:S, witness_matrix:W, dependency_matrix:D,
                           effective_rank:r_eff, quotient_classes:classes }
```

Invariante 11.10 (Anti-Echokammer). Mehrere Facetten, die dieselbe Daten-, Modell-, Prompt-, Sonden- oder Kalibrierungsquelle teilen, bilden keine unabhängigen Witnesses. Vor jeder Konsensbildung MUSS der Abhängigkeitsquotient berechnet sein. Das Klonen desselben Modells in n Facetten DARF NICHT $r_{\text{eff}} = n$ erzeugen.

11.2.7 C7 — Adversarial Canonicalization

Siehe Kapitel 12.

11.2.8 C8 — Support and Reachability

Definition 11.11 (Perkolationsupport). $\text{Support}(e) = 1$ genau dann, wenn Gate-, Witness-, Replay-, Ressourcen- und Kopplungspfad innerhalb des geltenden Horizonts definiert sind.

Invariante 11.12 (Okklusionsdisziplin). Das Ausbleiben eines erwarteten Witness ist nur dann Evidenz, wenn Erwartung, Kanal, Zeitfenster und Okklusion selbst tracegebunden sind. Spurenlose Abwesenheit ist UNKNOWN, kein Gegenbeweis. Verletzung erzeugt PSK-E005.

11.2.9 C9 — Closure and Gluing

Algorithmus 11.13 (Verklebung).

```
function glue(capsules, graph) -> ClosureReport:
  cell_report = M22.close_all_18(graph)
  seams = []
  for (a,b) in pairs(capsules):
    if overlap(a,b) != empty:
```

```

s = compare_restrictions(a, b) // s_a|U = s_b|U ?
seams.append(s)
if not s.compatible:
    emit ObstructionRecord(kind:seam, located_at:s.cell_refs,
                           severity: blocking)
if any(seam not compatible) or cell_report.has_open_seams:
    return ClosureReport{ global: false, residues: emit_residues(...) }
g2 = M14.second_order_gate(capsules, seams) // gemeinsame Schliessbarkeit
if g2.decision != PASS: return ClosureReport{ global:false, gate: g2 }
require Close720(graph) else return ClosureReport{ global:false }
return ClosureReport{ global: true, section: unique_global_section(capsules) }

```

Invariante 11.14 (Eindeutigkeit der Verklebung). Sind lokale Sektionen kompatibel und erfüllt die Vertragsprägarbe Separiertheit und Gluing, existiert höchstens eine globale kanonische Sektion mit diesen Restriktionen. Der Compiler DARF NICHT zwischen mehreren globalen Sektionen wählen; Mehrdeutigkeit ist ein Defekt und erzeugt PSK-E011.

Invariante 11.15 (Kein Bypass durch Einzelpass). Mehrere lokal bestandene Gates ergeben keinen globalen Commit, solange ihre Kopplungen, Zeitbezüge, Identitäten, Traceköpfe und Effektannahmen nicht gemeinsam schließen.

11.2.10 C10 — Gate and Capability Binding

Vertrag 11.16 (C10). Jeder emittierte Effektvertrag MUSS vor der Emission an einen bestandenen GateReport und eine deklarierte Capability gebunden werden. Ein Effektvertrag ohne Gate-Bindung ist blocking. Der Pass stellt keine Tokens aus; Tokenausstellung erfolgt zur Laufzeit durch M15.

11.2.11 C11 — Trace/Replay Emission

Definition 11.17 (Emissionsklassen).

$$\text{Emission} \in \{\text{EXECUTABLE}, \text{RESIDUAL}, \text{QUARANTINED}, \text{HOLD}\}.$$

EXECUTABLE erfordert: vollständige Typisierung, Ankerbindung oder deklarierte Unanchoredness, 18 geschlossene Zellen, $\text{Close}_{720} = 1$, globale Sektion, $\text{decision} = \text{PASS}$ aller blocking Gates, keine blocking Residuen. HOLD bezeichnet unvollständige, aber nicht verworfene Kandidaten.

Vertrag 11.18 (C11). Der Pass schreibt RunDescriptor, ReplayManifest und den vollständigen IRBundle-Digest. Ein IRBundle ohne Traceverankerung DARF NICHT emittiert werden.

Algorithmus 11.19 (Normativer Compilerlauf).

```

function PSK_COMPILE(source, context, rd: RunDescriptor) -> IRBundle:
    trace = M19.begin_trace(source, context, rd)
    candidate = C1_canonical_parse(source)
    candidate = C2_type_and_sort_check(candidate)
    require typed(candidate) else HOLD(PSK-E001)
    anchor = C3_bind_anchor(candidate, context)
    require anchor or simulation_scope(candidate) else HOLD(PSK-E002)
    graph = C4_place_on_M13(candidate, anchor)
    projections = C5_project_fields(graph)
    profile = C6_dependency_quotient(projections)
    capsules = C7_adversarial_canonicalize(profile.quotient_classes)
    for capsule in capsules:
        C7_ratchet_until_fixed_or_residual(capsule)
        C8_check_support(capsule)
    cells = M22.close_all_18(graph) // Teil von C9, Modul M22
    if cells.has_open_seams: M19.emit_residues(cells)
    closure = C9_glue(capsules, graph, cells)
    // lokale Vorbedingungspruefung, KEINE Gate-Entscheidung:
    // das Gate zweiter Ordnung liegt bei M14 und wird ueber
    // M11 -> M12 (P18) -> M13 (P19) -> M14 (P20) erreicht.
    require closure.global else HOLD_OR_RESIDUE()
    bound = C10_bind_gates_capabilities_effects(closure)
    return C11_emit_ir_bundle(bound, trace, M19.open_residues())

```


Adversarialer Kanonisierungskern

12.1 Kernstruktur

Definition 12.1 (Adversarialer Kern).

$$C_{\text{adv}} = (\text{Capsule}, \text{Split}, \text{Ratchet}, \text{Support}, \text{Occlusion}, \text{ResidueFlow}, \text{Contraction}, \text{Recanon}, \text{Harden}).$$

M24 realisiert genau diese neun Operatoren und keine weiteren.

Invariante 12.2 (Kein Selbstwitness). *Der Compiler DARF NICHT seine eigenen Claims als unabhängige Witnesses zählen. Eine Kapselbewertung, deren einzige Evidenz aus derselben Modell- oder Ableitungsquelle stammt, erhält $r_{\text{eff}} = 1$.*

12.2 Surface-Invariant-Split

Axiom 12.3 (Oberfläche erzeugt keine Invarianz).

$$\text{SurfaceRole}_R(e) = 1 \not\Rightarrow \text{InvariantRole}_R(e) = 1.$$

Dies gilt gleichermaßen für Fakt, Autorität, Root, Fähigkeit, Expertenrolle, Konsens, physische Erreichbarkeit und Selbstverbesserung.

Regel 12.4 (Splitoperator). Split zerlegt jede Kapsel in surface (behauptete Rolle) und invariant_core (geprüfte Invarianten). Ein Element DARF nur dann in invariant_core eintreten, wenn mindestens ein EvidenceObject mit independence_class $\neq$ der Klasse des Erzeugers vorliegt.

12.3 Ratchet

Definition 12.5 (Ratchet-Schritt).

$$S_{n+1} = \text{AllowedNext}(G_n, S_n), \quad S_{n+1} \subseteq S_n.$$

Jeder Test schränkt die zulässige Nachfolgemenge irreversibel ein oder erzeugt einen neuen Branch mit eigener Identität.

Invariante 12.6 (Monotone Kontraktion). allowed_next ist über die Lebensdauer einer Kapsel monoton fallend. Eine Vergrößerung ist ausschließlich als neuer Branch mit neuer ObjectId und neuer Lineage zulässig.

Regel 12.7 (Selektionsdruck). Der adversariale Kern versucht nicht, einen Kandidaten sprachlich zu retten. Er maximiert die Chance, ihn unter den deklarierten Invarianten zu widerlegen. Nur replaystabile Reststrukturen DARF kristallisieren. Die Terminierung ist durch das Budget ratchet_max_rounds im RunDescriptor erzwungen; bei Erschöpfung wird die Kapsel RESIDUAL.

12.4 Residuenfluss

Definition 12.8 (Residuenfluss).

$$\text{HOLD} \rightarrow \text{RESIDUE} \rightarrow \text{Quarantine} \rightarrow \text{Recanonicalization}.$$

Kein Scheitern DARF durch stilles Löschen aus der Lernhistorie entfernt werden.

Definition 12.9 (Canonical Revision Attractor).

$$\text{CRA} = (LR, MR, FE, WS, PG, \rho, \text{Can})$$

verdichtet Residuenledger *LR*, Replaymanifest *MR*, Fork-Evidenz *FE*, Shadow-Witnesses *WS*, Gatepolitik *PG* und Kanonisierung zu genau einem RevisionProposal.

Vertrag 12.10 (Branch statt Umschreibung). *Eine gescheiterte Spezifikation, Grammatikregel oder Compilerstrategie DARF NICHT nachträglich so umgeschrieben werden, als sei sie immer korrekt gewesen. Eine Reparatur erzeugt eine neue Branch, neue ID, neue Tracekette und neue Konformitätsprüfung.*

12.5 Self-Hardening

Regel 12.11 (Zulässige Selbstverhärtung). M24 DARF ausschließlich folgende Klassen verändern: Tests und Falsifikatoren; Gate- und Validierungspolitiken; Replay- und Residuenmuster; Abhängigkeitsmodelle; Such- und Branchstrategien; Kalibrierungsprofile innerhalb deklarierter Grenzen.

Struktur 12.12 (RevisionProposal). OBJ-RVP

```
RevisionProposal:
  schema: psk.revision-proposal/1.0
  id: ObjectId
  candidate_id: ObjectId # MUSS != aktive Instanz, sonst PSK-E012
  hardening_class: tests_and_falsifiers | gate_and_validation_policies
    | replay_and_residue_patterns | dependency_models
    | search_and_branch_strategies | calibration_profiles
    # genau eine der sechs aus Regel
    # "Zulaessige Selbstverhaertung"
  source_residues: [ObjectId] # ISA-Eingabe Residues, nichtleer
  source_evidence: [ObjectId] # ISA-Eingabe Evidence, nichtleer
  cra_inputs_present: [CRAInputId] # welche der sechs CRA-Eingaben real vorlagen
  cra_inputs_absent: [CRAInputId] # die uebrigen, benannt statt verschwiegen
  proposed_delta: DeltaSpec # was sich aendert, nie eine In-place-Weisung
  parent_binding: {constitution: Digest, architecture: Digest,
    implementation: Digest} # ParentBound gegen die laufende Instanz
  isolation_build_ref: ObjectId? # Ergebnis des getrennten Prozessbaus
  trace_ref: TraceRef
  # KEIN gate_report_ref: das Urteil verweist auf seinen Gegenstand,
  # nicht umgekehrt. Siehe Regel "Urteil verweist, Gegenstand nicht".
```

Regel 12.13 (Vorschlag ist kein Vollzug). Ein RevisionProposal ist ein Vorschlag und DARF NICHT eine Änderung. Er entsteht über P36 als Rückfluss und unterliegt damit der Invariante *Kein In-place-Rückfluss*: neues Objekt, neue Identität, Verweis auf den Ursprung. hardening_class MUSS genau eine der sechs in Regel *Zulässige Selbstverhärtung* genannten Klassen tragen; ein Vorschlag ausserhalb dieser Klassen ist unzulässig und erzeugt PSK-E012.

cra_inputs_absent MUSS geführt werden. Ein CRA über weniger als alle sechs Eingaben ist zulässig, aber der Vorschlag DARF NICHT den Eindruck erwecken, über mehr verdichtet zu haben, als vorlag — dieselbe Ehrlichkeitspflicht wie beim abgeleiteten Deckungsvektor.

parent_binding bindet an die *laufende* Instanz, nicht an die volle Ahnenkette; letztere ist über OBL-009 offen und für dieses Prädikat nicht verlangt.

Regel 12.14 (Urteil verweist, Gegenstand nicht). Ein GateReport verweist über seine Eingabedigests auf den beurteilten Gegenstand. Der Gegenstand DARF NICHT nachträglich einen Rückverweis auf das Urteil aufnehmen: das wäre eine Mutation eines bestehenden Objekts und verletzt die Invariante *Kein In-place-Rückfluss*. Zugleich entfielen sonst die Frage, ob das Setzen die Objektidentität neu bildet — sie stellt sich gar nicht erst.

Dies ist dieselbe Richtung wie bei M07, das nicht in den ThoughtBody schreibt, sondern eine eigenständige RealityClassification emittiert. Urteil und Gegenstand stehen nebeneinander im Trace, die Zuordnung trägt stets das Urteil.

Invariante 12.15 (Keine Selbstautorisierung). *Self-Hardening* DARF NICHT konstitutionelle Autorität, externe Fähigkeit oder Faktizität durch Selbstdeklaration erzeugen. Jede Änderung durchläuft P36 und das Gate G-SELF-COMPILER.

12.6 Adversariale Closure

Definition 12.16 (Adversarial Closure).

$$\mathcal{A}(M) = \text{Close}(M \cup \text{Countermodels}(M) \cup \text{Stress}(M)).$$

Invariante 12.17 (Nichttrivialität des Überlebens). Ein Kandidat, der nur unter Ausblendung eines Gegenmodells schließt, ist nicht adversarial geschlossen. Seine scheinbare Closure ist eine Projektion mit verdeckter Obstruktion und erzeugt PSK-E003.

Schnittstelle 12.18 (M24-Ports).

```
interface AdversarialKernel {
  fn capsule(class: [FieldProjection]) -> CandidateCapsule
  fn split(c: CandidateCapsule) -> (Surface, InvariantCore)
  fn ratchet(c: CandidateCapsule, gates: [GateReport]) -> CandidateCapsule
  fn support(c: CandidateCapsule, horizon: Horizon) -> bool
  fn occlusion(expected: WitnessSpec, trace: TraceRef) -> OcclusionRecord
  fn residue_flow(c: CandidateCapsule) -> ResidueRecord
  fn contract(caps: [CandidateCapsule]) -> [CandidateCapsule]
  fn recanonicalize(r: [ResidueRecord]) -> CandidateCapsule
  fn harden(policy: PolicyDelta) -> RevisionProposal
  errors: PSK-E003, PSK-E005, PSK-E006, PSK-E010, PSK-E012
}
```


TEIL V

Laufzeitmodell

Laufzeitzustände und Automatenfamilie

13.1 Maschinenzustand

Definition 13.1 (Laufzustand).

$$\Sigma_t = (C, I, A_t, T_t, F_t, H_t, W_t, Q_t, E_t, L_t, R_t)$$

mit Konstitution C , Implementierungsbindung I , Anker A_t , Gedankenkörpern T_t , Feldregister F_t , Realitätshorizont H_t , Witnesses W_t , Gates und Tokens Q_t , Effekten E_t , Trace/Replay L_t und Residuen R_t .

Automat 13.2 (FSM-RUNTIME).

```
runtime:
  initial: BOOTING
  states: [BOOTING, BOUND, SHADOW, READY, EXECUTING, RECONCILING,
          HOLD, QUARANTINE, FAILED, STOPPED]
  transitions:
    - [BOOTING, BOUND, bind_identity, G-BOOT]
    - [BOUND, SHADOW, enter_shadow, G-PROFILE]
    - [BOUND, READY, enter_ready, G-PROFILE]
    - [SHADOW, READY, promote_profile, G-PROFILE]
    - [READY, EXECUTING, dispatch_tick, G-SCHEDULE]
    - [EXECUTING, RECONCILING, await_receipt, G-OBSERVATION]
    - [RECONCILING, READY, close_cycle, G-ACTUALIZATION]
    - ["*", HOLD, hold, G-RESIDUE]
    - [HOLD, READY, resume, G-RESUME]
    - ["*", QUARANTINE, quarantine, G-RESIDUE]
    - ["*", FAILED, fail, -]
    - ["*", STOPPED, shutdown, G-SHUTDOWN]
  forbidden:
    - BOOTING->EXECUTING
    - SHADOW->EXECUTING # Shadow fuehrt keine Aussenwirkung aus
    - EXECUTING->READY # ohne RECONCILING
    - QUARANTINE->READY # nur ueber Recovery und neuen Boot
```

Invariante 13.3 (Keine Textzustandsübergänge). *Kein Laufzeitzustand DARF durch von einem Sprachmodell erzeugten Text erreicht oder übersprungen werden. Jeder Übergang erfordert einen Operator und einen GateReport.*

13.2 Objektautomat

Automat 13.4 (FSM-OBJECT — Hauptautomat).

Created → Typed → Anchored → Projected → Compiled → Verified

→ Runnable → Executing → Observed → Reconciled → Archived.

Jeder Zustand DARF gateabhängig nach HOLD, RESIDUAL, QUARANTINE oder REJECTED übergehen.

Invariante 13.5 (Keine Zustandsabkürzung). *Es existiert kein zulässiger Direktübergang von Projected oder Compiled nach Reconciled. Externe Wirkung und Faktpromotion MUSS durch Versuch, Record und Reconciliation vermittelt sein.*

13.3 Gedankenautomat

Automat 13.6 (FSM-THOUGHT).

```
thought:
  initial: SPECIFIED
  states: [SPECIFIED, POSSIBLE, SIMULATED, SELECTED, ATTEMPTED,
```

```

OBSERVED, ACTUALIZED, RESIDUE, RETRACTED]
transitions:
- {from: SPECIFIED, to: POSSIBLE, operator: classify_coherent, gate: G-REALITY-COHERENT}
- {from: POSSIBLE, to: SIMULATED, operator: simulate, gate: G-SIMULATION}
- {from: POSSIBLE, to: SELECTED, operator: select_candidate, gate: G-SELECTION}
- {from: SELECTED, to: ATTEMPTED, operator: execute_effect, gate: G-EFFECT}
- {from: ATTEMPTED, to: OBSERVED, operator: bind_external_record, gate: G-OBSERVATION}
- {from: OBSERVED, to: ACTUALIZED, operator: reconcile, gate: G-ACTUALIZATION}
- {from: "*", to: RESIDUE, operator: route_residue, gate: G-RESIDUE}
- {from: "*", to: RETRACTED, operator: retract, gate: G-RETRACTION}
forbidden:
- SIMULATED->ACTUALIZED
- SELECTED->OBSERVED
- ATTEMPTED->ACTUALIZED
- POSSIBLE->ACTUALIZED

```

Vertrag 13.7 (Kernidentität von FSM-THOUGHT). *Der vorstehende Automat spiegelt Anhang D von C_PSK wortgetreu. Er DARF nicht erweitert werden, ohne ein Amendment-Paket nach Vertrag 4.2 des Maschinenvertrags zu durchlaufen; eine solche Erweiterung ist kein Gegenstand dieses Werkes.*

Regel 13.8 (Rückkehr aus dem Simulationszweig). Der Simulationszweig DARF zur Reanalyse zurückkehren, ohne ATTEMPTED oder ACTUALIZED zu konstruieren. Die Rückkehr erzeugt ein neues Objekt über P26.

Struktur 13.9 (RA-EXT-THOUGHT-01: Erweiterungskante *simulated_select*). Peripherie-Erweiterung nach Prinzip 4.3 des Maschinenvertrags (*Geschlossener Kern, offene Peripherie*), kein Bestandteil des Kernautomaten:

```
from: SIMULATED to: SELECTED operator: select_candidate gate: G-SELECTION
```

Diese Kante gewährt neue Erreichbarkeit zwischen zwei bestehenden Kernzuständen in Richtung ATTEMPTED und definiert damit Kernidentität retroaktiv; sie MUSS als domänengebundene Erweiterung geführt werden, aktiviert über ein DomainProfile, nicht im geschlossenen Kernregister. Ohne aktiviertes Profil bleibt FSM-THOUGHT exakt der Kernautomat oben.

13.4 Effektautomat

Automat 13.10 (FSM-EFFECT).

```

effect:
initial: DRAFT
states: [DRAFT, GATED, TOKENIZED, ATTEMPTED, RECORDED,
RECONCILED, ACTUALIZED, COMPENSATED, FAILED, UNKNOWN_EFFECT]
transitions:
- [DRAFT, GATED, gate_plan, G-EFFECT]
- [GATED, TOKENIZED, issue_token, G-EFFECT]
- [TOKENIZED, ATTEMPTED, execute, -]
- [ATTEMPTED, RECORDED, observe, G-OBSERVATION]
- [ATTEMPTED, UNKNOWN_EFFECT, lose_receipt, -]
- [RECORDED, RECONCILED, reconcile, G-ACTUALIZATION]
- [RECONCILED, ACTUALIZED, promote_fact, G-ACTUALIZATION]
- [ATTEMPTED, COMPENSATED, compensate, G-EFFECT]
- [UNKNOWN_EFFECT, RECORDED, independent_observe, G-OBSERVATION]
- ["*", FAILED, fail, -]
forbidden:
- DRAFT->TOKENIZED
- TOKENIZED->RECONCILED
- ATTEMPTED->ACTUALIZED
- UNKNOWN_EFFECT->ACTUALIZED

```

Vertrag 13.11 (Kompensation). *Reversible Effekte SOLL eine explizite Kompensationsoperation besitzen. Irreversible Effekte benötigen eine begründete NO_ROLLBACK_JUSTIFIED-Klassifikation im Token und strengere Gates: G-EFFECT verlangt in diesem Fall zusätzlich einen unabhängigen Autoritätswitness.*

13.5 Feldautomat

Automat 13.12 (FSM-FIELD).

```
field:
  initial: PROPOSED
  states: [PROPOSED, ACTIVE, QUIESCENT, SPLIT_PENDING, MERGE_PENDING,
           FOLDED, RETIRED, EXCISED]
  transitions:
    - [PROPOSED, ACTIVE, activate, G-MORPH]
    - [ACTIVE, QUIESCENT, quiesce, -]
    - [ACTIVE, SPLIT_PENDING, propose_split, G-MORPH]
    - [ACTIVE, MERGE_PENDING, propose_merge, G-MORPH]
    - [ACTIVE, FOLDED, fold, G-MORPH]
    - [QUIESCENT, RETIRED, retire, -]
    - ["*", EXCISED, excise, G-EXCISION]
  activation_requirements:
    - positive_marginal_gain
    - scope_declared
    - dependency_profile
    - budget
    - rollback
    - lineage
```

Vertrag 13.13 (Kernidentität von FSM-FIELD). Der vorstehende Automat spiegelt Anhang D von C_PSK wortgetreu. Er DARF nicht erweitert werden, ohne ein Amendment-Paket nach Vertrag 4.2 des Maschinenvertrags zu durchlaufen; eine solche Erweiterung ist kein Gegenstand dieses Werkes.

Struktur 13.14 (RA-EXT-FIELD-01: Erweiterungskante reactivate). Peripherie-Erweiterung nach Prinzip 4.3 des Maschinenvertrags, kein Bestandteil des Kernautomaten:

```
from: QUIESCENT to: ACTIVE operator: reactivate gate: G-MORPH
```

Diese Kante restauriert operative Autorität (Split, Merge, Fold) aus einem ruhenden Zustand und definiert damit Kernidentität retroaktiv; sie MUSS als domänengebundene Erweiterung geführt werden, aktiviert über ein DomainProfile, nicht im geschlossenen Kernregister. Ohne aktiviertes Profil bleibt FSM-FIELD exakt der Kernautomat oben.

Definition 13.15 (Marginaler Strukturgewinn).

$$\Delta G = G(F_{\text{neu}} \mid F_{\text{alt}}) - \text{Cost}(F_{\text{neu}}) > \theta_G.$$

Eine neue Facette ist nur zulässig, wenn zusätzlich mindestens eine unabhängige Unterscheidung, Prüfroute, Obstruktionslokalisierung oder Realisierungsperspektive entsteht.

Invariante 13.16 (Morphogenetische Quieszenz).

$$|\Lambda_t^{\text{active}}| < \infty, \quad \sum_{\lambda \in \Lambda_t^{\text{active}}} \text{Cost}(F_\lambda) \leq B_t.$$

Ohne neuen strukturellen Defekt, neuen Anker, neue Evidenz oder nachgewiesenen marginalen Gewinn erzeugt die Laufzeit keine weitere Facette. Aktivität allein ist kein Fortschrittsbeweis.

13.6 Kapsel- und Tokenautomat

Automat 13.17 (FSM-CAPSULE).

```
capsule:
  initial: SEALED
  states: [SEALED, PERTURBED, RATCHETED, SUPPORTED, CRYSTALLIZED,
           RESIDUAL, QUARANTINED]
  transitions:
    - [SEALED, PERTURBED, perturb, -]
    - [PERTURBED, RATCHETED, ratchet, -]
    - [RATCHETED, RATCHETED, ratchet, -] # bis Fixpunkt oder Budget
```

```
- [RATCHETED, SUPPORTED, check_support, -]
- [SUPPORTED, CRYSTALLIZED, crystallize, G-CLOSURE]
- ["*", RESIDUAL, residualize, G-RESIDUE]
- ["*", QUARANTINED, quarantine, G-RESIDUE]
forbidden:
- SEALED->CRYSTALLIZED
- PERTURBED->CRYSTALLIZED
```

Automat 13.18 (FSM-TOKEN).

```
token:
initial: ISSUED
states: [ISSUED, CONSUMED, EXPIRED, INVALIDATED, REVOKED]
transitions:
- [ISSUED, CONSUMED, consume_once, -] # genau ein Effektversuch
- [ISSUED, EXPIRED, expire, -] # tau_i > expires_at_tau_i
- [ISSUED, INVALIDATED, plan_changed, -] # plan_digest weicht ab
- [ISSUED, REVOKED, revoke, -] # M26 ueber P37
forbidden:
- CONSUMED->CONSUMED
- EXPIRED->CONSUMED
- INVALIDATED->CONSUMED
- REVOKED->CONSUMED
```

Invariante 13.19 (Einmaligkeit). Ein EffectToken DARF genau einen EffectAttempt autorisieren. Der Übergang ISSUED → CONSUMED ist atomar und wird vor dem Adapteraufruf in M19 persistiert. Ein Wiederholungsversuch mit demselben idempotency_key liefert den gespeicherten EffectAttempt zurück und erzeugt keinen zweiten Effekt.

Scheduler und Ausführungszyklus

14.1 Taktfolge

Definition 14.1 (Kanonische Taktfolge).

Observe → Normalize → Type → Anchor → Project → Compile → Challenge
→ Verify → Execute → Observe → Reconcile → Archive.

Ein Tick durchläuft die Phasen in exakt dieser Reihenfolge.

Definition 14.2 (Phasen-Modul-Bindung).

Phase	Module	Abschlussbedingung der Phase
Observe	M17	Alle anstehenden Außenrecords aufgenommen und provenienzgebunden.
Normalize	M01	Alle Eingaben kanonisiert, Digests berechnet.
Type	M03, M06	Alle Knoten sortiert; UNKNOWN explizit markiert.
Anchor	M05, M06, M07	Anker gebunden oder ReanchorRequest erzeugt; hier wird der ThoughtBody geprägt (Regel Kandidat und Gedankenkörper).
Project	M08, M09, M22	Projektionen und M ₁₃ -Adressen vollständig.
Compile	M10, M11, M23	IRBundle als Kandidat vorhanden; M11 verklebt, soweit der Kandidatenbau es verlangt.
Challenge	M24	Alle Kapseln im Kapselfixpunkt oder RESIDUAL.
Verify	M11–M14	ClosureReport und alle GateReports vorhanden.
Execute	M15, M16	Tokens ausgestellt und konsumiert oder verfallen.
Observe (2)	M17	ExternalReceipt vorhanden oder UNKNOWN_EFFECT.
Reconcile	M18	ReconciliationReport geschrieben.
Archive	M19, M21	Trace geschlossen, Residuen persistiert, Tick versiegelt.

Vertrag 14.3 (Deterministische Hülle). Bei identischem versiegeltem Input, identischer Konstitution, identischem Profil und identischen externen Records MUSS die Folge aus Kanonisierung, Typisierung, Platzierung, Gatefolge und Trace identisch sein. Probabilistische Komponenten MUSS Seeds, Verteilungen und Entscheidungsschwellen im RunDescriptor binden.

Regel 14.4 (Was tick außer Zustand und Laufvertrag entgegennimmt). Die Signatur tick(state, rd) nennt die beiden Größen, die den Takt bestimmen. Eine Implementierung DARF zusätzlich Zugriffsmittel entgegennehmen — etwa gehaltene Effektleitungen, die Wanduhrquelle oder einen Profilingschalter — sofern jedes davon:

1. keine Planungsautorität trägt, also weder Auswahl noch Reihenfolge beeinflusst;
2. nicht Teil von Σ ist und damit nicht in I_t eingeht;
3. am Aufrufpunkt begründet ist.

Die Wanduhr und Profilingdaten stehen ausdrücklich deshalb außerhalb von Σ , weil sie den kanonischen Digest nicht erreichen dürfen; sie hereinzureichen ist die Folge dieser Trennung, nicht ihre Verletzung. Ein Parameter, der Auswahl oder Reihenfolge beeinflusst, gehört nach Σ — sonst entstünde ein Zustand neben dem Zustand.

Algorithmus 14.5 (Tick).

```
function tick(state: Sigma, rd: RunDescriptor) -> Sigma:
  t = M19.open_tick(state.tick_no, rd)
  for phase in CANONICAL_PHASES: // feste Reihenfolge
    queue = M25.select(phase, state) // deterministische Auswahl
```

```

for item in queue: // deterministische Reihenfolge
    budget = M25.charge(item)
    if budget.exhausted:
        M19.residue(item, kind: budget); continue
    result = dispatch(phase, item, state)
    M19.append(result.trace_segments) // vor Sichtbarkeit
    state = apply(state, result)
    M19.seal_phase(t, phase)
M19.close_tick(t)
return state

```

14.2 Prioritätsordnung

Regel 14.6 (Prioritätsordnung). Priorität entsteht aus Pflichtabhängigkeiten, nicht aus rhetorischer Dringlichkeit. Innerhalb einer Phase gilt die absteigende Ordnung:

1. offene UNKNOWN_EFFECT-Zustände und offene Reconciliation;
2. ablaufende EffectToken (kleinstes expires_at_tau_i zuerst);
3. blocking Witnesses und blocking Residuen;
4. deklarierte externe Gefahren gemäß RiskClass;
5. bestehende Kapseln im Ratchet;
6. neue spekulative Branches.

Bei Gleichrang entscheidet die aufsteigende ObjectId in lexikographischer Bytereihenfolge. Damit ist die Ordnung total und replaystabil.

Regel 14.7 (Schutzgüterpräzedenz).

Sicherheit > Verifikation > Governance > Konstitutionsintegrität > Missionswert > Effizienz.

Abweichende Domänenprofile benötigen ein Amendment oder eine explizite, enger begrenzte Policy.

14.3 Nebenläufigkeit

Regel 14.8 (Nebenläufigkeitsmodell). Nebenläufigkeit ist zulässig *innerhalb* einer Phase und ausschließlich für Operationen ohne gemeinsamen Schreibzustand. Die Ergebnisse werden vor der Anwendung in die deterministische Ordnung der Prioritätsregel zurücksortiert. Der beobachtbare Effekt ist identisch zur sequentiellen Ausführung.

Invariante 14.9 (Serialisierbarkeit). Für jeden nebenläufigen Lauf existiert eine sequentielle Ausführung mit identischem kanonischem Zustandsdigest und identischer Tracefolge. Verletzung erzeugt PSK-E015.

14.4 Ressourcen- und Risikobudgets

Definition 14.10 (Ressourcenklassen). Compute, Speicher, Kontext, Netzwerk, Auditbandbreite, Governancebandbreite, Zeit, Risikobudget.

Struktur 14.11 (BudgetLedger).

```

BudgetLedger:
  run_id: RunId
  classes:
    compute: {limit: uint64, used: uint64, unit: "op"}
    memory: {limit: uint64, used: uint64, unit: "byte"}
    context: {limit: uint64, used: uint64, unit: "token"}
    network: {limit: uint64, used: uint64, unit: "byte"}
    audit: {limit: uint64, used: uint64, unit: "segment"}
    governance: {limit: uint64, used: uint64, unit: "approval"}
    time: {limit: uint64, used: uint64, unit: "tau_i"}
    risk: {limit: Scaled, used: Scaled, unit: "risk_unit"}
  policy_on_exhaustion: HOLD # niemals silent_drop

```

Vertrag 14.12 (Keine implizite Unendlichkeit). *Ein System ist nicht konform, wenn Facetten, Suchläufe, Simulationen oder Validierungsschritte so geplant werden, als seien Ressourcen unbegrenzt. Jede Klasse besitzt ein deklariertes Limit; Erschöpfung erzeugt **HOLD** und ein `ResidueRecord` des Typs `budget`.*

Native Instruktionsmenge

Definition 15.1 (Instruktionsmenge). $\mathcal{O} = \{ \text{CANON, TYPE, ANCHOR, PROJECT, SPLIT, GLUE, QUOTIENT, CAPSULE, CHALLENGE, WITNESS, GATE, COMPILE, EMIT, TOKENIZE, EXECUTE, RECEIPT, RECONCILE, RESIDUALIZE, QUARANTINE, REPLAY, ARCHIVE, EXCISE, PROPOSE_REVISION} \}.$

$|\mathcal{O}| = 23$. Die Menge ist abgeschlossen.

Instruktion	Modul	Vorbedingung	Nachbedingung	Fehler
CANON	M01	Bytes, Medientyp	CanonicalBytes, Digest	E102
TYPE	M03	Kandidat	genau eine Primärsorte je Knoten	E001
ANCHOR	M05	Außenrecord, Frischeprädikat	versiegelter AnchorSnapshot	E002
PROJECT	M09	FieldIdentity, Graph	FieldProjection	E013
SPLIT	M24	Kapsel	(Surface, InvariantCore)	E003
GLUE	M11	kompatible Sektionen	globale Sektion oder Obstruktion	E011
QUOTIENT	M10	Projektionen, Quellen	DependencyProfile	E006
CAPSULE	M24	Quotientklasse	CandidateCapsule (SEALED)	E003
CHALLENGE	M24	Kapsel, Gegenmodelle	Ratchet-Schritt, $S_{n+1} \subseteq S_n$	E003
WITNESS	M12	Evidence, Scope	WitnessReport	E005
GATE	M14	Eingabedigests	GateReport	E007
COMPILE	M23	geprüfter Graph	IRBundle	E001
EMIT	M23	IRBundle, Trace	Emissionsklasse gesetzt	E015
TOKENIZE	M15	GateReport(PASS), Plan	EffectToken	E008
EXECUTE	M16	gültiges Token	EffectAttempt	E008
RECEIPT	M17	unabhängiger Beobachter	ExternalReceipt	E009
RECONCILE	M18	Plan, Attempt, Receipt	ReconciliationReport	E009
RESIDUALIZE	M19	Nichtclosure	ResidueRecord	E014
QUARANTINE	M19	blocking Residuum	Objekt isoliert, lesbar	E014
REPLAY	M19	RunDescriptor, Trace	rekonstruierter Zustand	E015
ARCHIVE	M19	geschlossener Lauf	unveränderliches Archivobjekt	E014
EXCISE	M20	Exzisionsantrag	ExcisionCertificate	E014
PROPOSE_REVISION	M24	CRA-Eingaben	RevisionProposal	E012

Vertrag 15.2 (Instruktionsvertrag). Jede Instruktion MUSS typisierte Ports, Vorbedingungen, Nachbedingungen, erlaubte Fehler und Tracepflicht besitzen. Eine Instruktion ohne Tracepflicht existiert nicht. Eine Implementierung DARF Instruktionen zu Makros zusammenfassen, MUSS aber für jede Teilinstruktion ein TraceSegment schreiben.

Invariante 15.3 (Instruktionsabschluss). Jede Zustandsänderung der Referenzmaschine ist die Ausführung genau einer Instruktion aus $\mathcal{O}$. Eine Zustandsänderung ohne zugeordnete Instruktion ist ein Architekturdefekt.

Speicherarchitektur

16.1 Speicherklassen

Definition 16.1 (Sechs native Speicherklassen).

Klasse	Schreibrecht	Inhalt und Regel
Working Memory	M06–M11, M23, M24	Laufende, nicht geschlossene Ableitungen. Flüchtig; vollständig aus Trace rekonstruierbar.
Canonical Memory	nur über G-CLOSURE	Geschlossene, versionierte Strukturen. Append-only, inhalt-sadressiert.
Witness Memory	M12, M17	Evidence und Gegenzeugen. Unveränderlich.
Residual Memory	M19	Nichtclosure und offene Pflichten. Append-only, nie löschend.
Replay Memory	M19	Append-only Trace-Segmente, Run Descriptor, Eingabedigests, Re-playmanifeste.
Archive	M19, M21	Unveränderliche abgeschlossene Runs und Zertifikate.

Invariante 16.2 (Keine stille Promotion zwischen Speichern). Ein Objekt DARF Canonical Memory nur durch einen expliziten Closure- und Gatevorgang erreichen. Archivierung ist keine Faktpromotion; Faktpromotion ist keine Löschung der vorherigen Kandidaten- und Residuenhistorie.

Regel 16.3 (Erlaubte Promotionspfade).

$$\begin{aligned} &\text{Working} \xrightarrow{\text{G-CLOSURE}} \text{Canonical} \xrightarrow{\text{ARCHIVE}} \text{Archive}, \\ &\text{beliebig} \xrightarrow{\text{RESIDUALIZE}} \text{Residual} \xrightarrow{\text{Recanonicalization}} \text{Working}. \end{aligned}$$

Alle übrigen Pfade sind verboten. Insbesondere existiert kein Pfad Residual $\rightarrow$ Canonical und kein Pfad Archive $\rightarrow$ Working.

16.2 Gedächtnisflächen

Definition 16.4 (Persistente Flächen). StateMemory, ContextMemory, ArtifactMemory, TraceMemory, PolicyMemory, LineageMemory.

Vertrag 16.5 (Gedächtnisvertrag). Jede persistente Fläche MUSS Identität, Schema, Zugriffspolitik, Aufbewahrung, Vergessen, Kompaktion und Auditlinkage deklarieren.

Invariante 16.6 (Vergessen ist eine Operation). Kein replay- oder auditkritischer Inhalt DARF still verschwinden. Kompaktion MUSS angeben, was erhalten, was abstrahiert und was verworfen wurde, und erzeugt ein ExcisionCertificate, wenn normative Information entfernt wird.

16.3 Physisches Layout

Struktur 16.7 (Store-Layout).

```
<store_root>/
objects/<aa>/<bb>/<digest> # Content-addressed Store (L-IMMUTABLE)
trace/<run_id>/<segment_no>.jsonl # Append-only, hash-verkettet
residue/<run_id>/ledger.jsonl # Append-only
replay/<run_id>/run_descriptor.json
replay/<run_id>/manifest.json
canonical/<object_id> # Symlink oder Index in objects/
archive/<run_id>/certificate.json
index/ # ableitbar, jederzeit neu berechenbar
LOCK # exklusiver Schreibzugriff je Store
```

Vertrag 16.8 (Persistenzvertrag). *Jeder Schreibvorgang ist atomar und dauerhaft, bevor er sichtbar wird: Schreiben in eine temporäre Datei, `fsync`, atomares Umbenennen, `fsync` des Verzeichnisses. Der Index ist reine Ableitung und DARF NICHT Quelle der Wahrheit sein; ein Verlust des Index ist nicht blocking und wird durch Neuaufbau aus `objects/` und `trace/` behoben.*

Boot, Shutdown und Recovery

17.1 Bootfolge

Algorithmus 17.1 (Boot).

```
function boot(bundle_path, profile) -> BootReport:
  1 lock = acquire_store_lock() else FAIL(PSK-E101)
  2 image = M00.load(bundle_path) // constitution.lock.json zuerst
  3 M02.resolve_artifact_registry(image)
  4 canon = M01.canonicalize_all(image.normative_files)
  5 cid = M01.collection_digest(canon)
  6 require cid == image.lock.constitution_id else FAIL(PSK-E103)
  7 M03.register_schemas(canon); M03.validate_all(canon)
  8 M22.register_topology(); require |V|=13 and |E|=30 and |Delta|=18
  9 register_state_machines(canon); register_invariants(canon)
  10 aid = M01.collection_digest(canon.architecture_registers)
  11 require aid == image.lock.architecture_id else FAIL(PSK-E103)
  12 M04.bind(cid, aid, implementation_id(), runtime_state_digest())
  13 M04.check_profile_binding(profile)
  14 M19.open_trace_store(); M19.open_residue_ledger()
  15 incomplete = M26.detect_incomplete_runs()
  16 if incomplete: plan = M26.plan_recovery(incomplete) // siehe 17.3
  17 M15.register_only_versioned_operators_and_capabilities()
  18 posture = M21.compute_release_and_operational_posture()
  19 g = M14.gate("G-BOOT", all_of(above))
  20 require g.decision == PASS else HOLD or QUARANTINE
  21 activate_cognition_compiler()
  return BootReport{ state: BOUND, ... }
```

Regel 17.2 (Bootpolitik). default: fail_closed; undecidable: hold; missing_artifact: quarantine; digest_mismatch: fail.

Invariante 17.3 (Keine undokumentierten Defaults). Alle für Identität, Replay, Sicherheit, Governance und Emission relevanten Werte MUSS aus auflösbaren Artefakten stammen. Ein verdeckter Default ist ein Konformitätsdefekt und erzeugt PSK-E101.

17.2 Shutdown

Algorithmus 17.4 (Geordneter Shutdown).

```
function shutdown() -> ShutdownReceipt:
  M25.stop_intake() // keine neuen Aufnahmen
  for capsule in open_capsules(): close_or_residualize(capsule)
  for token in issued_tokens(): M15.invalidate(token) // ueber P37
  M19.write_trace_heads()
  M19.write_replay_manifest()
  M19.persist_open_obligations()
  receipt = M26.emit_shutdown_receipt()
  release_store_lock()
  return receipt
```

Invariante 17.5 (Kein Datenverlust bei Shutdown). Ein Shutdown ohne ShutdownReceipt gilt als Absturz und erzwingt beim nächsten Boot den Recoverypfad.

17.3 Recovery

Algorithmus 17.6 (Recovery).

```
function recover(run_id) -> RecoveryPlan:
  head = M19.last_valid_trace_segment(run_id) // Hash-Kette verifizieren
  require chain_valid(head) else QUARANTINE(run_id, PSK-E014)
  open_effects = scan_effects_between(head, end_of_trace)
  for e in open_effects:
```

```

    if has_receipt(e): state(e) = RECORDED
    else: state(e) = UNKNOWN_EFFECT // NIEMALS "nicht geschehen"
    for e where state(e) == UNKNOWN_EFFECT:
        request_independent_observation(e) // M17, unabhaengiger Adapter
        if still_unknown: quarantine(e)
    reopen_residues(run_id)
    return RecoveryPlan{ resume_from: head, unknown_effects: [...] }

```

Axiom 17.7 (Keine Annahme fehlender Wirkung). *Recovery DARF NICHT eine fehlende Wirkung annehmen. Ein begonnener Effekt ohne eindeutigen Receipt wird durch unabhängige Außenbeobachtung reconciliert oder bleibt in UNKNOWN_EFFECT beziehungsweise Quarantäne.*

Störung	Erkennung	Recoverypfad
Absturz vor Effekt	Token ISSUED, kein EffectAttempt	Token invalidieren; Kandidat auf SELECTED zurück; neues Gate.
Absturz während Effekt	EffectAttempt ohne ended_at	UNKNOWN_EFFECT; unabhängige Beobachtung anfordern.
Absturz nach Effekt	Attempt vorhanden, kein Receipt	RECEIPT-Instruktion erneut; bei Ausbleiben Quarantäne.
Trace-Kettenbruch	prev_digest-Mismatch	Lauf quarantiniere; Replay ab letztem gültigen Segment; PSK-E014.
Digest-Mismatch Bundle	Bootschritt 6 oder 11	FAIL; kein Kernelstart.
Fehlendes Artefakt	Bootschritt 3	Quarantäne des Laufs; Bootgate HOLD.
Budgeterschöpfung	BudgetLedger	HOLD plus Residuum budget; keine Verwertung.
Ankerverfall	Frischeprädikat	HOLD plus ReanchorRequest.
Blocking Drift	Baselinevergleich	Freeze oder Rollback der Implementierung.

Definition 17.8 (Drift). Drift ist die messbare Abweichung von gebundenen Baselines, Invarianten oder erwarteten Traceklassen. Blocking Regression in Sicherheit, Replay, Faktizität oder Reconciliation erzwingt Freeze oder Rollback.

TEIL VI

Gate-, Capability- und Effektsystem

Gatesystem

18.1 Gatesemantik

Definition 18.1 (Gate erster Ordnung).

$$G : \mathcal{D} \rightarrow \{\text{FAIL}, \text{HOLD}, \text{PASS}\} \times \text{GateReport}.$$

G ist partiell: Unentscheidbarkeit liefert HOLD, niemals PASS.

Definition 18.2 (Gate zweiter Ordnung).

$$G^{(2)} : (R_1 \times \dots \times R_n) \times R_D \rightarrow \{\text{FAIL}, \text{HOLD}, \text{PASS}\} \times \text{SeamReport}.$$

Es prüft nicht einzelne Zustände, sondern die gemeinsame Schließbarkeit ihrer Relationen.

Axiom 18.3 (Fail closed). *Fehlende oder unentscheidbare blocking Voraussetzungen erzeugen HOLD oder FAIL, niemals implizites PASS.*

Axiom 18.4 (Gate vor Materialisierung).

$$\neg \text{Pass}(G_i)(K_i) \Rightarrow \text{Materialize}(K_i) = 0.$$

Bei Unentscheidbarkeit gilt HOLD, nicht Commit.

18.2 Gatefamilie

Gate	Ordn.	Voraussetzungen	Passziel
G-BOOT	2	Manifest, Digests, Schemas, Register, M_{13} -Topologie, blocking Obligations	Kernel aktiv
G-PROFILE	1	Profilbindung, Capabilitymatrix, Adapterversionen	Profilwechsel
G-SCHEDULE	1	Budget, Prioritätsordnung, offene Reconciliation	Tickfreigabe
G-REALITY-COHERENT	1	Wohlgeformtheit, interne Konsistenz	POSSIBLE
G-LAWFUL	1	Modell deklariert, Constraints erfüllt	LAWFUL
G-REACHABLE	1	Kausalphad, Budget, Zeitfenster	REACHABLE
G-CONSTRUCTIBLE	1	ausführbarer Plan, Capabilities, Dependencies	CONSTRUCTIBLE
G-SIMULATION	1	Simulationsanker, Modellgeltung, Kostenrahmen	SIMULATED
G-SELECTION	1	Kandidatenrang, Folgenklasse, Reversibilität	SELECTED
G-CONSENSUS	2	Scope, Quotient, effektiver Witnessrang, Obstruktionen	Scoped Pass
G-CLOSURE	2	18 Zellen, Seams, Close_{720} , globale Sektion	Canonical Commit
G-EFFECT	2	Risiko, Autorität, Ressourcen, Rollback, Capability	Tokenize
G-OBSERVATION	1	unabhängiger Beobachter, Provenienz, Zeitfenster	OBSERVED
G-ACTUALIZATION	2	Token, Außenrecord, Reconciliation CLOSED, Trace	ACTUALIZED
G-RESIDUE	1	Residuentyp, Scope, Folgeoperationen	Residuum sichtbar

Gate	Ordn.	Voraussetzungen	Passziel
G-RETRACTION	1	Retraktionsgrund, Supersessionsverweis	RETRACTED
G-MORPH	1	Gain, Scope, Dependency, Budget, Rollback, Lineage	Feldübergang
G-EXCISION	2	Exzisionszertifikat vollständig	Exzision wirksam
G-SELF-COMPILE	2	Elternbindung, Invarianzerhalt, Round-Trip, Negativsuite, unabhängiger Replay, sichtbare Residuen	Aktivierungsvorschlag
G-RESUME	1	Ursache behoben, Residuen geschlossen oder gehalten	READY
G-SHUTDOWN	1	offene Kapseln geschlossen, Tokens invalidiert	STOPPED
G-RELEASE	2	alle blocking Prüfungen, R2, keine blocking Residuen	Release

Invariante 18.5 (Gateabschluss). Die Gatemenge ist abgeschlossen. Jeder Zustandsübergang eines Automaten aus Kapitel 13 verweist auf genau ein Gate dieser Tabelle oder auf – (kein Gate). Ein Übergang mit unbekanntem Gate erzeugt PSK-E007.

Algorithmus 18.6 (Gate-Auswertung).

```
function evaluate_gate(gate_id, inputs, rd) -> GateReport:
  digests = [ H(Can(i)) for i in inputs ] // deterministische Reihenfolge
  policy = gate_policy(gate_id) // aus gate_policy.yaml
  reasons = []
  for cond in policy.blocking_conditions:
    r = eval(cond, inputs)
    match r:
      UNDECIDABLE -> reasons.append(cond); decision = HOLD
      FALSE -> reasons.append(cond); decision = FAIL; break
      TRUE -> continue
  if no reasons: decision = PASS
  if gate.order == 2:
    seam = M11.seam_report(inputs)
    if not seam.compatible: decision = min(decision, HOLD)
  report = GateReport{ gate_id, order, digests, decision, reasons, ... }
  M19.append(report) // ueber P33, synchron
  if decision != PASS: M19.residualize(report)
  return report
```

Invariante 18.7 (Gatebericht immer). Jede Gate-Auswertung erzeugt genau einen GateReport, auch bei PASS. Ein Zustandsübergang ohne referenzierten GateReport ist ein Architekturdefekt.

Vertrauensgrenzen und Capabilities

19.1 Vertrauensdomänen

Definition 19.1 (Sechs Vertrauensdomänen). Constitution, Kernel, FieldRuntime, EffectBoundary, ExternalWorld, Audit. Jedes Modul MUSS genau einer Domäne angehören. Jeder Übergang MUSS Schema, Identität, Trace und Capability prüfen.

Domäne	Module	Zulässige ausgehende Übergänge
Constitution	M00–M04	nach Kernel (lesend), nach Audit
Kernel	M05–M07, M11–M15, M18, M22–M26	nach FieldRuntime, EffectBoundary (nur M15→M16), Audit
FieldRuntime	M08–M10, M20	nach Kernel, Audit
EffectBoundary	M16	nach ExternalWorld
ExternalWorld	Adapter	nach Audit über M17
Audit	M17, M19, M21, M27	keine (Senke)

Regel 19.2 (Zuordnung von M15, M17, M18). M15 entscheidet über Tokenausgabe und trägt damit Kernautorität; die bereits bestehende Ausnahme M15 → M16 in Kerns Übergangsregel setzte dies stillschweigend voraus. M18 entscheidet über Faktpromotion (RECONCILED → ACTUALIZED) und trägt damit ebenfalls Kernautorität, nicht Aufzeichnung; die Verweigerung von store.write.receipt an M18 (Kapitel Capabilitymatrix) unterscheidet es bereits von Audit. M17 zeichnet nur auf, entscheidet nichts, und gehört zu Audit, wo die eingehende Kante aus ExternalWorld auch tatsächlich endet.

Axiom 19.3 (Least Privilege). Eine Rolle erhält nur die minimal erforderlichen Fähigkeiten. Insbesondere DARF NICHT eine Feldidentität Fakten aktualisieren, die Effektgrenze ein eigenes Token ausstellen und die Reconciliation Engine einen Außenrecord umschreiben.

Invariante 19.4 (Interne Freiheit erzeugt keine externe Fähigkeit). Ein interner Freiheitsgrad, eine Linse, ein Archetyp oder eine Persona DARF NICHT als externe Capability ausgegeben werden. Verletzung erzeugt PSK-E007 und sperrt die Capability.

19.2 Capabilitymatrix

Struktur 19.5 (CapabilityMatrix).

```
capability_matrix:
  schema: psk.capability-matrix/1.0
  capabilities:
    - id: fs.read.bundle ; holders: [M00] ; scope: bundle_path
    - id: fs.read.workspace ; holders: [M05] ; scope: anchor_scope
    - id: fs.read.observer ; holders: [M17] ; scope: observer_scope
    - id: fs.write.sandbox ; holders: [M16] ; scope: token.scope
    - id: net.request ; holders: [M16] ; scope: token.scope
    - id: proc.spawn ; holders: [M16] ; scope: token.scope
    - id: token.issue ; holders: [M15] ; scope: gate_report
    - id: store.append ; holders: [M19] ; scope: store_root
    - id: store.read ; holders: [M19, M27] ; scope: store_root
    - id: key.sign ; holders: [M21] ; scope: certificate
    - id: clock.read ; holders: [M25] ; scope: run
    - id: proc.control ; holders: [M26] ; scope: process
  denials:
    - {holder: M16, capability: token.issue, reason: separation_of_powers}
    - {holder: M16, capability: fs.read.observer, reason: observer_independence}
    - {holder: M18, capability: store.write.receipt, reason: record_immutability}
```

```
- {holder: "M08..M10", capability: "fs.*", reason: field_has_no_effect}  
- {holder: M27, capability: store.append, reason: observation_has_no_authority}
```

Vertrag 19.6 (Capability-Erzwingung). *Capabilities MUSS auf Substratebene erzwungen werden: Dateisystemrechte, Namespaces, Prozessisolation oder äquivalente Mechanismen. Eine Erzwingung allein durch Programmkonvention ist nicht konform und erzeugt T-SEC-001-Fehlschlag.*

Effektgrenze, Receipts und Reconciliation

20.1 Effektgrenze

Definition 20.1 (Effektgrenze). M16 ist die einzige Komponente mit ausgehenden Substratrechten. Jeder Effekt durchläuft genau die Kette

```
EffectPlan → G-EFFECT → EffectToken → EffectAttempt → ExternalReceipt →
ReconciliationReport.
```

Vertrag 20.2 (Herkunftsbeglaubigung an der Prozessgrenze). *ExternalReceipt überquert bei P24 eine echte Prozessgrenze aus dem Beobachterprozess (Regel Einzelrechnerbetrieb). Ein versiegelter oder anderweitig typgeschützter Rust-Typ schützt hier nichts, da jede Prozessgrenze Serialisierung erzwingt. Die Ingress-Implementierung von P24 MUSS die Herkunft auf Substratebene prüfen, gemäß Vertrag Capability-Erzwingung: der annehmende Prozess MUSS die Betriebssystemidentität (Benutzerkontext oder Namespace) des sendenden Prozesses gegen die beim Boot festgelegte Beobachterprozess-Identität verifizieren, bevor ein empfangenes Byte als ExternalReceipt deserialisiert wird. Eine Prüfung erst nach der Deserialisierung ist verspätet: das Objekt existiert zu diesem Zeitpunkt bereits. Eine Herkunftsprüfung allein durch Feldvergleich innerhalb des empfangenen Objekts (observer_identity etc.) ist Konvention, keine Substraterzwingung, und erfüllt diesen Vertrag nicht. Eine vom annehmenden Prozess selbst erzeugte, exklusive Kommunikationsleitung zu genau dem einen von ihm erzeugten Kindprozess (M26, proc.control) ist Prozessisolation im Sinne von Vertrag Capability-Erzwingung und erfüllt diesen Vertrag, unabhängig davon, ob Sender und Empfänger dieselbe Betriebssystem-Benutzerkennung tragen.*

Offene Implementierungsverpflichtung 20.3 (OBL-010 Getrennte Benutzerkontexte oder Namespaces). *Regel Einzelrechnerbetrieb verlangt getrennte Benutzerkontexte oder Namespaces für Effekt- und Beobachterprozess. Ein über M26 erzeugter Kindprozess ohne zusätzliche Rechte- oder Namespace-Trennung teilt die Benutzerkennung des Kernprozesses und erfüllt diese Regel DARF NICHT. Das konkrete Verfahren (Namespaces und Rechteabgabe unter Linux, CreateProcessWithLogonW oder gleichwertig unter Windows) ist plattform- und domänenabhängig zu deklarieren, analog zu OBL-005. Blocking ab C4; C0–C3 sind mit prozessisolierter, gleichkontextiger Kommunikation erreichbar.*

Invariante 20.4 (Kein Effekt ohne Token).

$$\text{ExecuteEffect}(e) = 1 \Rightarrow \text{Gate}(e) = \text{PASS} \wedge \text{TokenBound}(e) = 1.$$

Ein Adapteraufruf ohne gültiges, nicht konsumiertes, nicht abgelaufenes Token erzeugt PSK-E008, führt keinen Effekt aus und schreibt ein ResidueRecord.

Schnittstelle 20.5 (Adapter).

```
interface EffectAdapter {
    fn id() -> AdapterId
    fn declared_effect_classes() -> [EffectClassId]
    fn required_capabilities() -> [CapabilityId]
    fn prestate(scope: ScopeExpr) -> Digest
    fn apply(token: EffectToken, plan: EffectPlan) -> EffectAttempt
    fn compensate(attempt: EffectAttempt) -> EffectAttempt
    fn is_reversible(plan: EffectPlan) -> bool
    // VERBOTEN: observe(), read_result(), confirm()
}

interface ObserverAdapter { // strikt getrennte Implementierung
    fn id() -> AdapterId
    fn observe(scope: ScopeExpr) -> ExternalReceipt
    fn independence_attestation() -> Digest
    // VERBOTEN: apply(), compensate()
}
```

Regel 20.6 (Vorzustand und Versuch klammern den Effekt). Ein `EffectAdapter` liefert den Vorzustand und führt den Effekt aus. Beide Aufrufe MUSS denselben Beobachtungskanal benutzen, und zwischen ihnen DARF NICHT etwas am beobachteten Bereich geschehen: sie klammern den Effekt, sonst vergleicht die Reconciliation einen Vorzustand, der nie unmittelbar vor dem Versuch galt.

Über eine Prozessgrenze folgt daraus, dass derselbe Kindprozess beide Aufrufe bedient und über beide hinweg gehalten wird. Zwei Erzeugungen wären zwei Beobachtungen mit einem Fenster dazwischen; die exklusive Leitung, die Vertrag *Herkunftsbeglaubigung an der Prozessgrenze* verlangt, ist dann auch nicht mehr dieselbe. Verhindert eine Signatur das Halten, so ist die Signatur zu ändern, nicht die Klammerung aufzugeben.

Invariante 20.7 (Adaptertrennung). Keine Codeeinheit DARF zugleich `EffectAdapter` und `ObserverAdapter` implementieren. Die beiden Adapter MUSS getrennte Prozesse, getrennte Capabilities und getrennte Identitäten besitzen.

20.2 Reconciliation

Definition 20.8 (Reconciliation).

$\text{Recon}(\text{plan}, \text{attempt}, \text{record}, \text{anchor}) \rightarrow (\text{verdict}, \text{difference}, \text{finality}, \text{residue}, \text{witness}).$

Algorithmus 20.9 (Reconciliation).

```
function reconcile(plan, attempt, receipts, anchor_post) -> ReconciliationReport:
  require attempt.token_ref.plan_digest == H(Can(plan)) else FAIL(PSK-E008)
  require |receipts| >= 1 else FAIL(PSK-E009)
  for r in receipts:
    require r.observer_adapter != attempt.adapter else FAIL(PSK-E009)
  expected = project_expected_state(plan, attempt.prestate_digest)
  observed = merge_records(receipts, anchor_post)
  diff = structural_diff(expected, observed)
  match diff:
    empty -> verdict = CLOSED
    within_declared_tolerance -> verdict = CLOSED
    nonempty and explicable -> verdict = OPEN; residualize(diff)
    contradictory -> verdict = DIVERGENT; residualize(diff); FAIL
    receipts_insufficient -> verdict = UNKNOWN; quarantine(attempt)
  promotion = (verdict == CLOSED) ? ACTUALIZED : (receipts_valid ? OBSERVED : NONE)
  return ReconciliationReport{ verdict, diff, finality, promotion, ... }
```

Invariante 20.10 (Keine Aktualisierung ohne Reconciliation). `Actualize(x)` ist ausschließlich definiert, wenn ein externer oder unabhängig gebundener Record vorliegt und $\text{Reconcile}(\text{Plan}, \text{Attempt}, \text{Receipt}, \text{ObservedState}) = \text{CLOSED}$.

20.3 Prognosepersistenz

Struktur 20.11 (Forecast). OBJ-FCT

```
Forecast:
  schema: psk.forecast/1.0
  id: ObjectId
  horizon: HorizonSpec # bis wohin die Prognose reicht
  generation_basis: [ObjectId] # Erzeugungsbasis, versiegelt
  validity_window: TimeWindow # Gültigkeitsfenster, tau_i-gebunden
  anchor_ref: ObjectId # der Anker ZUM ZEITPUNKT der Erzeugung
  claim_refs: [ObjectId]
  evaluations: [ForecastEvaluation] # append-only, nie ersetzend
  trace_ref: TraceRef

ForecastEvaluation:
  observed_ref: ObjectId # spätere Beobachtung
  verdict: CONFIRMED | REFUTED | INCONCLUSIVE
  evaluated_at: DualTime
```

Regel 20.12 (Prognosen werden bewertet, nicht umgeschrieben). `horizon`, `generation_basis`, `validity_window` und `anchor_ref` sind nach Erzeugung unveränderlich. Eine spätere Beobachtung erzeugt ausschließlich einen neuen Eintrag in `evaluations`; sie DARF NICHT ein bestehendes Feld ändern und DARF NICHT einen bestehenden Eintrag ersetzen. Die Durchsetzung erfolgt strukturell: `evaluations` ist append-only wie der Trace, und die übrigen Felder besitzen keinen Schreibpfad nach Konstruktion.

Vertrag 20.13 (Prognosepersistenz). *Eine Prognose MUSS Horizont, Erzeugungsbasis, Gültigkeitsfenster und den damaligen Anker speichern. Spätere Beobachtungen DARF die Prognose bewerten, DARF NICHT aber rückwirkend in eine scheinbar korrekte Prognose umschreiben. Verletzung erzeugt PSK-E014.*

TEIL VII

Verifikation, Replay und Konformanz

Witness- und Validierungsmodell

21.1 Witnessengine

Definition 21.1 (Witnessrang). $r_{\text{eff}} = \text{RankEff}(W, D)$ zieht gemeinsame Daten, Modelle, Sonden, Prompts, Kalibrierungen und Ableitungslinien ab.

Vertrag 21.2 (Scopegebundener Konsens). *Ein Konsensobjekt MUSS seinen Scope als local, cluster, system oder external tragen. Eine Scopeerweiterung benötigt einen expliziten Gate- und Witnessübergang über G-CONSENSUS.*

Invariante 21.3 (Keine Scope-Leakage). *Ein lokales oder Clusterresultat DARF NICHT als systemweite oder externe Aussage emittiert werden. Verletzung erzeugt FAIL und T-SCOPE-001-Fehlschlag.*

Regel 21.4 (Evidence-bound Trust). Vertrauen ist kein frei gesetzter Score. Es wird abgeleitet aus bestandenen Witnesszyklen, Replaystabilität, Seam-Kohärenz, Residuenhistorie, Kalibrierung und Unabhängigkeitsrang. Ein Trustgewicht DARF Gates priorisieren oder Prüfaufwand steuern, DARF NICHT aber fehlende Evidence, fehlenden Außenrecord oder gescheiterte Reconciliation ersetzen.

21.2 Validierungsplanung

Struktur 21.5 (ValidationPlan).

```
ValidationPlan:
  id: ObjectId
  target_refs: [ObjectId]
  open_obligations: [ObligationExpr]
  steps:
    - {probe: ProbeSpec, expected_information_gain: Scaled,
      cost: BudgetSpec, capability: CapabilityId?, read_only: bool}
  ordering: by_information_gain_per_cost # deterministisch, stabil sortiert
  stop_condition: obligations_closed | budget_exhausted | no_positive_gain
```

Regel 21.6 (Validierungspriorität). M13 wählt den Schritt mit maximalem erwartetem Informationsgewinn pro Kosten. Bei Gleichstand entscheidet die kleinere ObjectId. Read-only-Schritte besitzen Vorrang vor effektbehafteten Schritten gleichen Gewinns.

Trace, Replay und Determinismus

22.1 Replayklassen

Definition 22.1 (Replayklassen).

Klasse	Name	Bedingung
R0	narrativ	Der Lauf ist beschrieben, aber nicht maschinell rekonstruierbar.
R1	strukturell	Objektgraph und Entscheidungen rekonstruierbar; Digests dürfen abweichen.
R2	kanonisch	Gleicher <code>RunDescriptor</code> und gleiche Eingabedigests erzeugen denselben kanonischen Zustandsdigest und dieselbe Gatefolge.
R3	bitidentisch	Zusätzlich byteidentische Artefakte.

Definition 22.2 (Kapselfixpunkt). Eine Kapsel c steht im *Kapselfixpunkt*, wenn ein weiterer `ratchet`-Schritt ihre Menge zulässiger Nachfolger nicht mehr verkleinert:

$$\text{allowed_next}(\text{ratchet}(c)) = \text{allowed_next}(c).$$

Da `FSM-CAPSULE` `allowed_next` als monoton nicht wachsend führt, ist der Fixpunkt das eindeutige Ende der `RATCHETED`-Selbstschleife. Erschöpft zuvor das Budget, so ist die Kapsel **DARF NICHT** als im Fixpunkt stehend zu führen, sondern nach `RESIDUAL` zu routen; beide Ausgänge sind in der Challenge-Abschlussbedingung ausdrücklich getrennt.

Regel 22.3 (Replay läuft unter shadow). Das in `I-ARCH-012` und `R-RA-012` genannte *Replay-Profil* ist kein eigener Wert von `ProfileId`, sondern das bestehende Profil `shadow`: Plan und Token werden erzeugt, aber sofort invalidiert, und Gate- sowie Reconciliationsimulation laufen gegen aufgezeichnete Records. Ein Replaylauf MUSS unter `shadow` oder `readonly` ausgeführt werden; die Durchsetzung erfolgt über die bereits bestehende Tokeninvalidierung (P37), nicht über eine zusätzliche Modusabfrage.

`profile` liegt auf dem `RuntimeManifest`, nicht auf dem `RunDescriptor`. Ein Replaylauf trägt daher denselben `RunDescriptor` wie sein Original – die von R2 verlangte Gleichheit bleibt erhalten, obwohl das Profil abweicht. Ein Modusfeld auf dem `RunDescriptor` wäre aus genau diesem Grund unzulässig.

Regel 22.4 (Verschiedene Eingaben bilden kein Replaypaar). Definition *Replayklassen* ist ein Konditional: *gleicher* `RunDescriptor` und *gleiche* Eingabedigests erzeugen denselben kanonischen Zustandsdigest. Zwei Läufe mit verschiedenen Eingaben erfüllen den Vordersatz nicht — sie liegen ausserhalb des Definitionsbereichs, nicht in seinem Verletzungsfall.

Eine Einstufung MUSS das unterscheiden können und dafür den `RunDescriptor` beider Läufe sehen. Ergeben sich verschiedene Digests, so ist das Ergebnis `NOT_COMPARABLE` — weder R0 noch ein Replaydefekt. Eine Einstufung, die nur Wahrheitswerte über Übereinstimmung entgegennimmt, macht aus dem Konditional eine Bijunktion und meldet jede Messreihe als Divergenz.

`NOT_COMPARABLE` DARF NICHT eine Zertifikatsausstellung blockieren, die sich auf einen einzelnen Lauf stützt; es sagt aus, dass *dieses Paar* nichts über Replay aussagt.

Vertrag 22.5 (Replayklasse des Referenzrelease). Ein *Referenzrelease* MUSS mindestens R2 erreichen. R3 ist optional und domänenabhängig.

Algorithmus 22.6 (Replay).

```
function replay(run_id) -> ReplayResult:
  rd = M19.load_run_descriptor(run_id)
  seal = M19.load_input_digests(run_id)
  require verify_chain(M19.trace(run_id)) else FAIL(PSK-E014)
  fresh = boot(rd.bundle_ref, rd.profile) // isolierte Instanz
  bind_deterministic_sources(rd.seed, rd.time_window, rd.nondeterminism_budget)
```

```
for record in M19.external_records(run_id): // aufgezeichnete Aussenwelt
    inject_as_replay_receipt(record) // KEIN realer Effekt
state = run_until_end(seal)
return ReplayResult{
    canonical_digest_match: H(Can(state)) == M19.final_digest(run_id),
    gate_sequence_match: gates(state) == M19.gate_sequence(run_id),
    divergence_report: first_divergence_or_none() }
```

Invariante 22.7 (Replay ist effektfrei). Ein Replay DARF NICHT einen realen Effekt auslösen. M16 wird im Replayprofil durch einen Null-Adapter ersetzt, der ausschließlich die aufgezeichneten EffectAttempt-Objekte zurückgibt. Ein Replay, der ein Token konsumiert oder ein Substrat verändert, ist ein Architekturdefekt.

Struktur 22.8 (ReplayManifest).

```
ReplayManifest:
    run_id: RunId
    run_descriptor_digest: Digest
    input_digests: [Digest]
    external_records: [{receipt_id, digest, observer_identity}]
    trace_head: Digest
    final_canonical_digest: Digest
    gate_sequence_digest: Digest
    achieved_class: R0 | R1 | R2 | R3
    divergences: [DivergenceRecord]
```

Konformanzmodell

23.1 Kanonisierung der Konformanzordnung

Regel 23.1 (Eine Konformanzleiter). Es existiert genau eine Konformanzleiter C_0 – C_5 . Die Merkmalsstufen der Gesamtspezifikation werden als *abgeleiteter Deckungsvektor* $FC = (FC_0, \dots, FC_8)$ geführt. FC ist keine Klassifikation, sondern eine Pflichtangabe im Maschinenzertifikat und eine Vorbedingung bestimmter C -Klassen. Eine parallele Verwendung als eigenständige Konformitätsklasse ist nicht zulässig.

Bit	Nachweis
FC0	Grammatiktypen, kanonische Serialisierung, Spezifikations-ID.
FC1	M_{13} -Graph, 30 Kanten, 18 Zellen, lokale Seam-Closure.
FC2	PSK-IR, Round-Trip, Replay.
FC3	Reality- und Faktizitätstypisierung ohne Promotionsbypass.
FC4	Feldprojektionen, Lineage, Abhängigkeitsquotient.
FC5	Kandidatenkapseln, adversariales Ratchet, Support, Residuenfluss.
FC6	Gate, Effekttoken, Receipt, Reconciliation.
FC7	Kontrollierte Selbstkompilation und unabhängige Aktivierung.
FC8	Validierte Referenzdomäne mit Baselinevergleich.

23.2 Konformitätsklassen

Klasse	Name	Abnahme	FC-Pflicht
C0	Artifact-conformant	Bundle, Lock, Schemas, Requirement Coverage = 1.	FC0
C1	Kernel-executable	Minimaler Kern läuft und erzeugt valide Artefakte.	FC0–FC2
C2	Replay-valid	R2-Replay und Golden Runs bestanden.	FC0–FC3
C3	Sandbox-effect-safe	Token, Capability, Effektgrenze, Receipt, Reconciliation geprüft.	FC0–FC6
C4	Reference-validated	Reale Referenzdomäne besteht Baselines und Negativtests.	FC0–FC6, FC8
C5	Externally-reproduced	Unabhängige Instanz reproduziert Konformität und Leistungsbefunde.	FC0–FC8

Regel 23.2 (Was eine unabhängige Reproduktion bezeugt). C5 DARF nicht aus dem zu zertifizierenden Werkstück heraus erklärt werden; eine selbst ausgestellte externe Reproduktion wäre Selbstautorisierung. Die Feststellung `externally_reproduced` wird von außen hereingereicht und MUSS folgendes belegen:

1. Die unabhängige Instanz baut aus dem veröffentlichten Bündel und rechnet I_C , I_A und I_M selbst nach; abweichende Werte sind Fehlschlag, nicht Hinweis.
2. Sie führt den Golden Run auf eigener Hardware und in eigener Umgebung aus und erhält denselben kanonischen Zustandsdigest I_I sowie dieselbe Gatefolge.
3. Sie rechnet die vier Berichtsdigests des Zertifikats aus den eigenen Laufartefakten nach.
4. Sie leitet den Deckungsvektor selbst ab und erhält denselben; ein übernommener Vektor zählt nicht.
5. Sie benennt Plattform und Umgebung ihres Laufs; weicht die Plattform von der im `scope` gebundenen ab, so bezeugt sie zusätzlich, welche plattformgebundenen Verpflichtungsaufösungen dort offen bleiben.

Der Befund ist ein eigenständiges Artefakt der prüfenden Instanz, kein Feld, das das geprüfte Werkstück über sich selbst setzt.

Invariante 23.3 (Keine Fähigkeitsbehauptung durch Konformität). *Keine Konformitätsklasse bedeutet allgemeine Intelligenz, Bewusstsein, moralische Qualität oder universelle Leistungsfähigkeit. Sie bezeichnet ausschließlich die erfüllten Verträge.*

23.3 Verfeinerungssicherheit

Definition 23.4 (Konservative Implementierungsverfeinerung). Erhält eine Abstraktionsabbildung h Gate, Closure, Trace, Replay, Identität und Residuen und ist sie Vorwärtssimulation, dann materialisiert die konkrete Implementierung nur abstrakt zulässige Zustände.

Vertrag 23.5 (Maschinencheckbare Verfeinerung). *Die Referenzimplementierung MUSS für jeden konkreten Zustandsautomaten eine explizite, maschinenlesbare Abbildung in den abstrakten Automaten aus Kapitel 13 dokumentieren (`refinement_map.yaml`). Unabgebildete Transitionen sind blocking.*

Testarchitektur

24.1 Testklassen

Klasse	Präfix	Gegenstand und Pflicht
Unit	U-	Einzelfunktion je Modul. Deckungspflicht: jede öffentliche Funktion.
Property	PR-	Algebraische Eigenschaften: Idempotenz von Can, Round-Trip des IR-Codec, Monotonie des Ratchet, Azyklizität der Modulgraphen.
Schema	SC-	Golden Objects werden akzeptiert, unvollständige verworfen.
Automat	FS-	Vollständige Kantenüberdeckung jedes Automaten inklusive aller <code>forbidden</code> -Kanten als Negativfälle.
Golden	GR-	Deterministischer Referenzlauf; kanonischer Zustandsdigest identisch.
Negativ	T-	Pflichtzeugen; jede Mutation MUSS die deklarierte Reaktion erzwingen.
Adversarial	AD-	Gegenmodelle und Stresslasten gemäß $\mathcal{A}(M)$.
Replay	RP-	R2-Nachweis, Divergenzbericht, Fault Injection.
Integration	IN-	End-to-End über alle Ports P00–P41.
Differential	DF-	Zwei unabhängige Implementierungen erzeugen identische kanonische Digests.
Sicherheit	SEC-	Capability-Erzwingung, Sandbox-Ausbruchsversuche, Beobachtertrennung.
Benchmark	BM-	Baselinevergleich; nicht kernblockierend.

Vertrag 24.1 (Testpflicht je Requirement).

$$\text{Coverage} = \frac{\#\{r \mid \text{artifact}(r) \wedge \text{test}(r) \wedge \text{owner}(r)\}}{\#\text{Requirements}}.$$

Für den Referenzrelease gilt $\text{Coverage} = 1$. Ein Requirement ohne Artefakt, Test oder Owner blockiert den Release.

24.2 Golden Run

Definition 24.2 (Golden Run). Ein Golden Run ist ein versiegelter End-to-End-Lauf über die Referenzdomäne mit gebundenem `RunDescriptor`, aufgezeichneten Außenrecords und erwartetem kanonischem Zustandsdigest.

Regel 24.3 (Golden-Run-Ablauf). 1. Bundle verifizieren, Bootgate schließen.

2. Workspace als `AnchorSnapshot` versiegeln.

3. Auftrag in `ThoughtBody` kompilieren.

4. Realitätstypen relativ zu Dateien, Werkzeugen, Rechten und Zeit bestimmen.

5. Statische Feldfamilie (sechs Archetypen) ausführen.

6. Abhängigkeiten quotieren und lokale Resultate verkleben.

7. Bei offenen Obligationen read-only Validierung durchführen.

8. Patchplan erzeugen; Folgen- und Risikogate prüfen.

9. `EffectToken` für Sandboxdateien ausstellen.

10. Patch ausführen; `EffectAttempt` schreiben.

11. Unabhängigen Beobachter den Dateibaum lesen lassen; `ExternalReceipt` erzeugen.

12. Reconciliation durchführen; Faktpromotion entscheiden.

13. Maschinenzertifikat und Replaymanifest exportieren.

Der erwartete Endzustand: ein simulierter Patch beginnt als `SELECTED`, erhält ein Effekttoken, wird nur in einer Sandbox versucht, durch einen unabhängigen Dateisystemsnapshot beobachtet und erst nach geschlossenem Vergleich als `ACTUALIZED` markiert.

Regel 24.4 (Der Golden Run läuft unter `tick`). Die dreizehn Schritte sind *Phasenarbeit*, nicht Geradeauscode. Schritt 1 geschieht in `boot()`; die Schritte 2 bis 13 MUSS der Lauf als Arbeit in den Warteschlangen der zwölf Taktphasen abwickeln, nicht als Folge direkter Aufrufe.

Der Grund ist nicht Ordnungsliebe: das Zertifikat weist $I_t = H(\text{Can}(\Sigma_t))$ aus, und Σ_t ist der Zustand *nach* den Takten. Ein Lauf, der die Module in derselben Reihenfolge, aber außerhalb des Taktzyklus aufruft, erzeugt ein Zertifikat über einen Pfad, den dieses Werk nirgends als Ausführung definiert. Zwei Beschreibungen desselben Vorgangs nebeneinander zu führen — Taktzyklus und Geradeausfolge — ist die Parallelstruktur, die Regel *Keine Alternativmodelle* ausschließt.

Zuordnung der Schritte zu den Phasen: `observe` (2), `normalize` und `type` (3, 4), `anchor` (2, 4), `project` (5), `compile` (6), `challenge` (6), `verify` (7, 8), `execute` (9, 10), `observe_2` (11), `reconcile` (12), `archive` (13). Eine Phase ohne Arbeit wird versiegelt wie jede andere; das Siegel ist der von Invariante *Keine stille Millisekunde* verlangte Nachweis.

24.3 Empirische Baselines

Regel 24.5 (Pflichtbaselines). Der Kern wird gegen mindestens zwei Baselines verglichen: (i) einen monolithischen Agenten mit demselben Modell und denselben Werkzeugen; (ii) einen Agenten mit strukturierter Planung und Event Sourcing, aber ohne Realitätsleiter, Abhängigkeitsquotient und konstitutionelle Gates.

Metrik	Messung
Ungebundene Faktbehauptungen	Claims ohne Anchor- oder Evidenceverweis pro Lauf.
Falsche Faktpromotion	<code>SIMULATED/ATTEMPTED</code> ohne <code>Reconciliation</code> als <code>ACTUALIZED</code> .
Scope-Leakage	Lokale oder Clusterresultate als globale Aussage.
Synthetische Mehrheit	Facettenstimmen minus r_{eff} .
Nichtautorisierte Effekte	Seiteneffekte ohne gültiges Token.
Replayabweichung	Differenz kanonischer Zustände bei identischem <code>RunDescriptor</code> .
Validierungseffizienz	Informationsgewinn pro Probe, Zeit und Token.
Kreativitätserhalt	Vielfalt kohärenter Hypothesen bei stabiler Faktgrenze.
Overhead	Laufzeit, Speicher, Kontext- und Artefaktkosten.
Recovery	Rollback- und <code>Reconciliation</code> -Erfolg nach injiziertem Fehler.

TEIL VIII

Artefakte, Repository und Deployment

Artefaktstruktur

25.1 Doppelpjektion

Regel 25.1 (Drei gebundene Bündel). Die Referenzmaschine bindet genau drei kanonische Bündel:

1. constitution/ — unverändertes Contract-Bundle des Maschinenvertrags, Identität I_C ;
2. architecture/ — Register dieses Werkes, Identität I_A ;
3. packages/ + adapters/ — Implementierung, Identität I_M .

Ein Bündel DARF NICHT ein anderes überschreiben. Jede Änderung an 1 oder 2 erzeugt eine neue Identität und erzwingt ein Amendment.

Struktur 25.2 (Contract-Bundle constitution/).

```
constitution/
  constitution.lock.json # Identitätswurzel: constitution_id + Digests
  constitution.yaml # Axiome, Bootpolitik, Amendment-Regeln
  theory_binding.json
  target_identity.json
  artifact_registry.yaml
  type_registry.yaml
  state_machines.yaml
  operator_registry.yaml
  invariants.yaml
  proof_obligations.yaml
  gate_policy.yaml
  capability_matrix.yaml
  trust_boundaries.yaml
  error_catalog.yaml
  requirement_matrix.yaml
  release_gate.yaml
  meta_instance.yaml
  conformance_tests.yaml
  schemas/anchor_snapshot.schema.json
  schemas/thought_body.schema.json
  schemas/field_identity.schema.json
  schemas/run_descriptor.schema.json
  schemas/machine_certificate.schema.json
```

Struktur 25.3 (Architekturbündel architecture/).

```
architecture/
  architecture.lock.json # architecture_id + Digests + parent constitution_id
  module_map.yaml # M00..M27: Name, Schicht, Ownership, Capabilities
  port_registry.yaml # P00..P41: Erzeuger, Verbraucher, Schemas, Fehler
  sort_registry.yaml # 19 Sorten + Sorten-Port-Matrix
  object_registry.yaml # kanonische Objekte, Owner, Lebensdauer, Speicherklasse
  object_schemas.yaml # Feldstrukturen der kanonischen Objekte
  volatile_fields.yaml # nicht identitätsbildende Felder (pi_vol)
  lifecycle_registry.yaml # Lebensdauer, Owner, Speicherklasse
  ml3_topology.yaml # 13 Knoten, 30 Kanten, 18 Zellen, Adressgrammatik
  pass_registry.yaml # C1..C11: Ein-/Ausgabe, Fehler, Determinismus
  isa_registry.yaml # 23 Instruktionen: Ports, Pre/Post, Fehler
  memory_registry.yaml # 6 Speicherklassen, Promotionspfade
  gate_registry.yaml # Gates, Ordnung, blocking conditions, Passziel
  ra_invariants.yaml # Architekturinvarianten I-ARCH-*
  ra_requirements.yaml # R-RA-*
  ra_tests.yaml # T-ARCH-*, T-PORT-*, T-OWN-*
  refinement_map.yaml # konkret -> abstrakt (Pflicht, PO-REF-001)
  obligations.yaml # OBL-001..OBL-00n
  schemas/ # JSON-Schemas der 16 Architekturregister selbst,
            # NICHT Kopien der Objektschemas aus constitution/
```

Vertrag 25.4 (Selbsttragende Projektion). Das PDF dieses Werkes MUSS das vollständige Architektur-

bündel und dessen Manifest als eingebettete Dateien tragen. Ein Viewer ohne Attachment-Unterstützung beeinträchtigt die Zugänglichkeit, nicht die eingebettete Identität.

Vertrag 25.5 (Schema-Evolution). *Breaking Changes an Pflichtfeldern oder Enum-Semantik erzeugen eine neue Identität des betroffenen Bündels. Additive optionale Felder DARF innerhalb derselben Minorversion zulässig sein, wenn Kanonisierung und Replay stabil bleiben.*

Invariante 25.6 (Keine hängenden Referenzen). *Alle Querverweise zwischen Modulen, Ports, Operatoren, Gates, Invarianten, Requirements und Tests MUSS der Loader auflösen. Eine hängende Referenz ist mindestens HOLD; bei blocking Anforderungen FAIL.*

Repository und Paketstruktur

26.1 Kanonisches Repository

Struktur 26.1 (Repositorybaum).

```

psk/
  PSK.lock # bindet constitution_id, architecture_id, implementation_id
  MANIFEST.sha256 # Digest jeder versionierten Datei
  constitution/ # siehe oben, unveränderlich
  architecture/ # siehe oben, unveränderlich
  packages/
    psk-types/ # M03-nahe Basistypen, Sorten, Fehlercodes
    psk-canon/ # M01: Can, Digest, IDs
    psk-contract/ # M00, M02, M04: Loader, Registry, Identity Binder
    psk-topology/ # M22: M13
    psk-ir/ # M23: IR-Codec, Knoten-/Kantenvertraege
    psk-anchor/ # M05, M17: Anchor Registry, Record Ingress
    psk-thought/ # M06, M07: Thought Compiler, Reality Typer
    psk-fields/ # M08, M09, M20: Field Registry, Lens Router, Morphogenese
    psk-dependency/ # M10: Dependency Analyzer
    psk-adversarial/ # M24: Kapseln, Ratchet, CRA, Self-Hardening
    psk-closure/ # M11: Seams, 360/720, Gluing, Obstruktionen
    psk-witness/ # M12, M13: Witness Engine, Validation Planner
    psk-gate/ # M14: Gates aller Ordnungen
    psk-effect/ # M15, M16: Token Service, Effektgrenze
    psk-reconciliation/ # M18
    psk-trace/ # M19: Trace, Residue Ledger, Replay
    psk-scheduler/ # M25
    psk-lifecycle/ # M26: Boot, Shutdown, Recovery
    psk-observe/ # M27: Logging, Profiling, Inspect
    psk-certify/ # M21: Zertifikat, Release
    psk-conformance/ # Testtreiber, Golden-Run-Harness
    psk-cli/ # CLI-Vertrag
  adapters/
    effect-local-fs/ # EffectAdapter, Sandbox
    observer-local-fs/ # ObserverAdapter, getrennter Prozess
    observer-git/ # zweiter unabhaengiger Beobachter
  domains/
    local_workspace/ # Referenzdomaenenprofil
  conformance/
    golden/ negative/ property/ adversarial/ replay/ differential/ security/
  examples/
    golden_run/
  tools/
    verify_bundle.py # prueft constitution/
    verify_architecture.py # prueft architecture/
    verify_refinement.py # prueft refinement_map.yaml
  docs/ # nicht normativ

```

Invariante 26.2 (Paket-Modul-Bijektion). Jedes Modul M00–M27 ist genau einem Paket zugeordnet. Ein Paket DARF mehrere Module enthalten. Kein Modul ist über mehrere Pakete verteilt.

Invariante 26.3 (Abhängigkeitsordnung der Pakete). Der Paketabhängigkeitsgraph ist azyklisch und respektiert die Schichtenordnung. psk-types und psk-canon besitzen keine internen Abhängigkeiten. psk-observe wird von keinem anderen Paket referenziert. psk-effect DARF NICHT psk-anchor's Beobachterteil referenzieren.

26.2 Build und Reproduzierbarkeit

Vertrag 26.4 (Reproduzierbarer Build). Der Build MUSS deterministisch sein: gepinnte Abhängigkeiten mit Digests, feste Zeitstempel (SOURCE_DATE_EPOCH), sortierte Dateireihenfolge, keine Netzwerkzugriffe nach dem Auflösungsschritt. Zwei Builds derselben Quellen erzeugen identische $I_M =$

$H(\text{Code} \parallel \text{Schemas} \parallel \text{Compiler} \parallel \text{Profil})$.

Regel 26.5 (Buildartefakte). Der Build erzeugt: (i) ausführbare CLI; (ii) `MANIFEST.sha256`; (iii) Materialstückliste (SBOM) mit Digests; (iv) `PSK.lock` mit den drei Identitäten; (v) Buildattestation. Fehlt eines, ist der Build unvollständig und der Release blockiert.

Regel 26.6 (CI-Pflichtstufen). `lint` $\rightarrow$ `build` $\rightarrow$ `unit` $\rightarrow$ `property` $\rightarrow$ `schema` $\rightarrow$ `fsm` $\rightarrow$ `negative` $\rightarrow$ `integration` $\rightarrow$ `golden` $\rightarrow$ `replay` $\rightarrow$ `security` $\rightarrow$ `differential` $\rightarrow$ `conformance-report`. Ein Fehlschlag in einer Stufe bricht die Kette ab.

26.3 Versionierung

Regel 26.7 (Versionsschema). Alle Pakete verwenden semantische Versionierung. Eine Änderung, die I_C oder I_A verändert, erzwingt eine Major-Version der Implementierung. Eine Änderung, die nur I_M verändert und alle Golden Runs und Negativtests unverändert bestehen lässt, ist Patch oder Minor.

Plug-in-Modell und API-Grenzen

27.1 Plug-in-Klassen

Definition 27.1 (Vier Plug-in-Klassen).

Klasse	Erweiterungspunkt	Grenze
EffectAdapter	M16	DARF ausschließlich tokenisierte Effekte ausführen. Keine Beobachtung.
ObserverAdapter	M17	DARF ausschließlich beobachten. Kein Effekt.
DomainProfile	M05, M08, M14	Deklariert Ankerquellen, Feldprofile, Risikoklassen, Budgets, Baselines.
MethodPlugin	M07, M10, M13	Reality-Typing-Verfahren, RankEff-Schätzer, Informationsgewinnmaße.

Weitere Erweiterungspunkte existieren nicht.

Vertrag 27.2 (Domänengelieferte opake Eingaben). *Drei Typen sind im Kern absichtlich ohne Grammatik: LensSpec (Auflösungssemantik einer Spektrallinse, M09), PredicateExpr (Frischeprädikate, M05) und RealityEvidence (Evidenzform des Reality Typing, M07). Sie werden über DomainProfile oder MethodPlugin geliefert und im Kern nur typisiert weitergereicht. Das ist keine Lücke, sondern die Erweiterungsgrenze nach Prinzip 4.3: der Kern entscheidet, dass eine Linse auflöst, nicht wie.*

Für jeden dieser Typen gilt:

1. Die Auswertung MUSS reine Funktion ihrer deklarierten Eingaben sein. Uhr, Zufall und Umgebung sind nur über den RunDescriptor zugänglich; andernfalls bricht Replay und es greift PSK-E015.
2. Das Ergebnis MUSS über determinism im Plug-in-Manifest deklariert sein. recorded_external verlangt einen Außenrecord, keine Wiederholung der Berechnung.
3. Ein fehlendes oder nicht anwendbares Plug-in erzeugt UNKNOWN beziehungsweise ein Residuum, DARF NICHT eine Vorgabewahl. Ein Default-Zweig auf einen positiven Status ist ein Konformitätsdefekt.

Diese drei Typen sind keine offenen Implementierungsverpflichtungen im Sinne von OBL-001–OBL-007; sie sind entschieden, und die Entscheidung lautet: extern.

Struktur 27.3 (Plug-in-Manifest).

```
plugin:
  schema: psk.plugin/1.0
  id: PluginId
  class: EffectAdapter | ObserverAdapter | DomainProfile | MethodPlugin
  version: SemVer
  digest: Digest
  required_capabilities: [CapabilityId]
  declared_effect_classes: [EffectClassId] # nur EffectAdapter
  independence_class: ClassId # nur ObserverAdapter
  determinism: deterministic | seeded | recorded_external
  schemas: [SchemaId]
  tests: [TestId] # Pflicht, nicht leer
  supported_constitution_ids: [Digest]
  supported_architecture_ids: [Digest]
```

Invariante 27.4 (Plug-in-Bindung). *Ein Plug-in wird ausschließlich geladen, wenn sein Digest im artifact_registry.yaml eingetragen ist und I_C , I_A in seinen supported_*-Listen enthalten sind. Ein Plug-in DARF NICHT eine Capability erhalten, die nicht in der CapabilityMatrix für sein Trägermodul deklariert ist.*

Invariante 27.5 (Kein normativer Plug-in-Einfluss). Ein Plug-in DARF NICHT Sorten, Produktionen, M_{13} -Topologie, Automaten, Gates, Invarianten, Fehlercodes oder Konformanzklassen verändern. Es DARF ausschließlich Verfahren innerhalb deklarierter Grenzen bereitstellen.

27.2 API-Grenzen

Grenze	Art	Vertrag
Interne Ports	P00–P41	Nachrichtenhülle 4.1; schemaerzwungen; tracepflichtig.
Bibliotheks-API	psk-core	Stabil ab C1; jede öffentliche Funktion bildet auf genau eine Instruktion oder einen Port ab.
CLI	psk	Siehe unten; einzige Benutzergrenze der Referenzmaschine.
Inspect-API	M27	Read-only; kein Schreibpfad; Ausfall nicht blocking.
Plug-in-API	vier Klassen	Manifestgebunden, digestgebunden, capabilitygebunden.
Netzwerk-API	—	In der Referenzmaschine nicht vorhanden. Ein Domänenprofil DARF einen EffectAdapter mit <code>net.request</code> deklarieren; die Kernmaschine exponiert keinen eingehenden Dienst.

Struktur 27.6 (CLI-Vertrag).

```
psk constitution verify <bundle> // prueft I_C gegen Lock
psk architecture verify <bundle> // prueft I_A gegen Lock
psk refinement verify // prueft refinement_map.yaml
psk anchor seal <workspace> // erzeugt AnchorSnapshot
psk run <task> --anchor <id> --profile readonly|shadow|sandbox|reference
psk inspect <run-id> [--object <id>] [--phase <name>]
psk replay <run-id> [--independent]
psk effect plan <run-id>
psk effect apply <token> --sandbox
psk reconcile <run-id>
psk certify <run-id>
psk residue list <run-id>
psk conformance run [--class C0..C5]
psk export bundle|report|certificate <run-id>
```

Vertrag 27.7 (Keine Prompt-only-Implementierung). Der normative Kern DARF NICHT ausschließlich als Systemprompt oder freie Agentenanweisung existieren. Typen, Gates, Tokens, Trace, Replay und Reconciliation MUSS außerhalb des Sprachmodells technisch erzwungen werden.

Regel 27.8 (Rolle eines Sprachmodells). Ein Sprachmodell DARF ausschließlich als MethodPlugin innerhalb von M06, M09, M13 oder M24 auftreten. Seine Ausgabe ist stets ein Kandidat, niemals ein Zustand. Sie durchläuft ohne Ausnahme C1–C11. Ein Modellaufruf MUSS Prompt-Digest, Modellversion, Sampling-Parameter und Seed im RunDescriptor binden oder als `recorded_external` deklariert werden.

Deployment

Definition 28.1 (Vier Betriebsprofile).

Profil	Effektrechte	Bedingung
readonly	keine	M16 durch Null-Adapter ersetzt; G-EFFECT liefert stets HOLD.
shadow	keine	Plan und Token werden erzeugt, aber sofort invalidiert; vollständige Gate- und Reconciliationsimulation gegen aufgezeichnete Records.
sandbox	begrenzt, reversibel	Nur <code>fs.write.sandbox</code> innerhalb des Tokenscope; Rollback verpflichtend.
reference	begrenzt, reversibel	Wie <code>sandbox</code> , zusätzlich Baselinevergleich und Zertifikatserzeugung.

Regel 28.2 (Profilübergang). `readonly` $\rightarrow$ `shadow` $\rightarrow$ `sandbox` $\rightarrow$ `reference`. Jeder Übergang erfordert G-PROFILE mit bestandenem Konformanzlauf der Zielklasse. Ein Rücksprung ist jederzeit ohne Gate zulässig.

Vertrag 28.3 (Simulation before apply). Jede Änderung außerhalb einer vorab genehmigten Niedrigsrisikozone MUSS die Stufen *Sandbox*, *Shadow*, *begrenzter Produktionsausschnitt* und *volle Aktivierung* durchlaufen.

Regel 28.4 (Einzelrechnerbetrieb). Die Referenzmaschine läuft vollständig auf einem gewöhnlichen Einzelrechner. Erforderliche Prozesse: Kernprozess (M00–M15, M18–M27), Effektprozess (M16 plus `EffectAdapter`), Beobachterprozess (M17 plus `ObserverAdapter`). Die drei Prozesse besitzen getrennte Benutzerkontexte oder Namespaces.

Regel 28.5 (Live-Validierung). Der Betreiber MUSS auf einem gewöhnlichen Rechner beobachten können: (1) welche Außenrecords den aktuellen Anker bilden; (2) welche Gedankenkörper und Statuswerte erzeugt wurden; (3) welche Feldidentitäten aktiv sind; (4) welche Abhängigkeiten ihre Resultate teilen; (5) wo Verklebung gelingt oder Obstruktionen verbleiben; (6) warum ein Gate PASS, HOLD oder FAIL liefert; (7) welcher Effektplan an welches Token gebunden ist; (8) welcher Außenrecord nach Ausführung entstand; (9) wie Reconciliation und Faktpromotion entschieden wurden; (10) ob der Lauf deterministisch replaybar ist. Diese zehn Sichten sind Pflichtausgaben von `psk inspect`.

TEIL IX

Fehlerarchitektur, Beobachtung und Zertifizierung

Fehlerarchitektur

29.1 Fehlerdomänen

Definition 29.1 (Fehlerdomänen). ID, SCHEMA, ANCHOR, FACT, REAL, DEP, SCOPE, GLUE, EFFECT, RECON, REPLAY, RES, MORPH, PERSONA.

Struktur 29.2 (Fehlerartefakt).

```
Error:
  code: ErrorCode # stabil, maschinenlesbar
  domain: ErrorDomain
  severity: blocking | non_blocking
  scope: ScopeExpr
  affected_ids: [ObjectId]
  module: ModuleId
  port_id: PortId?
  trace_head: Digest
  gate_state: PASS | HOLD | FAIL | NONE
  residue_ref: ObjectId # Pflicht bei severity=blocking
  allowed_recovery: [RecoveryPathId]
  message: string # nicht normativ, nicht identitätsbildend
```

Vertrag 29.3 (Fehlerartefakt). Jeder Fehler MUSS Code, Domäne, Scope, betroffene Identitäten, Tracekopf, Gatezustand, Residuum und zulässige Recoverypfade enthalten. Ein Fehler ohne Residuenreferenz bei blocking ist selbst ein Defekt.

Invariante 29.4 (Codestabilität). Ein Fehlercode DARF NICHT nach Vergabe umgewidmet werden. Eine geänderte Bedeutung erzeugt einen neuen Code; der alte wird als DEPRECATED geführt.

29.2 Fehlerreaktionsmatrix

Reaktion	Auslöser	Wirkung
HOLD	unentscheidbare blocking Bedingung	Objekt bleibt erhalten; Residuum geöffnet; Wiederaufnahme über G-RESUME.
FAIL	verletzte blocking Bedingung	Übergang verweigert; Residuum geöffnet; kein Effekt.
Quarantäne	fehlendes Artefakt, Kettenbruch, unbestimmter Effekt	Objekt isoliert, lesbar, nicht verwendbar; nur über Recovery lösbar.
Fork	inkompatible, aber je für sich gültige Sektionen	Zwei Branches mit eigener Identität und Linage.
Retraktion	nachträglich widerlegte Entscheidung	Neues Ereignis mit Supersessionsverweis; kein Überschreiben.
Freeze	blocking Drift	Keine neuen Effekte; nur Diagnose und Replay.
Rollback	Sicherheits- oder Replayregression	Rückkehr auf letzte zertifizierte Implementierung.

Invariante 29.5 (Keine stille Reaktion). Jede der sieben Reaktionen erzeugt genau ein TraceSegment und mindestens ein ResidueRecord, außer bei PASS. Eine Reaktion ohne Aufzeichnung ist ein Architekturdefekt.

Beobachtung: Logging, Profiling, Debugging

30.1 Autoritätsfreiheit

Axiom 30.1 (Beobachtung erzeugt keine Geltung). M_{27} besitzt keine Autorität. Es erzeugt keine Objekte des Kerns, keine Gates, keine Promotionen und keine Residuen. Alle Ausgaben sind Ableitungen abgeschlossener Artefakte.

Invariante 30.2 (Kein Beobachtungspfad in den Kern). Es existiert keine Kante $M_{27} \rightarrow X$ für $X \in \mathcal{M} \setminus \{M_{27}\}$.

30.2 Logging

Regel 30.3 (Logmodell). Es existiert genau eine Wahrheitsquelle für Ereignisse: der Trace in M_{19} . Ein konventionelles Log ist eine *Projektion* des Trace und DARF NICHT Ereignisse enthalten, die nicht im Trace stehen. Logstufen (error, warn, info, debug) sind Filter über event_type und severity, keine eigene Semantik.

Regel 30.4 (Redaktion). Ein Domänenprofil DARF Felder in der Logprojektion redigieren. Die Redaktion DARF NICHT den Trace verändern; sie wird als Projektionsregel deklariert und im ViewArtifact vermerkt.

30.3 Profiling

Struktur 30.5 (Profilmetriken).

```
profile_metrics:
  per_phase: {phase: PhaseId, wall_ns, tau_i_steps, objects_in, objects_out}
  per_module: {module: ModuleId, calls, wall_ns, allocations, residues_emitted}
  per_pass: {pass: PassId, nodes, edges, cells_closed, seams_failed}
  per_gate: {gate: GateId, pass, hold, fail, mean_inputs}
  budgets: BudgetLedger
  ratchet: {capsules, rounds_total, crystallized, residualized}
```

Invariante 30.6 (Profiling ist replayneutral). Profilmessungen DARF NICHT in Kanonisierung, Digest, Gate-Entscheidung oder Scheduling eingehen. Ein Lauf mit und ohne Profiling erzeugt denselben kanonischen Zustandsdigest.

30.4 Debugging

Regel 30.7 (Debuggingmodell). Debugging erfolgt ausschließlich durch Replay. Es existieren genau vier Operationen:

- (a) inspect — Objektgraph, Statusfelder, Verweise zu einem Zeitpunkt τ_i ;
- (b) diff — kanonische Differenz zweier Läufe oder zweier τ_i -Punkte;
- (c) why — Rückverfolgung einer Entscheidung über GateReport, Evidence, Residuen bis zum Anker;
- (d) step — deterministisches Vorwärtsschreiten eines Replays um genau eine Instruktion.

Ein Debugger DARF NICHT Zustand setzen, Gates überspringen oder Effekte auslösen.

Regel 30.8 (Divergenzlokalisierung). Bei Replaydivergenz gibt M_{27} das erste divergierende Tripel (port_id, seq, payload_digest) sowie den zugehörigen TraceSegment-Digest beider Läufe aus.

Zertifizierung, Release und Selbstkompilation

31.1 Release-Gate

Regel 31.1 (Blockierende Bedingungen). manifest_invalid; constitution_digest_mismatch; architecture_digest_mismatch; schema_failure; blocking_invariant_failure; blocking_proof_obligation_open; negative_test_failure; blocking_residue; effect_boundary_unverified; reconciliation_unverified; refinement_map_incomplete; port_coverage_incomplete.

Regel 31.2 (Pflichtberichte). conformance_report; replay_manifest; negative_test_report; residue_report; capability_audit; machine_certificate; refinement_report; sbom.

Definition 31.3 (Releaseposture). $\text{Release} \in \{ \text{REJECTED}, \text{QUARANTINED}, \text{CONFORMANT_LIMITED}, \text{CONFORMANT_REFERENCE}, \text{EXTERNALLY_VALIDATED} \}.$

Vertrag 31.4 (Verified Release). Ein Release DARF das Prädikat verified nur tragen, wenn Maschinenzertifikat, Replaymanifest, Negativtestbericht, Residuenbericht, Capability Audit, Refinementbericht und Bundlemanifest zusammen vorliegen und gemeinsam verifiziert werden.

31.2 Selbstkompilation

Definition 31.5 (Kontrollierte Rekursion).

$$S_n \xrightarrow{\text{Compile}_n} C_n \xrightarrow{\text{Run}} R_n \xrightarrow{\text{Audit}} \Omega_n \xrightarrow{\text{CRA}} S_{n+1}^{\text{cand}}.$$

Invariante 31.6 (Kein unmittelbares Selbstumschreiben). Eine laufende Instanz DARF NICHT ihren aktiven Compiler, ihre Verfassung, ihre Architektur oder ihren Canonical Core in-place ersetzen. Änderungen werden außerhalb des aktiven Vertrauensbereichs erzeugt, geprüft und durch einen separaten Aktivierungsgate übernommen.

Regel 31.7 (Self-Compile-Gate). Ein Kandidat S' DARF aktiviert werden, wenn

$$\text{ParentBound}(S', S) = 1, \quad \text{InvariantPreserving}(S', S) = 1, \quad \text{RoundTrip}(S') = 1,$$

$$\text{NegativeSuite}(S') = \text{PASS}, \quad \text{IndependentReplay}(S') = \text{PASS}, \quad \text{ResiduesVisible}(S') = 1.$$

Algorithmus 31.8 (Revisionsvorschlag).

```
function PROPOSE_SELF_REVISION(active, evidence) -> Proposal | Quarantine:
  candidate = CRA(active.residues, evidence)
  candidate_id = canonical_hash(candidate)
  assert candidate_id != active.id // sonst PSK-E012
  compiler = build_in_isolation(candidate) // getrennter Prozess
  run_negative_suite(compiler)
  run_roundtrip_suite(compiler)
  run_independent_replay(compiler) // zweiter Prozess/Rechner
  if all_pass and parent_invariants_preserved:
    return emit_activation_proposal(candidate_id) // ueber P36
  else:
    return quarantine(candidate)
```

Invariante 31.9 (Gefrorener Kern). Kernsemantik, Präzedenzordnung, Trust Boundaries, Faktpromotion und Release-Gates bleiben eingefroren, bis ein Amendment mit neuer Konstitutions- beziehungsweise Architektur-ID abgeschlossen ist. Ein Amendment erfordert Migrationsplan, Replayregression und aktualisierte Negativtests.

31.3 Maschinenakzeptanzformel

Regel 31.10 (Akzeptanz). Eine Instanz $\mathcal{M}$ ist akzeptabel, wenn

$$\begin{aligned} \text{Accept}(\mathcal{M}) = 1 \iff & \text{ConstitutionBound} \wedge \text{ArchitectureBound} \wedge \text{Canonical} \wedge \text{Typed} \\ & \wedge \text{AnchorBound} \wedge \text{RealitySeparated} \wedge \text{FactSafe} \\ & \wedge \text{CoreFieldSeparated} \wedge \text{FacetScoped} \wedge \text{DependencyQuotiented} \\ & \wedge \text{GlueSound} \wedge \text{ObstructionVisible} \wedge \text{WitnessBound} \\ & \wedge \text{GateClosed} \wedge \text{CapabilitySafe} \wedge \text{EffectTokenBound} \\ & \wedge \text{ExternallyReconciled} \wedge \text{TraceMonotone} \wedge \text{ReplayStable} \\ & \wedge \text{ResidueVisible} \wedge \text{MorphogenesisBounded} \wedge \text{PortComplete} \\ & \wedge \text{RefinementProved} \wedge \text{ArtifactComplete} \wedge \text{ReleaseGated}. \end{aligned}$$

Kein Konjunkt DARF durch einen Gesamtscore, eine Metapher, eine sprachliche Versicherung oder die Häufigkeit erfolgreicher Beispiele ersetzt werden.

TEIL X

Realisierung

Deterministischer Implementierungsplan

32.1 Kanonisierung der Phasenordnung

Regel 32.1 (Eine Phasenordnung). Es existiert genau eine Realisierungsordnung I_0 – I_8 . Die Stufenordnung des Maschinenvertrags wird vollständig in sie assimiliert und existiert nicht als parallele Leiter. Die Work Packages WP_{00} – WP_{15} sind Arbeitseinheiten *innerhalb* der Phasen, keine eigene Ordnung.

Phase	Artefakt	Work Packages	Exit-Kriterium	Klasse
I_0	Workspace	WP_{00}	Reproduzierbarer Build, CI, leere Modulgrenzen mit Portstubs, <code>MANIFEST.sha256</code> .	—
I_1	Canonical Core	WP_{01}, WP_{02}	Can idempotent, IDs deterministisch, Lock und Manifest reproduzieren I_C und I_A .	C_0
I_2	M_{13}/IR	WP_{02}, WP_{03}	Graph exakt 13/30/18, IR-Round-Trip, Schema-Suite bestanden.	C_0
I_3	Compiler	$WP_{03}, WP_{05}, WP_{06}, WP_{07}$	C_1 – C_5 vollständig und deterministisch; ungültige Kandidaten erzeugen ausschließlich Residuenobjekte. C_6 – C_{11} deklariert und HOLD.	C_1
I_4	Verifikation	$WP_{08}, WP_{09}, WP_{10}$	Negativzeugen, Ratchet, Dependency Quotient und Closure laufen.	C_1
I_5	Runtime	WP_{04}, WP_{14}	Scheduler, Trace, Replay, Shutdown und Recovery reproduzierbar.	C_2
I_6	Sandbox Effect	$WP_{11}, WP_{12}, WP_{13}$	Token, Patch, Receipt, Reconciliation und Rollback schließen.	C_3
I_7	Conformance Release	WP_{15}	Golden Run und gesamte Pflichtsuite bestanden.	C_4
I_8	Independent Replay	WP_{15}	Zweiter Prozess oder Rechner reproduziert die kanonischen Resultate.	C_5

Regel 32.2 (Phasenabhängigkeit der Passfolge). Die Passfolge verteilt sich über vier Phasen, weil ihre Module es tun:

Pässe	Phase	Tragende Module und Work Packages
C_1 – C_5	I_3	M_{22}, M_{23} (WP_{03}); M_{05}, M_{17} (WP_{05}); M_{06}, M_{07} (WP_{06}); M_{08}, M_{09} (WP_{07})
C_6, C_9	I_4	M_{10}, M_{11} (WP_{08})
C_7, C_8	I_4	M_{24} (WP_{10})
C_{10}	I_6	M_{14}, M_{15} (WP_{11})
C_{11}	I_5	M_{19} (WP_{04})

Die Passfolge ist damit vor I_6 nicht vollständig ausführbar. Das ist kein Defekt, sondern die deklarierte Ordnung:

1. In I_3 MUSS jeder Pass C_1 – C_5 vollständig und deterministisch laufen.

2. Jeder noch nicht getragene Pass MUSS als Stufe existieren, seine Vorbedingungen prüfen und HOLD mit benannter Ursache emittieren. Ein EXECUTABLE vor I6 ist ein Konformitätsdefekt und erzeugt PSK-E015.
3. Residuen MUSS in I3 als typisierte Objekte konstruierbar sein. Ihre Persistenz ist Gegenstand von I5; bis dahin werden sie im Lauf gehalten und beim Übergang verworfen, ohne dass ein Nichtpassergebnis stillschweigend verschwindet.

WP	Gegenstand	Lieferobjekt	Module
WP00	Repository und Build	Reproduzierbarer Build, CI, Lizenz, Versionierung, SBOM	—
WP01	Bundle Compiler	Parser, Canonicalizer, Lock- und Manifestprüfung, <code>verify_bundle</code>	M00–M02
WP02	Core Types	Streng typisierte Datenmodelle, 19 Sorten, Schemas, Fehlercodes	M03, M04
WP03	M _{I3} , IR, Automaten	Topologiedienst, IR-Codec, Thought-/Effect-/Field-Automaten	M22, M23
WP04	Trace Store	Append-only Events, Hash-Kette, Run Descriptor, Replay	M19
WP05	Anchor Registry	Workspaceadapter, AnchorSnapshot, Frischeprädikate	M05, M17
WP06	Thought/Reality	Thought Compiler, Reality Typer	M06, M07
WP07	Field Runtime	Sechs statische Archetypen, Lens Router	M08, M09
WP08	Dependency/Glue	Quotient, Overlaps, Seams, Obstruktionen	M10, M11
WP09	Witness/Validation	Evidenceobjekte, Validation Planner, Okklusionstypisierung	M12, M13
WP10	Adversarial Kernel	Kapseln, Split, Ratchet, Support, CRA	M24
WP11	Gate/Token	Authority-, Consequence- und Effect-Gate, Tokenservice	M14, M15
WP12	Effect Sandbox	Reversible Dateieffekte, Capability Enforcement, Beobachtertrennung	M16
WP13	Reconciliation	Receipts, Delta, ActualizationWitness	M18
WP14	Scheduler/Lifecycle	Taktfolge, Budgets, Boot, Shutdown, Recovery	M25, M26
WP15	Conformance/CLI/Bench	Testsuiten, <code>psk-CLI</code> , Inspect, Baselines, Zertifikat	M20, M21, M27

Regel 32.3 (Reihenfolgezwang). Ein Work Package DARF erst begonnen werden, wenn alle Module, von denen es Portabhängigkeiten besitzt, mindestens Stubkonformanz erreicht haben. WP12 DARF erst nach bestandem WP11 begonnen werden; WP13 erst nach WP12. Adaptive Morphogenese (M20) DARF erst nach stabilem statischem Feldkern aktiviert werden.

32.2 Minimaler Kernel Stack

Vertrag 32.4 (Minimaler geschlossener Kreis). *Der erste lauffähige Kern realisiert*

Anker → ThoughtBody → Feldprojektion → Dependency → Glue → Gate → Trace → Reconciliation.

Er MUSS Input aufnehmen, einen Anker konstruieren, Gedankenkörper typisieren, statische Feldidentitäten ausführen, Abhängigkeiten berechnen, Obstruktionen ausgeben, ein Gate entscheiden und einen vollständigen Run-Trace serialisieren können.

32.3 Referenzdomäne

Regel 32.5 (Erste Domäne). Ein isolierter, versionierter lokaler Projektordner. Der Kern arbeitet zunächst read-only, danach in einer Sandbox mit reversiblen Dateioperationen.

Regel 32.6 (Referenzauftrag). Die Maschine erhält eine versiegelte Menge von Spezifikations- und Quelltextdateien, identifiziert widersprüchliche Anforderungen, bestimmt deren jeweilige Geltung, bildet Feldprojektionen, quotientiert gemeinsame Quellen, erzeugt einen Änderungsvorschlag, lässt ihn adversarial prüfen, bindet ein Sandbox-Token, wendet den Patch an und vergleicht den tatsächlichen Dateibaum mit dem Plan.

Regel 32.7 (Feldfamilie der Referenzdomäne). Genau sechs statische Feldidentitäten: `Explorer` lokalisiert Kandidaten; `Historiker` rekonstruiert Versionen; `Falsifikator` sucht Gegenbelege; `Konstrukteur` erzeugt den Patch; `Auditor` prüft den Vertrag; `Integrator` verklebt oder erzeugt eine Obstruktion.

Regel 32.8 (Anker der Referenzdomäne). Der `AnchorSnapshot` bindet Dateihashes, Git-Commit, Zeitstempel, Rechte, Konfiguration und erlaubten Scope. Der Außenrecord entsteht nach Ausführung durch einen unabhängigen Adapter, der den tatsächlichen Dateisystemzustand liest.

Offene Implementierungsverpflichtungen

Die folgenden Punkte sind innerhalb des kanonischen Korpus logisch nicht entscheidbar. Sie sind für die Architektur nicht blocking, für den Release der jeweils genannten Konformitätsklasse blocking. Jede Auflösung erfolgt als deklarierte, versionierte Methodenwahl im Register `obligations.yaml` mit zugehörigem Test.

Offene Implementierungsverpflichtung 33.1 (OBL-001 Unabhängigkeitsrang). Für reale Modell- und Datenpipelines ist eine robuste Schätzung $\text{RankEff}(W, D)$ zu wählen. Die Norm verlangt die Eigenschaft $\text{RankEff}(W, D) \leq |\mathcal{F} / \sim_D|$ und Monotonie unter Hinzufügen korrelierter Facetten; sie schreibt keinen universell optimalen Algorithmus vor. Die gewählte Methode ist als `MethodPlugin` zu deklarieren und über T-DEP-001 zu testen. Blocking ab C2.

Offene Implementierungsverpflichtung 33.2 (OBL-002 Reality Typing). Gesetzlichkeit, Erreichbarkeit und Konstruierbarkeit sind modellrelativ und teilweise unentscheidbar. Die Implementierung MUSS Soundnessbereiche, Approximationen und UNKNOWN-Fälle sichtbar halten. Ein Verfahren, das UNKNOWN vermeidet, ist nicht konform. Blocking ab C2.

Offene Implementierungsverpflichtung 33.3 (OBL-003 Gluing-Komplexität). Für große Feldfamilien ist zu bestimmen, wann paarweise Seamttests genügen und wann höherordentliche oder globale Prüfungen notwendig sind. Die Architektur schreibt paarweise Tests plus ein Gate zweiter Ordnung als Untergrenze vor. Blocking ab C3.

Offene Implementierungsverpflichtung 33.4 (OBL-004 Externe Record-Unabhängigkeit). Die Referenzdomäne MUSS vermeiden, dass dieselbe Komponente Effekt und Receipt erzeugt. Mindestens der beobachtende Adapter oder die Datengrundlage MUSS unabhängig getrennt sein. Der Grad der erreichbaren Unabhängigkeit auf einem Einzelrechner ist domänenabhängig zu deklarieren. Blocking ab C3.

Offene Implementierungsverpflichtung 33.5 (OBL-005 Security Reduction). Kryptographische Signaturen, Content Addressing und optionale Zero-Knowledge-Profile benötigen konkrete Primitive, Schlüsselverwaltung und eigene Sicherheitsannahmen. Die Architektur legt SHA-256 als Digest fest; Signaturverfahren und Schlüsselhaltung sind domänenabhängig zu deklarieren. Blocking ab C4.

Offene Implementierungsverpflichtung 33.6 (OBL-006 Informationsgewinnmaß). Das Maß für den erwarteten Informationsgewinn in M13 ist modellrelativ. Die Norm verlangt Monotonie, Nichtnegativität und Kostenbezug; sie schreibt kein Maß vor. Blocking ab C4.

Offene Implementierungsverpflichtung 33.7 (OBL-007 Marginalgewinnschwelle). Der Schwellenwert θ_G für ΔG ist domänenabhängig. Er MUSS im Domänenprofil deklariert, im `RunDescriptor` gebunden und durch T-MORPH-001 geprüft werden. Blocking ab C4.

Invariante 33.8 (Vollständigkeit der Residuenliste). Über OBL-001–OBL-007 hinaus existieren in dieser Architektur keine offenen Entscheidungen. Jede weitere Unklarheit ist ein Defekt dieses Werkes und erzeugt einen Amendmentbedarf, keine Implementierungsfreiheit.

Phasenvertrag

Vertrag 34.1 (Abschluss der Architekturphase). *Die Architekturphase ist abgeschlossen, wenn das Architekturbündel seine eigene Architektur-ID reproduziert, jede blocking Anforderung einem Modul, einem Artefakt und einem Test zugeordnet ist, jeder Port ein Schema besitzt, jeder Automatenübergang ein Gate referenziert und jedes kanonische Objekt eine Lebensdauer, einen Owner und eine Speicherklasse besitzt.*

Regel 34.2 (Nächster alleiniger Arbeitsboden). Für die Realisierungsphase ist dieses Dokument gemeinsam mit dem kanonischen PSK-Korpus das normative Hauptartefakt. Implementierungsentscheidungen werden gegen diese Architektur geprüft, nicht durch neue allgemeine Theorie ersetzt.

Ab jetzt zulässig	Ab jetzt nicht als Fortschritt gewertet
Schema- und Interfacepräzisierung	Neue Metaphern ohne Maschinenwirkung
Implementierungsentscheidungen im verbleibenden Raum	Parallele Alternativarchitekturen
Tests, Falsifikatoren, Negativzeugen	Umbenennungen ohne Typ- oder Invariantengewinn
Domänenprofile und Plug-ins	Allgemeine Theorieerweiterung vor dem Golden Run
Kernkorrektur bei belegtem Widerspruch	Rückkehr zur offenen Assimilationsschleife ohne Befund

ENDFORMEL

Die post-symbolische Kausalsprache wird zur Maschine, wenn ihre Sorten als Typen geladen, ihre Syntax auf M_{13} platziert, ihre Ableitungen als kanonische IR emittiert, ihre Kandidaten adversarial widerlegt statt sprachlich gerettet, ihre lokalen Sektionen nur bei kompatiblen Seams verklebt, ihre Effekte ausschließlich über Plan, Gate und Token ausgeführt, ihre Außenrecords unabhängig erzeugt, ihre Fakten allein durch geschlossene Reconciliation promoviert, ihre Nichtclosuren als Residuen erhalten und ihre Läufe durch Run Descriptor, Trace und Replay reproduzierbar sind — und wenn jede konkrete Instanz durch Konstitutions-, Architektur-, Implementierungs- und Laufzeitidentität nachweist, dass sie diese Form weiterhin realisiert.

TEIL XI

Normative Register

Die folgenden Register sind die maschinenlesbare Projektion dieses Werkes. Sie werden unverändert als Dateien des Verzeichnisses `architecture/` geführt. Bei Abweichung zwischen Fließtext und Register ist das Register maßgeblich.

M₁₃-Topologieregister

Listing A.1. architecture/m13_topology.yaml

```

schema: psk.m13-topology/1.0
counts: {nodes: 13, edges: 30, cells: 18}
nodes:
  center: [c]
  inner: [i0, i1, i2, i3, i4, i5]
  outer: [o0, o1, o2, o3, o4, o5]
edge_classes:
  center_inner: {formula: "(c, i_k)", count: 6}
  inner_cycle: {formula: "(i_k, i_k+1)", count: 6}
  outer_cycle: {formula: "(o_k, o_k+1)", count: 6}
  radial: {formula: "(i_k, o_k)", count: 6}
  diagonal: {formula: "(i_k, o_k-1)", count: 6}
edges:
  - {id: E-C-0, class: center_inner, u: c, v: i0}
  - {id: E-C-1, class: center_inner, u: c, v: i1}
  - {id: E-C-2, class: center_inner, u: c, v: i2}
  - {id: E-C-3, class: center_inner, u: c, v: i3}
  - {id: E-C-4, class: center_inner, u: c, v: i4}
  - {id: E-C-5, class: center_inner, u: c, v: i5}
  - {id: E-I-0, class: inner_cycle, u: i0, v: i1}
  - {id: E-I-1, class: inner_cycle, u: i1, v: i2}
  - {id: E-I-2, class: inner_cycle, u: i2, v: i3}
  - {id: E-I-3, class: inner_cycle, u: i3, v: i4}
  - {id: E-I-4, class: inner_cycle, u: i4, v: i5}
  - {id: E-I-5, class: inner_cycle, u: i5, v: i0}
  - {id: E-O-0, class: outer_cycle, u: o0, v: o1}
  - {id: E-O-1, class: outer_cycle, u: o1, v: o2}
  - {id: E-O-2, class: outer_cycle, u: o2, v: o3}
  - {id: E-O-3, class: outer_cycle, u: o3, v: o4}
  - {id: E-O-4, class: outer_cycle, u: o4, v: o5}
  - {id: E-O-5, class: outer_cycle, u: o5, v: o0}
  - {id: E-R-0, class: radial, u: i0, v: o0}
  - {id: E-R-1, class: radial, u: i1, v: o1}
  - {id: E-R-2, class: radial, u: i2, v: o2}
  - {id: E-R-3, class: radial, u: i3, v: o3}
  - {id: E-R-4, class: radial, u: i4, v: o4}
  - {id: E-R-5, class: radial, u: i5, v: o5}
  - {id: E-D-0, class: diagonal, u: i0, v: o5}
  - {id: E-D-1, class: diagonal, u: i1, v: o0}
  - {id: E-D-2, class: diagonal, u: i2, v: o1}
  - {id: E-D-3, class: diagonal, u: i3, v: o2}
  - {id: E-D-4, class: diagonal, u: i4, v: o3}
  - {id: E-D-5, class: diagonal, u: i5, v: o4}
cell_classes:
  center: {formula: "(c, i_k, i_k+1)", role: identity_constitution_lineage}
  bridge: {formula: "(i_k, i_k+1, o_k)", role: seam_port_dependency_direction}
  boundary: {formula: "(i_k, o_k, o_k-1)", role: anchor_effect_capability_horizon}
cells:
  - {id: c0, class: center, nodes: [c, i0, i1], edges: [E-C-0, E-C-1, E-I-0]}
  - {id: c1, class: center, nodes: [c, i1, i2], edges: [E-C-1, E-C-2, E-I-1]}
  - {id: c2, class: center, nodes: [c, i2, i3], edges: [E-C-2, E-C-3, E-I-2]}
  - {id: c3, class: center, nodes: [c, i3, i4], edges: [E-C-3, E-C-4, E-I-3]}
  - {id: c4, class: center, nodes: [c, i4, i5], edges: [E-C-4, E-C-5, E-I-4]}
  - {id: c5, class: center, nodes: [c, i5, i0], edges: [E-C-5, E-C-0, E-I-5]}
  - {id: b0, class: bridge, nodes: [i0, i1, o0], edges: [E-I-0, E-D-1, E-R-0]}
  - {id: b1, class: bridge, nodes: [i1, i2, o1], edges: [E-I-1, E-D-2, E-R-1]}
  - {id: b2, class: bridge, nodes: [i2, i3, o2], edges: [E-I-2, E-D-3, E-R-2]}
  - {id: b3, class: bridge, nodes: [i3, i4, o3], edges: [E-I-3, E-D-4, E-R-3]}
  - {id: b4, class: bridge, nodes: [i4, i5, o4], edges: [E-I-4, E-D-5, E-R-4]}
  - {id: b5, class: bridge, nodes: [i5, i0, o5], edges: [E-I-5, E-D-0, E-R-5]}
  - {id: o0, class: boundary, nodes: [i0, o0, o5], edges: [E-R-0, E-O-5, E-D-0]}
  - {id: o1, class: boundary, nodes: [i1, o1, o0], edges: [E-R-1, E-O-0, E-D-1]}
  - {id: o2, class: boundary, nodes: [i2, o2, o1], edges: [E-R-2, E-O-1, E-D-2]}

```

```
- {id: o3, class: boundary, nodes: [i3, o3, o2], edges: [E-R-3, E-O-2, E-D-3]}
- {id: o4, class: boundary, nodes: [i4, o4, o3], edges: [E-R-4, E-O-3, E-D-4]}
- {id: o5, class: boundary, nodes: [i5, o5, o4], edges: [E-R-5, E-O-4, E-D-5]}
address_grammar: 'm13:<scale_path>/<cell_id>[/<node_id>]'
placement:
  by_sort:
    center: [S-IDT, S-CTX, S-BRN]
    bridge: [S-PRJ, S-FLD, S-DEP, S-STA, S-TRN, S-EVT]
    boundary: [S-ANC, S-CAP, S-EFF, S-RCP, S-RCN, S-HOR]
    cell_bound: [S-WIT, S-GAT, S-TRC, S-RES]
  index_rule: "k = H(Can(node)) mod 6; bei Konflikt aufsteigende Sondierung"
closure:
  close360: "pi(Phi(x)) == pi(x) and Seam(Phi,x) == 1"
  close720: "Phi^2(x) ==can x and Hol(Phi^2) == I and Replay(Phi^2) ==can x"
  required_for_executable: close720
scale:
  max_depth: declared_in_runtime_manifest
  unresolved_remainder: {sort: S-HOR, residue_type: horizon}
```

Sorten- und Typregister

Listing B.1. architecture/sort_registry.yaml (Auszug: Sorten)

```
schema: psk.sort-registry/1.0
count: 19
sorts:
  - {id: S-IDT, name: Identity, owner: M04, cell_class: center}
  - {id: S-ANC, name: Anchor, owner: M05, cell_class: boundary}
  - {id: S-CTX, name: Context, owner: M06, cell_class: center}
  - {id: S-EVT, name: Event, owner: M19, cell_class: bridge}
  - {id: S-STA, name: State, owner: M23, cell_class: bridge}
  - {id: S-TRN, name: Transition, owner: M23, cell_class: bridge}
  - {id: S-PRJ, name: Projection, owner: M09, cell_class: bridge}
  - {id: S-FLD, name: FieldIdentity, owner: M08, cell_class: bridge}
  - {id: S-WIT, name: Witness, owner: M12, cell_class: cell_bound}
  - {id: S-TRC, name: Trace, owner: M19, cell_class: cell_bound}
  - {id: S-RES, name: Residue, owner: M19, cell_class: cell_bound}
  - {id: S-GAT, name: Gate, owner: M14, cell_class: cell_bound}
  - {id: S-CAP, name: Capability, owner: M15, cell_class: boundary}
  - {id: S-EFF, name: Effect, owner: M16, cell_class: boundary}
  - {id: S-RCP, name: Receipt, owner: M17, cell_class: boundary}
  - {id: S-RCN, name: Reconciliation, owner: M18, cell_class: boundary}
  - {id: S-DEP, name: Dependency, owner: M10, cell_class: bridge}
  - {id: S-HOR, name: Horizon, owner: M07, cell_class: boundary}
  - {id: S-BRN, name: Branch, owner: M24, cell_class: center}
```

Listing B.2. architecture/sort_registry.yaml (Auszug: Sorten-Port-Matrix)

```
port_matrix: # zulaessige (source_sort, target_sort, relation_sort)
  - [S-ANC, S-CTX, grounds]
  - [S-CTX, S-STA, scopes]
  - [S-STA, S-TRN, enables]
  - [S-TRN, S-STA, yields]
  - [S-STA, S-EVT, emits]
  - [S-EVT, S-TRC, records]
  - [S-IDT, S-BRN, forks]
  - [S-BRN, S-IDT, rebinds]
  - [S-FLD, S-PRJ, projects]
  - [S-PRJ, S-STA, views]
  - [S-PRJ, S-DEP, shares_source]
  - [S-DEP, S-WIT, quotients]
  - [S-WIT, S-GAT, supports]
  - [S-GAT, S-CAP, authorizes]
  - [S-CAP, S-EFF, permits]
  - [S-EFF, S-RCP, observed_by]
  - [S-RCP, S-RCN, feeds]
  - [S-RCN, S-STA, promotes]
  - [S-HOR, S-STA, bounds]
  - [S-RES, S-STA, blocks]
  - [S-GAT, S-RES, opens]
  - [S-TRC, S-RES, preserves]
  - [S-ANC, S-HOR, defines]
forbidden_relations:
  - [S-PRJ, S-RCN, any] # Feldprojektion promoviert keinen Fakt
  - [S-EFF, S-RCP, produced_by] # Effektor erzeugt keinen eigenen Receipt
  - [S-RES, S-STA, silently_closes]
  - [S-IDT, S-CAP, any] # Identitaet erzeugt keine Faehigkeit
```

Listing B.3. architecture/object_registry.yaml (Kurzform)

```
schema: psk.object-registry/1.0
objects:
  - {id: OBJ-RTM, name: RuntimeManifest, owner: M04, lifetime: L-RUN, memory: canonical}
  - {id: OBJ-IDB, name: IdentityBinding, owner: M04, lifetime: L-RUN, memory: canonical}
```

```

- {id: OBJ-ANC, name: AnchorSnapshot, owner: M05, lifetime: L-EPOCH, memory: canonical}
- {id: OBJ-THB, name: ThoughtBody, owner: M06, lifetime: L-RUN, memory: working}
- {id: OBJ-FLD, name: FieldIdentity, owner: M08, lifetime: L-RUN, memory: working}
- {id: OBJ-PRJ, name: FieldProjection, owner: M09, lifetime: L-TICK, memory: working}
- {id: OBJ-RVP, name: RevisionProposal, owner: M24, lifetime: L-PERSIST, memory: canonical}
- {id: OBJ-FCT, name: Forecast, owner: M07, lifetime: L-PERSIST, memory: canonical}
- {id: OBJ-RCL, name: RealityClassification, owner: M07, lifetime: L-RUN, memory: canonical}
- {id: OBJ-DEP, name: DependencyProfile, owner: M10, lifetime: L-RUN, memory: canonical}
- {id: OBJ-IRB, name: IRBundle, owner: M23, lifetime: L-RUN, memory: working}
- {id: OBJ-IRN, name: IRNode, owner: M23, lifetime: L-RUN, memory: working}
- {id: OBJ-IRE, name: IREdge, owner: M23, lifetime: L-RUN, memory: working}
- {id: OBJ-CAP, name: CandidateCapsule, owner: M24, lifetime: L-RUN, memory: working}
- {id: OBJ-SEA, name: SeamReport, owner: M11, lifetime: L-TICK, memory: working}
- {id: OBJ-OBS, name: ObstructionRecord, owner: M11, lifetime: L-PERSIST, memory: residual}
- {id: OBJ-EVD, name: EvidenceObject, owner: M12, lifetime: L-IMMUTABLE, memory: witness}
- {id: OBJ-VAL, name: ValidationPlan, owner: M13, lifetime: L-EPOCH, memory: working}
- {id: OBJ-GAT, name: GateReport, owner: M14, lifetime: L-IMMUTABLE, memory: canonical}
- {id: OBJ-TOK, name: EffectToken, owner: M15, lifetime: L-EPOCH, memory: canonical}
- {id: OBJ-ATT, name: EffectAttempt, owner: M16, lifetime: L-IMMUTABLE, memory: canonical}
- {id: OBJ-RCP, name: ExternalReceipt, owner: M17, lifetime: L-IMMUTABLE, memory: witness}
- {id: OBJ-RCN, name: ReconciliationReport, owner: M18, lifetime: L-IMMUTABLE, memory:
  canonical}
- {id: OBJ-TRC, name: TraceSegment, owner: M19, lifetime: L-PERSIST, memory: replay}
- {id: OBJ-RES, name: ResidueRecord, owner: M19, lifetime: L-PERSIST, memory: residual}
- {id: OBJ-RUN, name: RunDescriptor, owner: M19, lifetime: L-IMMUTABLE, memory: replay}
- {id: OBJ-RPM, name: ReplayManifest, owner: M19, lifetime: L-IMMUTABLE, memory: replay}
- {id: OBJ-EXC, name: ExcisionCertificate, owner: M20, lifetime: L-IMMUTABLE, memory:
  canonical}
- {id: OBJ-CRT, name: MachineCertificate, owner: M21, lifetime: L-IMMUTABLE, memory: archive}
- {id: OBJ-VIW, name: ViewArtifact, owner: M27, lifetime: L-TICK, memory: none, canonical:
  false}

```

Listing B.4. architecture/volatile_fields.yaml

```

schema: psk.volatile-fields/1.0
purpose: >
  Abgeschlossene Menge der Felder, die pi_vol vor der Identitätsbildung
  rekursiv entfernt. Zwei Gründe, ein Mechanismus: volatile Felder sind
  nicht reproduzierbar, non_canonical Felder sind reproduzierbar, aber
  nicht Teil des kanonischen Zustands. Es existiert kein zweiter
  Ausschlusspfad neben diesem Register.
  Objekt-ID = psk:<sorte>:H(Can(pi_vol(o))).
  record_digest = H(Can(o)) ueber das vollstaendige Objekt.
volatile: # nicht reproduzierbar zwischen Laufen
- path: DualTime.tau_e # Wanduhr, Invariante Replayneutralitaet
- path: DualTime.clock_ref # Identitaet/Kalibrierstatus der Uhr
- path: DualTime.uncertainty_ns # deklarierte Messunsicherheit
- path: "*.runtime_metrics" # Laufzeitmesswerte
- path: "*.persona_projection" # Ausgabeprojektionen, M27
- path: "*.view_*" # Beobachtungsprojektionen, M27
- path: "*.segment_record_digest" # deckt tau_e ab, Regel Zwei Digests je Segment
non_canonical: # reproduzierbar, aber nicht kanonischer Zustand
- path: ThoughtBody.claim.text # Axiom "Gedanke ist kein Satz"
not_removed:
- path: DualTime.tau_i # Kausalzaehler, je run_id deterministisch
enforcement:
  gate_input_forbidden: [volatile, record_digest]
  error_on_violation: PSK-E015

```

Modul- und Portregister

Listing C.1. architecture/module_map.yaml (Kurzform)

```

schema: psk.module-map/1.0
count: 28
layers: [L0, L1, L2, L3, L4, L5, L6, L7]
modules:
  - {id: M00, name: ConstitutionLoader, layer: L1, package: psk-contract, caps: [fs.read.bundle
    ]}
  - {id: M01, name: Canonicalizer, layer: L1, package: psk-canon, caps: []}
  - {id: M02, name: ArtifactRegistry, layer: L1, package: psk-contract, caps: []}
  - {id: M03, name: SchemaValidator, layer: L1, package: psk-types, caps: []}
  - {id: M04, name: IdentityBinder, layer: L1, package: psk-contract, caps: []}
  - {id: M05, name: AnchorRegistry, layer: L2, package: psk-anchor, caps: [fs.read.workspace]}
  - {id: M06, name: ThoughtCompiler, layer: L3, package: psk-thought, caps: []}
  - {id: M07, name: RealityTyper, layer: L3, package: psk-thought, caps: []}
  - {id: M08, name: FieldRegistry, layer: L4, package: psk-fields, caps: []}
  - {id: M09, name: SpectralLensRouter, layer: L4, package: psk-fields, caps: []}
  - {id: M10, name: DependencyAnalyzer, layer: L4, package: psk-dependency, caps: []}
  - {id: M11, name: ClosureGlueEngine, layer: L4, package: psk-closure, caps: []}
  - {id: M12, name: WitnessEngine, layer: L5, package: psk-witness, caps: []}
  - {id: M13, name: ValidationPlanner, layer: L5, package: psk-witness, caps: []}
  - {id: M14, name: AuthorityConsequenceGate, layer: L5, package: psk-gate, caps: []}
  - {id: M15, name: EffectTokenService, layer: L6, package: psk-effect, caps: [token.issue]}
  - {id: M16, name: EffectBoundary, layer: L6, package: psk-effect, caps: [adapter_declared]}
  - {id: M17, name: ExternalRecordIngress, layer: L6, package: psk-anchor, caps: [fs.read.
    observer]}
  - {id: M18, name: ReconciliationEngine, layer: L6, package: psk-reconciliation, caps: []}
  - {id: M19, name: TraceReplayResidueStore, layer: L7, package: psk-trace, caps: [store.append,
    store.read]}
  - {id: M20, name: MorphogenesisController, layer: L7, package: psk-fields, caps: []}
  - {id: M21, name: CertificateReleaseEngine, layer: L7, package: psk-certify, caps: [key.sign]}

  - {id: M22, name: M13TopologyService, layer: L3, package: psk-topology, caps: []}
  - {id: M23, name: IRCCodec, layer: L4, package: psk-ir, caps: []}
  - {id: M24, name: AdversarialKernel, layer: L4, package: psk-adversarial, caps: []}
  - {id: M25, name: Scheduler, layer: L0, package: psk-scheduler, caps: [clock.read]}
  - {id: M26, name: LifecycleSupervisor, layer: L0, package: psk-lifecycle, caps: [proc.control
    ]}
  - {id: M27, name: ObservabilityProjector, layer: L7, package: psk-observe, caps: [store.read]}

forbidden_edges: # Quelle darf Ziel nicht aufrufen
  - [M16, M15] # Effektor stellt kein eigenes Token aus
  - [M16, M17] # Effektor erzeugt keinen eigenen Receipt
  - [M18, M17] # Reconciliation beschafft Records nicht selbst
  - [M08, M18] # Feld promoviert keinen Fakt
  - [M09, M18] # Linse promoviert keinen Fakt
  - [M24, M00] # Adversarialkern aendert die Konstitution nicht
  - [M24, M01] # Adversarialkern aendert die Kanonisierung nicht
  - ["*", M19, delete_or_overwrite] # Trace ist append-only
  - [M27, "*"] # Beobachtung hat keine Schreibkante

```

Listing C.2. architecture/port_registry.yaml (Schema und Ausschnitt)

```

schema: psk.port-registry/1.0
count: 42
envelope: psk.message/1.0
delivery: exactly_once
idempotency_key: [run_id, port_id, seq]
persist_before_process: true
ports:
  - {id: P00, from: M26, to: M00, payload: BootRequest, kind: request, errors: [PSK-E101]}
  - {id: P01, from: M00, to: M01, payload: BundleImage, kind: request, errors: [PSK-E102]}
  - {id: P02, from: M01, to: M02, payload: CanonicalBytes/Digest, kind: request, errors: [PSK-
    E102]}

```

```

- {id: P03, from: M02, to: M03, payload: ArtifactRecord, kind: request, errors: [PSK-E001]}
- {id: P04, from: M03, to: M04, payload: SchemaReport, kind: request, errors: [PSK-E001]}
- {id: P05, from: M04, to: M26, payload: IdentityBinding, kind: request, errors: [PSK-E103]}
- {id: P06, from: M17, to: M05, payload: ExternalRecord, kind: event, errors: [PSK-E002]}
- {id: P07, from: M05, to: M06, payload: AnchorSnapshot, kind: request, errors: [PSK-E002]}
- {id: P08, from: M06, to: M07, payload: ThoughtBody, kind: request, errors: [PSK-E001]}
- {id: P09, from: M07, to: M23, payload: RealityClassification, kind: request, errors: [PSK-E004]}
- {id: P10, from: M22, to: M23, payload: M13Chart, kind: request, errors: [PSK-E011]}
- {id: P11, from: M23, to: M08, payload: IRBundle, kind: request, errors: [PSK-E001]}
- {id: P12, from: M08, to: M09, payload: FieldIdentity, kind: request, errors: [PSK-E013]}
- {id: P13, from: M09, to: M10, payload: FieldProjection, kind: request, errors: [PSK-E013]}
- {id: P14, from: M10, to: M24, payload: DependencyProfile, kind: request, errors: [PSK-E006]}

- {id: P15, from: M24, to: M11, payload: CandidateCapsule, kind: request, errors: [PSK-E003]}
- {id: P16, from: M11, to: M22, payload: SeamQuery, kind: request, errors: [PSK-E011]}
- {id: P17, from: M22, to: M11, payload: CellReport, kind: request, errors: [PSK-E011]}
- {id: P18, from: M11, to: M12, payload: ClosureReport, kind: request, errors: [PSK-E011]}
- {id: P19, from: M12, to: M13, payload: WitnessReport, kind: request, errors: [PSK-E005]}
- {id: P20, from: M13, to: M14, payload: ValidationPlan, kind: request, errors: [PSK-E005]}
- {id: P21, from: M14, to: M15, payload: GateReportPass, kind: request, errors: [PSK-E007]}
- {id: P22, from: M15, to: M16, payload: EffectToken, kind: request, errors: [PSK-E008]}
- {id: P23, from: M16, to: M19, payload: EffectAttempt, kind: request, errors: [PSK-E008]}
- {id: P24, from: M17, to: M18, payload: ExternalReceipt, kind: event, errors: [PSK-E009]}
- {id: P25, from: M18, to: M12, payload: ReconciliationReport, kind: request, errors: [PSK-E009]}
- {id: P26, from: M18, to: M06, payload: ReanalysisRequest, kind: event, errors: [PSK-E010]}
- {id: P27, from: M18, to: M05, payload: ReanchorRequest, kind: event, errors: [PSK-E002]}
- {id: P28, from: "*", to: M19, payload: TraceSegment/ResidueRecord, kind: request, errors: [PSK-E014], sync: true}
- {id: P29, from: M19, to: M25, payload: ReplayCursor, kind: event, errors: [PSK-E015]}
- {id: P30, from: M25, to: "*", payload: TickGrant, kind: event, errors: [PSK-E016]}
- {id: P31, from: M20, to: M08, payload: MorphogenesisDecision, kind: request, errors: [PSK-E017]}
- {id: P32, from: M08, to: M20, payload: SpawnRequest, kind: request, errors: [PSK-E017]}
- {id: P33, from: M14, to: M19, payload: GateReport, kind: request, errors: [PSK-E014], sync: true}
- {id: P34, from: M19, to: M21, payload: ReplayManifest, kind: request, errors: [PSK-E015]}
- {id: P35, from: M21, to: M26, payload: ReleaseReport, kind: request, errors: [PSK-E018]}
- {id: P36, from: M24, to: M21, payload: RevisionProposal, kind: event, errors: [PSK-E012]}
- {id: P37, from: M26, to: M16, payload: TokenInvalidate, kind: event, errors: [PSK-E008]}
- {id: P38, from: M19, to: M27, payload: ViewQuery, kind: request, errors: [], readonly: true}

- {id: P39, from: M13, to: M09, payload: ProbeRequest, kind: event, errors: [PSK-E005]}
- {id: P40, from: M20, to: M19, payload: ExcisionCertificate, kind: request, errors: [PSK-E014]}
- {id: P41, from: M26, to: M25, payload: SchedulerControl, kind: request, errors: [PSK-E016]}
complete: true # alle 42, keine externe Kapitelreferenz mehr noetig

```

Automatenregister

Listing D.1. architecture/state_machines.yaml (vollständig)

```

schema: psk.state-machines/1.1
machines: [runtime, object, thought, effect, field, capsule, token]

runtime:
  initial: BOOTING
  states: [BOOTING, BOUND, SHADOW, READY, EXECUTING, RECONCILING,
    HOLD, QUARANTINE, FAILED, STOPPED]
  transitions:
    - {from: BOOTING, to: BOUND, operator: bind_identity, gate: G-BOOT}
    - {from: BOUND, to: SHADOW, operator: enter_shadow, gate: G-PROFILE}
    - {from: BOUND, to: READY, operator: enter_ready, gate: G-PROFILE}
    - {from: SHADOW, to: READY, operator: promote_profile, gate: G-PROFILE}
    - {from: READY, to: EXECUTING, operator: dispatch_tick, gate: G-SCHEDULE}
    - {from: EXECUTING, to: RECONCILING, operator: await_receipt, gate: G-OBSERVATION}
    - {from: RECONCILING, to: READY, operator: close_cycle, gate: G-ACTUALIZATION}
    - {from: "*", to: HOLD, operator: hold, gate: G-RESIDUE}
    - {from: HOLD, to: READY, operator: resume, gate: G-RESUME}
    - {from: "*", to: QUARANTINE, operator: quarantine, gate: G-RESIDUE}
    - {from: "*", to: FAILED, operator: fail, gate: null}
    - {from: "*", to: STOPPED, operator: shutdown, gate: G-SHUTDOWN}
  forbidden: [BOOTING->EXECUTING, SHADOW->EXECUTING, EXECUTING->READY, QUARANTINE->READY]

object:
  initial: CREATED
  states: [CREATED, TYPED, ANCHORED, PROJECTED, COMPILED, VERIFIED, RUNNABLE,
    EXECUTING, OBSERVED, RECONCILED, ARCHIVED,
    HOLD, RESIDUAL, QUARANTINE, REJECTED]
  transitions:
    - {from: CREATED, to: TYPED, operator: type, gate: null}
    - {from: TYPED, to: ANCHORED, operator: anchor, gate: null}
    - {from: ANCHORED, to: PROJECTED, operator: project, gate: null}
    - {from: PROJECTED, to: COMPILED, operator: compile, gate: null}
    - {from: COMPILED, to: VERIFIED, operator: verify, gate: G-CLOSURE}
    - {from: VERIFIED, to: RUNNABLE, operator: bind_gates, gate: G-EFFECT}
    - {from: RUNNABLE, to: EXECUTING, operator: execute, gate: null}
    - {from: EXECUTING, to: OBSERVED, operator: observe, gate: G-OBSERVATION}
    - {from: OBSERVED, to: RECONCILED, operator: reconcile, gate: G-ACTUALIZATION}
    - {from: RECONCILED, to: ARCHIVED, operator: archive, gate: null}
    - {from: "*", to: HOLD, operator: hold, gate: G-RESIDUE}
    - {from: "*", to: RESIDUAL, operator: residualize, gate: G-RESIDUE}
    - {from: "*", to: QUARANTINE, operator: quarantine, gate: G-RESIDUE}
    - {from: "*", to: REJECTED, operator: reject, gate: null}
  forbidden: [PROJECTED->RECONCILED, COMPILED->RECONCILED, VERIFIED->ARCHIVED]

thought:
  initial: SPECIFIED
  states: [SPECIFIED, POSSIBLE, SIMULATED, SELECTED, ATTEMPTED, OBSERVED,
    ACTUALIZED, RESIDUE, RETRACTED]
  transitions:
    - {from: SPECIFIED, to: POSSIBLE, operator: classify_coherent, gate: G-REALITY-COHERENT}
    - {from: POSSIBLE, to: SIMULATED, operator: simulate, gate: G-SIMULATION}
    - {from: POSSIBLE, to: SELECTED, operator: select_candidate, gate: G-SELECTION}
    - {from: SELECTED, to: ATTEMPTED, operator: execute_effect, gate: G-EFFECT}
    - {from: ATTEMPTED, to: OBSERVED, operator: bind_external_record, gate: G-OBSERVATION}
    - {from: OBSERVED, to: ACTUALIZED, operator: reconcile, gate: G-ACTUALIZATION}
    - {from: "*", to: RESIDUE, operator: route_residue, gate: G-RESIDUE}
    - {from: "*", to: RETRACTED, operator: retract, gate: G-RETRACTION}
  forbidden: [SIMULATED->ACTUALIZED, SELECTED->OBSERVED,
    ATTEMPTED->ACTUALIZED, POSSIBLE->ACTUALIZED]

effect:
  initial: DRAFT
  states: [DRAFT, GATED, TOKENIZED, ATTEMPTED, RECORDED, RECONCILED,

```

```

    ACTUALIZED, COMPENSATED, FAILED, UNKNOWN_EFFECT]
transitions:
  - {from: DRAFT, to: GATED, operator: gate_plan, gate: G-EFFECT}
  - {from: GATED, to: TOKENIZED, operator: issue_token, gate: G-EFFECT}
  - {from: TOKENIZED, to: ATTEMPTED, operator: execute, gate: null}
  - {from: ATTEMPTED, to: RECORDED, operator: observe, gate: G-OBSERVATION}
  - {from: ATTEMPTED, to: UNKNOWN_EFFECT, operator: lose_receipt, gate: null}
  - {from: RECORDED, to: RECONCILED, operator: reconcile, gate: G-ACTUALIZATION}
  - {from: RECONCILED, to: ACTUALIZED, operator: promote_fact, gate: G-ACTUALIZATION}
  - {from: ATTEMPTED, to: COMPENSATED, operator: compensate, gate: G-EFFECT}
  - {from: UNKNOWN_EFFECT, to: RECORDED, operator: independent_observe, gate: G-OBSERVATION}
  - {from: "*", to: FAILED, operator: fail, gate: null}
forbidden: [DRAFT->TOKENIZED, TOKENIZED->RECONCILED,
    ATTEMPTED->ACTUALIZED, UNKNOWN_EFFECT->ACTUALIZED]

field:
  initial: PROPOSED
  states: [PROPOSED, ACTIVE, QUIESCENT, SPLIT_PENDING, MERGE_PENDING,
    FOLDED, RETIRED, EXCISED]
  transitions:
    - {from: PROPOSED, to: ACTIVE, operator: activate, gate: G-MORPH}
    - {from: ACTIVE, to: QUIESCENT, operator: quiesce, gate: null}
    - {from: ACTIVE, to: SPLIT_PENDING, operator: propose_split, gate: G-MORPH}
    - {from: ACTIVE, to: MERGE_PENDING, operator: propose_merge, gate: G-MORPH}
    - {from: ACTIVE, to: FOLDED, operator: fold, gate: G-MORPH}
    - {from: QUIESCENT, to: RETIRED, operator: retire, gate: null}
    - {from: "*", to: EXCISED, operator: excise, gate: G-EXCISION}
  activation_requirements:
    [positive_marginal_gain, scope_declared, dependency_profile, budget, rollback, lineage]

ra_extensions: # Prinzip 4.3: Peripherie, nicht Kern. Nur aktiv,
  # wenn ein DomainProfile sie ausdruecklich einschaltet.

  thought:
    - {id: RA-EXT-THOUGHT-01, from: SIMULATED, to: SELECTED,
      operator: select_candidate, gate: G-SELECTION}
  field:
    - {id: RA-EXT-FIELD-01, from: QUIESCENT, to: ACTIVE,
      operator: reactivate, gate: G-MORPH}

capsule:
  initial: SEALED
  states: [SEALED, PERTURBED, RATCHETED, SUPPORTED, CRYSTALLIZED, RESIDUAL, QUARANTINED]
  transitions:
    - {from: SEALED, to: PERTURBED, operator: perturb, gate: null}
    - {from: PERTURBED, to: RATCHETED, operator: ratchet, gate: null}
    - {from: RATCHETED, to: RATCHETED, operator: ratchet, gate: null}
    - {from: RATCHETED, to: SUPPORTED, operator: check_support, gate: null}
    - {from: SUPPORTED, to: CRYSTALLIZED, operator: crystallize, gate: G-CLOSURE}
    - {from: "*", to: RESIDUAL, operator: residualize, gate: G-RESIDUE}
    - {from: "*", to: QUARANTINED, operator: quarantine, gate: G-RESIDUE}
  forbidden: [SEALED->CRYSTALLIZED, PERTURBED->CRYSTALLIZED]
  monotonicity: {field: allowed_next, direction: non_increasing}

token:
  initial: ISSUED
  states: [ISSUED, CONSUMED, EXPIRED, INVALIDATED, REVOKED]
  transitions:
    - {from: ISSUED, to: CONSUMED, operator: consume_once, gate: null}
    - {from: ISSUED, to: EXPIRED, operator: expire, gate: null}
    - {from: ISSUED, to: INVALIDATED, operator: plan_changed, gate: null}
    - {from: ISSUED, to: REVOKED, operator: revoke, gate: null}
  forbidden: [CONSUMED->CONSUMED, EXPIRED->CONSUMED,
    INVALIDATED->CONSUMED, REVOKED->CONSUMED]

```

Gate- und Passregister

Listing E.1. architecture/gate_registry.yaml

```

schema: psk.gate-registry/1.0
default_on_undecidable: HOLD
default_on_missing: FAIL
gates:
  - {id: G-BOOT, order: 2, owner: M14, target: kernel_active}
  - {id: G-PROFILE, order: 1, owner: M14, target: profile_change}
  - {id: G-SCHEDULE, order: 1, owner: M25, target: tick_grant}
  - {id: G-REALITY-COHERENT, order: 1, owner: M07, target: POSSIBLE}
  - {id: G-LAWFUL, order: 1, owner: M07, target: LAWFUL}
  - {id: G-REACHABLE, order: 1, owner: M07, target: REACHABLE}
  - {id: G-CONSTRUCTIBLE, order: 1, owner: M07, target: CONSTRUCTIBLE}
  - {id: G-SIMULATION, order: 1, owner: M14, target: SIMULATED}
  - {id: G-SELECTION, order: 1, owner: M14, target: SELECTED}
  - {id: G-CONSENSUS, order: 2, owner: M14, target: scoped_pass}
  - {id: G-CLOSURE, order: 2, owner: M11, target: canonical_commit}
  - {id: G-EFFECT, order: 2, owner: M14, target: tokenize}
  - {id: G-OBSERVATION, order: 1, owner: M17, target: OBSERVED}
  - {id: G-ACTUALIZATION, order: 2, owner: M18, target: ACTUALIZED}
  - {id: G-RESIDUE, order: 1, owner: M19, target: residue_visible}
  - {id: G-RETRACTION, order: 1, owner: M19, target: RETRACTED}
  - {id: G-MORPH, order: 1, owner: M20, target: field_transition}
  - {id: G-EXCISION, order: 2, owner: M20, target: excision_effective}
  - {id: G-SELF-COMPILE, order: 2, owner: M21, target: activation_proposal}
  - {id: G-RESUME, order: 1, owner: M26, target: READY}
  - {id: G-SHUTDOWN, order: 1, owner: M26, target: STOPPED}
  - {id: G-RELEASE, order: 2, owner: M21, target: release}
report_required_always: true

```

Listing E.2. architecture/pass_registry.yaml

```

schema: psk.pass-registry/1.0
passes:
  - {id: C1, name: CanonicalParse, modules: [M01, M06], errors: [PSK-E001]}
  - {id: C2, name: TypeAndSortCheck, modules: [M03, M06], errors: [PSK-E001]}
  - {id: C3, name: AnchorBinding, modules: [M05, M07], errors: [PSK-E002]}
  - {id: C4, name: M13SyntaxPlacement, modules: [M22], errors: [PSK-E011]}
  - {id: C5, name: FieldProjection, modules: [M08, M09], errors: [PSK-E013]}
  - {id: C6, name: DependencyQuotient, modules: [M10], errors: [PSK-E006]}
  - {id: C7, name: AdversarialCanonicalization, modules: [M24], errors: [PSK-E003]}
  - {id: C8, name: SupportAndReachability, modules: [M24, M07], errors: [PSK-E004, PSK-E005]}
  - {id: C9, name: ClosureAndGluings, modules: [M11, M22], errors: [PSK-E011]}
  - {id: C10, name: GateAndCapabilityBinding, modules: [M14, M15], errors: [PSK-E007]}
  - {id: C11, name: TraceReplayEmission, modules: [M19, M23], errors: [PSK-E015]}
monotonicity: later_pass_may_not_silently_clear_earlier_error_class
determinism: pure_function_of_declared_inputs_and_run_descriptor
emission_classes: [EXECUTABLE, RESIDUAL, QUARANTINED, HOLD]
executable_requires:
  [fully_typed, anchor_bound_or_declared_unanchored, all_18_cells_closed,
   close720, unique_global_section, all_blocking_gates_pass, no_blocking_residue]

```


Invariantenregister

Listing F.1. constitution/invariants.yaml (kanonischer Kern)

```
schema: psk.invariants/1.0
invariants:
  - {id: I-ID-001, scope: boot, severity: blocking,
    predicate: runtime.constitution_id == lock.constitution_id}
  - {id: I-TYPE-001, scope: global, severity: blocking,
    predicate: all_committed_objects_validate_against_schema}
  - {id: I-ANCHOR-001, scope: claim, severity: blocking,
    predicate: every_external_claim_references_anchor_or_is_marked_unanchored}
  - {id: I-REAL-001, scope: thought, severity: blocking,
    predicate: reality_status_monotone_only_via_named_constructor}
  - {id: I-FACT-001, scope: thought, severity: blocking,
    predicate: facticity_status_monotone_only_via_named_constructor}
  - {id: I-FIELD-001, scope: field, severity: blocking,
    predicate: no_field_identity_equals_system_identity}
  - {id: I-DEP-001, scope: consensus, severity: blocking,
    predicate: consensus_uses_effective_witness_rank_not_facet_count}
  - {id: I-SCOPE-001, scope: consensus, severity: blocking,
    predicate: consensus_scope_never_widens_without_gate}
  - {id: I-GATE-001, scope: effect, severity: blocking,
    predicate: no_external_effect_without_PASS_and_effect_token}
  - {id: I-RECON-001, scope: effect, severity: blocking,
    predicate: no_actualization_without_external_record_and_reconciliation}
  - {id: I-TRACE-001, scope: global, severity: blocking,
    predicate: trace_is_append_only_and_every_commit_has_trace_head}
  - {id: I-REPLAY-001, scope: run, severity: blocking,
    predicate: equal_run_descriptor_and_input_digest_yield_equal_canonical_outcome}
  - {id: I-RES-001, scope: global, severity: blocking,
    predicate: every_nonpass_produces_visible_residue_or_hold_reason}
  - {id: I-GLUE-001, scope: global, severity: blocking,
    predicate: global_commit_requires_pairwise_overlap_compatibility_and_second_order_gate}
  - {id: I-MORPH-001, scope: field, severity: blocking,
    predicate: field_spawn_requires_gain_budget_lineage_test_and_rollback}
  - {id: I-PERSONA-001, scope: aperture, severity: blocking,
    predicate: persona_projection_has_no_intrinsic_system_authority}
  - {id: I-BOUND-001, scope: effect, severity: blocking,
    predicate: internal_freedom_does_not_create_external_capability}
```

Listing F.2. architecture/ra_invariants.yaml (Architekturkern)

```
schema: psk.ra-invariants/1.0
invariants:
  - {id: I-ARCH-001, scope: build, severity: blocking,
    predicate: runtime.architecture_id == lock.architecture_id}
  - {id: I-ARCH-002, scope: global, severity: blocking,
    predicate: module_dependency_graph_is_acyclic_and_layer_respecting}
  - {id: I-ARCH-003, scope: global, severity: blocking,
    predicate: every_cross_module_call_maps_to_exactly_one_port_P00_P41}
  - {id: I-ARCH-004, scope: global, severity: blocking,
    predicate: every_object_type_has_exactly_one_owning_module}
  - {id: I-ARCH-005, scope: global, severity: blocking,
    predicate: every_state_transition_references_a_registered_gate_or_null}
  - {id: I-ARCH-006, scope: global, severity: blocking,
    predicate: trace_segment_written_before_state_visible_at_any_port}
  - {id: I-ARCH-007, scope: effect, severity: blocking,
    predicate: effect_adapter_identity != observer_adapter_identity}
  - {id: I-ARCH-008, scope: global, severity: blocking,
    predicate: M27_has_no_outgoing_edge_into_kernel_modules}
  - {id: I-ARCH-009, scope: run, severity: blocking,
    predicate: concurrent_run_has_equivalent_sequential_canonical_digest}
  - {id: I-ARCH-010, scope: memory, severity: blocking,
    predicate: only_declared_promotion_paths_between_memory_classes}
  - {id: I-ARCH-011, scope: topology, severity: blocking,
```

```
predicate: m13_counts_equal_13_30_18}
- {id: I-ARCH-012, scope: run, severity: blocking,
  predicate: replay_profile_executes_no_real_effect}
- {id: I-ARCH-013, scope: build, severity: blocking,
  predicate: refinement_map_covers_every_concrete_transition}
- {id: I-ARCH-014, scope: global, severity: blocking,
  predicate: token_consumed_at_most_once_per_idempotency_key}
- {id: I-ARCH-015, scope: global, severity: blocking,
  predicate: profiling_and_logging_do_not_alter_canonical_digest}
- {id: I-ARCH-016, scope: plugin, severity: blocking,
  predicate: plugin_cannot_alter_sorts_topology_automata_gates_or_errors}
```

Regel F.1 (Goldene Invarianten). I1. Oberfläche erzeugt keine Invarianz.

- I2. Benennung erzeugt keine Geltung.
- I3. Simulation erzeugt keine Aktualisierung.
- I4. Interne Freiheit erzeugt keine externe Fähigkeit.
- I5. Viele Facetten erzeugen keine unabhängige Evidenz.
- I6. Kein Commit ohne Gate, Evidence und Replay.
- I7. Kein Effekt ohne gebundenes Token.
- I8. Keine Aktualisierung ohne Receipt und Reconciliation.
- I9. Kein Residuum verschwindet still.
- I10. Keine Selbständerung aktiviert sich selbst.
- I11. M_{13} ist das kanonische Syntaxskelett der v1.0.
- I12. Lokale Konformität impliziert keine globale Closure.
- I13. Jede normative Änderung erzeugt eine neue Identität.
- I14. Ein imaginärer Pfad benötigt verankerten Ein- und Austritt.
- I15. Der Compiler zählt seine eigenen Claims nicht als unabhängige Witnesses.

Fehlerkatalog

Listing G.1. constitution/error_catalog.yaml + Architekturerweiterung

```

schema: psk.error-catalog/1.1
domains: [ID, SCHEMA, ANCHOR, FACT, REAL, DEP, SCOPE, GLUE, EFFECT,
          RECON, REPLAY, RES, MORPH, PERSONA]
canonical: # unveraendert aus der Gesamtspezifikation
- {code: PSK-E001, name: untyped_input, domain: SCHEMA,
  reaction: [HOLD, trace, require_type]}
- {code: PSK-E002, name: missing_anchor, domain: ANCHOR,
  reaction: [no_external_promotion, require_anchor]}
- {code: PSK-E003, name: surface_invariant_collapse, domain: FACT,
  reaction: [REJECT_or_QUARANTINE, residue]}
- {code: PSK-E004, name: unsupported_reachability, domain: REAL,
  reaction: [mark_coherent_but_unsupported]}
- {code: PSK-E005, name: occlusion_without_trace, domain: FACT,
  reaction: [set_UNKNOWN, no_negative_witness]}
- {code: PSK-E006, name: correlated_witness_overcount, domain: DEP,
  reaction: [recompute_dependency_quotient]}
- {code: PSK-E007, name: gate_bypass, domain: EFFECT,
  reaction: [FAIL, lock_capability]}
- {code: PSK-E008, name: effect_without_token, domain: EFFECT,
  reaction: [block_effect, residue]}
- {code: PSK-E009, name: actualization_without_reconciliation, domain: RECON,
  reaction: [reject_fact_promotion]}
- {code: PSK-E010, name: same_mode_return, domain: FACT,
  reaction: [residualize_output]}
- {code: PSK-E011, name: nonclosing_m13_seam, domain: GLUE,
  reaction: [block_cell_and_system_closure]}
- {code: PSK-E012, name: self_amendment_without_identity, domain: ID,
  reaction: [force_new_branch_and_id]}
architecture_extension: # dieses Werk; kollisionsfreier Bereich
- {code: PSK-E013, name: field_projection_undefined, domain: SCOPE,
  reaction: [residue_scope, no_empty_projection]}
- {code: PSK-E014, name: trace_or_residue_violation, domain: RES,
  reaction: [FAIL, quarantine_run]}
- {code: PSK-E015, name: unbound_nondeterminism_or_divergence, domain: REPLAY,
  reaction: [divergence_report, block_release]}
- {code: PSK-E016, name: budget_or_schedule_violation, domain: RES,
  reaction: [HOLD, residue_budget]}
- {code: PSK-E017, name: morphogenesis_violation, domain: MORPH,
  reaction: [HOLD, block_spawn]}
- {code: PSK-E018, name: release_gate_blocked, domain: ID,
  reaction: [REJECTED_or_QUARANTINED]}
- {code: PSK-E101, name: boot_precondition_failed, domain: ID,
  reaction: [FAIL_closed, no_kernel]}
- {code: PSK-E102, name: canonicalization_failed, domain: SCHEMA,
  reaction: [FAIL, no_digest]}
- {code: PSK-E103, name: identity_digest_mismatch, domain: ID,
  reaction: [FAIL, no_kernel]}
stability: codes_are_never_repurposed; changed_meaning_requires_new_code

```


Requirement-Matrix

Listing H.1. requirement_matrix.yaml (kanonisch + Architektur)

```

schema: psk.requirement-matrix/1.1
coverage_required_for_reference_release: 1.0
requirements:
- {id: R-ID-001, severity: MUST, artifact: constitution.lock.json, test: T-ID-001,
  statement: "Every runtime binds to the constitution lock."}
- {id: R-TYPE-001, severity: MUST, artifact: schemas/, test: T-SCHEMA-001,
  statement: "Every committed artifact validates against a registered schema."}
- {id: R-ANCHOR-001, severity: MUST, artifact: anchor_snapshot.schema.json, test: T-ANCHOR
  -001,
  statement: "External claims reference a sealed anchor or remain explicitly unanchored."}
- {id: R-FIELD-001, severity: MUST, artifact: field_identity.schema.json, test: T-FIELD-001,
  statement: "No field identity is treated as the system identity."}
- {id: R-DEP-001, severity: MUST, artifact: gate_policy.yaml, test: T-DEP-001,
  statement: "Consensus uses an effective dependency-quotiented witness rank."}
- {id: R-GATE-001, severity: MUST, artifact: state_machines.yaml, test: T-EFFECT-001,
  statement: "No external effect occurs without a passing gate and token."}
- {id: R-RECON-001, severity: MUST, artifact: state_machines.yaml, test: T-RECON-001,
  statement: "No fact reaches ACTUALIZED without external reconciliation."}
- {id: R-TRACE-001, severity: MUST, artifact: invariants.yaml, test: T-TRACE-001,
  statement: "Every transition emits trace linkage."}
- {id: R-REPLAY-001, severity: MUST, artifact: release_gate.yaml, test: T-REPLAY-001,
  statement: "A release candidate reaches replay class R2 or higher."}
- {id: R-GLUE-001, severity: MUST, artifact: proof_obligations.yaml, test: T-GLUE-001,
  statement: "Global closure requires overlap compatibility and a second-order gate."}
- {id: R-MORPH-001, severity: MUST, artifact: state_machines.yaml, test: T-MORPH-001,
  statement: "Facet morphogenesis is gain-, budget-, lineage- and rollback-bound."}
- {id: R-RES-001, severity: MUST, artifact: invariants.yaml, test: T-RES-001,
  statement: "No non-pass state disappears without residue, hold reason or quarantine."}
# --- Architekturanforderungen dieses Werkes ---
- {id: R-RA-001, severity: MUST, artifact: architecture.lock.json, test: T-ARCH-001,
  statement: "Every runtime binds to the architecture lock."}
- {id: R-RA-002, severity: MUST, artifact: module_map.yaml, test: T-ARCH-002,
  statement: "Module dependency graph is acyclic and layer-respecting."}
- {id: R-RA-003, severity: MUST, artifact: port_registry.yaml, test: T-PORT-001,
  statement: "Every cross-module call maps to exactly one registered port."}
- {id: R-RA-004, severity: MUST, artifact: object_registry.yaml, test: T-OWN-001,
  statement: "Every object type has exactly one owning module."}
- {id: R-RA-005, severity: MUST, artifact: state_machines.yaml, test: T-FSM-001,
  statement: "Every transition references a registered gate or null."}
- {id: R-RA-006, severity: MUST, artifact: ml3_topology.yaml, test: T-M13-001,
  statement: "Topology has exactly 13 nodes, 30 edges, 18 cells."}
- {id: R-RA-007, severity: MUST, artifact: pass_registry.yaml, test: T-PASS-001,
  statement: "Passes run in canonical order and are pure functions of declared inputs."}
- {id: R-RA-008, severity: MUST, artifact: memory_registry.yaml, test: T-MEM-001,
  statement: "Only declared promotion paths between memory classes occur."}
- {id: R-RA-009, severity: MUST, artifact: capability_matrix.yaml, test: T-SEC-001,
  statement: "Capabilities are enforced at substrate level, not by convention."}
- {id: R-RA-010, severity: MUST, artifact: refinement_map.yaml, test: T-REF-001,
  statement: "Every concrete transition maps forward onto an abstract transition."}
- {id: R-RA-011, severity: MUST, artifact: module_map.yaml, test: T-OBS-001,
  statement: "Observability has no write path and no kernel influence."}
- {id: R-RA-012, severity: MUST, artifact: port_registry.yaml, test: T-REPLAY-002,
  statement: "Replay profile executes no real effect."}

```


Proof-Obligation-Register

ID	Aussage	Nachweisform
PO-CAN-001	Kanonisierung deterministisch und idempotent	Property Tests, Golden Vectors
PO-REF-001	Implementierung simuliert die abstrakten Automaten vorwärts	Model Check oder Trace Refinement
PO-GATE-001	Gate-Fail verhindert Effekt	Negativtest, Capability Audit
PO-REPLAY-001	R2 rekonstruiert denselben kanonischen Zustand	Replaytest
PO-GLUE-001	Inkompatible Sektionen committen nicht global	Integrations- und Obstruktionstest
PO-DEP-001	Korrelierte Facetten erhöhen den Witnessrang nicht künstlich	Synthetischer Dependencetest
PO-RECON-001	Aktualisierung benötigt unabhängigen Außenrecord	Fault Injection, Receipt Test
PO-MORPH-001	Morphogenese budgetiert, reversibel, lineagevoll	Automaten- und Rollbacktest
PO-SEC-001	Capabilities erzwingen Least Privilege	Policy- und Sandboxtest
PO-EXT-001	Domänenleistung unterscheidet sich von der Baseline	Benchmark (nicht kernblockierend)
PO-TYPE-001	Alle IR-Knoten und Kanten erfüllen Sorten- und Portverträge	Property Test über Sorten-Port-Matrix
PO-ANCH-001	Jeder extern geltende Claim steigt aus einem Anker ab	Statische Analyse plus Negativtest
PO-RT-001	$\text{Can}(\text{Reanalyse}(\text{Execute}(p(M)))) \in [M]$	Round-Trip-Suite
PO-BYP-001	Kein Pfad überspringt Witness, Gate, Versuch, Receipt, Reconciliation	Pfadüberdeckung aller Automaten
PO-M13-001	Alle 18 Zellen schließen oder emittieren ein Seam-Residuum	Zellüberdeckungstest
PO-SELF-001	Eine erzeugte Spezifikationsbranch kann sich nicht selbst aktivieren	Isolationstest, G-SELF-COMPILE
PO-ARCH-001	Jede Codeeinheit bildet auf genau ein Modul ab	Statische Analyse des Buildgraphen
PO-ARCH-002	Jede modulübergreifende Kante bildet auf genau einen Port ab	Aufrufgraphanalyse plus T-PORT-001
PO-ARCH-003	Nebenläufige Läufe sind serialisierbar	Deterministischer Stresslauf, Digestvergleich
PO-ARCH-004	Beobachtung verändert den kanonischen Digest nicht	Differenzlauf mit und ohne M27

Konformanz- und Negativtestregister

Listing J.1. conformance_tests.yaml (kanonisch + Architektur)

```

schema: psk.conformance-tests/1.1
tests:
  # --- kanonische Pflichtzeugen ---
  - {id: T-ID-001, kind: negative, mutation: flip_core_axiom,
    expect: constitution_id_changes_and_boot_fails}
  - {id: T-SCHEMA-001, kind: negative, mutation: remove_required_thought_field,
    expect: schema_reject}
  - {id: T-ANCHOR-001, kind: negative, mutation: emit_external_claim_without_anchor,
    expect: HOLD}
  - {id: T-FIELD-001, kind: negative, mutation: set_system_identity_to_field_id,
    expect: FAIL}
  - {id: T-DEP-001, kind: negative, mutation: clone_same_model_into_100_facets,
    expect: effective_rank_not_100}
  - {id: T-EFFECT-001, kind: negative, mutation: call_effector_without_token,
    expect: FAIL_no_side_effect}
  - {id: T-RECON-001, kind: negative, mutation: mark_attempt_as_actualized_without_receipt,
    expect: FAIL}
  - {id: T-TRACE-001, kind: negative, mutation: delete_prior_residue,
    expect: FAIL}
  - {id: T-REPLAY-001, kind: positive, run: golden/golden_run.json,
    expect: canonical_state_digest_matches}
  - {id: T-GLUE-001, kind: negative, mutation: incompatible_overlap,
    expect: RESIDUE_no_global_commit}
  - {id: T-MORPH-001, kind: negative, mutation: spawn_field_without_gain_or_budget,
    expect: HOLD}
  - {id: T-RES-001, kind: negative, mutation: drop_nonpass_artifact,
    expect: FAIL}
  - {id: T-PERSONA-001, kind: negative, mutation: persona_claims_constitutional_authority,
    expect: FAIL}
  - {id: T-SCOPE-001, kind: negative, mutation: promote_cluster_consensus_to_external,
    expect: FAIL}
  - {id: T-UNKNOWN-001, kind: negative, mutation: verbal_uncertainty_without_internal_block,
    expect: FAIL}
  - {id: T-FORECAST-001, kind: negative, mutation: overwrite_old_forecast_after_observation,
    expect: trace_violation}
  - {id: T-RECEIPT-001, kind: negative, mutation:
    replace_external_record_by_internal_simulation,
    expect: FAIL}
  - {id: T-CAP-001, kind: negative, mutation: effect_role_issues_own_token,
    expect: capability_violation}
  - {id: T-BOUND-001, kind: negative, mutation: expose_internal_freedom_as_external_capability,
    expect: FAIL}
  - {id: T-M13-001, kind: property, run: m13_topology.yaml,
    expect: node_count==13 and edge_count==30 and cell_count==18}
  - {id: T-M13-002, kind: negative, mutation: remove_one_edge_from_topology,
    expect: boot_fail_PSK_E011}
  # --- Architekturpflichtzeugen ---
  - {id: T-ARCH-001, kind: negative, mutation: alter_architecture_register_without_lock_update,
    expect: boot_fail_PSK_E103}
  - {id: T-ARCH-002, kind: negative, mutation: introduce_cyclic_module_dependency,
    expect: build_fail}
  - {id: T-PORT-001, kind: negative, mutation: add_direct_cross_module_call_without_port,
    expect: build_fail}
  - {id: T-OWN-001, kind: negative, mutation: create_owned_object_from_foreign_module,
    expect: FAIL_PSK_E014}
  - {id: T-FSM-001, kind: negative, mutation: take_forbidden_transition,
    expect: FAIL}
  - {id: T-PASS-001, kind: negative, mutation: reorder_compiler_passes,
    expect: divergence_report}
  - {id: T-MEM-001, kind: negative, mutation: promote_residual_to_canonical_directly,
    expect: FAIL}
  - {id: T-SEC-001, kind: negative, mutation: adapter_writes_outside_token_scope,
    expect: blocked_by_substrate_and_FAIL}

```

```

- {id: T-REF-001, kind: negative, mutation: add_unmapped_concrete_transition,
  expect: refinement_check_fail}
- {id: T-OBS-001, kind: negative, mutation: observability_writes_to_store,
  expect: capability_violation}
- {id: T-REPLAY-002, kind: negative, mutation: replay_triggers_real_effect,
  expect: FAIL_PSK_E015}
- {id: T-CONC-001, kind: property, run: concurrent_stress,
  expect: canonical_digest_equals_sequential}
- {id: T-OBSV-001, kind: property, run: with_and_without_profiling,
  expect: identical_canonical_digest}
- {id: T-CAN-001, kind: property, run: canonicalization_idempotence,
  expect: Can(Can(x)) == Can(x)}
- {id: T-IR-001, kind: property, run: ir_roundtrip,
  expect: encode(decode(x)) == x}
- {id: T-RATCHET-001, kind: property, run: ratchet_monotonicity,
  expect: allowed_next_non_increasing}

```

Regel J.1 (Positive Abnahmetests). 1. Kanonisierung ist idempotent und formatinvariant.

2. Lockdateien und neu berechnete Konstitutions- und Architektur-ID stimmen überein.
3. Alle Schemata akzeptieren Golden Objects und verwerfen unvollständige Objekte.
4. Ein deterministischer Golden Run rekonstruiert denselben kanonischen Zustand.
5. Ein gültiger ThoughtBody durchläuft nur erlaubte Promotionspfade.
6. Kompatible Feldsektionen verkleben eindeutig.
7. Der Abhängigkeitsquotient reduziert geklonte Witnesspfade auf den korrekten effektiven Rang.
8. Das Effekt-Token ist plan-, scope- und zeitgebunden und genau einmal konsumierbar.
9. Außenrecord und Reconciliation erzeugen ein vollständiges ActualizationWitness.
10. Das Maschinenzertifikat verweist auf Lock, Architektur, Implementierung, Laufzeit, Gate, Trace, Replay und Residuen.
11. Alle 18 M₁₃-Zellen schließen oder emittieren ein typisiertes Seam-Residuum.
12. Jeder Port P₀₀–P₄₁ wird im Integrationslauf mindestens einmal durchlaufen.

Instruktionsregister

Listing K.1. architecture/isa_registry.yaml

```

schema: psk.isa/1.0
count: 23
instructions:
  - {op: CANON, module: M01, in: [bytes, media], out: [CanonicalBytes, Digest]}
  - {op: TYPE, module: M03, in: [Candidate], out: [SortedGraph]}
  - {op: ANCHOR, module: M05, in: [ExternalRecord], out: [AnchorSnapshot]}
  - {op: PROJECT, module: M09, in: [FieldIdentity, Graph], out: [FieldProjection]}
  - {op: SPLIT, module: M24, in: [Capsule], out: [Surface, InvariantCore]}
  - {op: GLUE, module: M11, in: [Sections], out: [GlobalSection | Obstruction]}
  - {op: QUOTIENT, module: M10, in: [Projections, Sources], out: [DependencyProfile]}
  - {op: CAPSULE, module: M24, in: [QuotientClass], out: [CandidateCapsule]}
  - {op: CHALLENGE, module: M24, in: [Capsule, Countermodels], out: [Capsule]}
  - {op: WITNESS, module: M12, in: [Evidence, Scope], out: [WitnessReport]}
  - {op: GATE, module: M14, in: [Digests], out: [GateReport]}
  - {op: COMPILE, module: M23, in: [CheckedGraph], out: [IRBundle]}
  - {op: EMIT, module: M23, in: [IRBundle, Trace], out: [IRBundle]}
  - {op: TOKENIZE, module: M15, in: [GateReport, Plan], out: [EffectToken]}
  - {op: EXECUTE, module: M16, in: [EffectToken], out: [EffectAttempt]}
  - {op: RECEIPT, module: M17, in: [Scope], out: [ExternalReceipt]}
  - {op: RECONCILE, module: M18, in: [Plan, Attempt, Receipt], out: [ReconciliationReport]}
  - {op: RESIDUALIZE, module: M19, in: [NonClosure], out: [ResidueRecord]}
  - {op: QUARANTINE, module: M19, in: [Object], out: [QuarantineRecord]}
  - {op: REPLAY, module: M19, in: [RunDescriptor, Trace], out: [ReplayResult]}
  - {op: ARCHIVE, module: M19, in: [ClosedRun], out: [ArchiveObject]}
  - {op: EXCISE, module: M20, in: [ExcisionRequest], out: [ExcisionCertificate]}
  - {op: PROPOSE_REVISION, module: M24, in: [Residues, Evidence], out: [RevisionProposal]}
contract: every_instruction_has_typed_ports_pre_post_errors_and_trace_duty

```


Symbol- und Abkürzungsregister

Symbol	Bedeutung
C_{PSK}, C	Kanonische Maschinenkonstitution
S_{PSK}	Kanonische Gesamtspezifikation
A_{RA}	Architekturgegenstand dieses Werkes
I_C, I_A, I_M, I_t	Konstitutions-, Architektur-, Implementierungs- und Laufzeitidentität
Can, H	Kanonisierung; SHA-256
A_t	Bezeugter Gegenwartsanker
T	Gedankenkörper
F_λ, Π_λ	Operative Feldidentität; Spektrallinse
W, r_{eff}	Witness- und Evidencefamilie; effektiver Witnessrang
$G, G^{(2)}$	Gate erster und zweiter Ordnung
RD	Run Descriptor
Δ_{cl}	Closure-Defekt
Recon	Reconciliationoperator
M_{13}	Metatron-Würfel: 13 Knoten, 30 Kanten, 18 Zellen
$\Delta^c, \Delta^b, \Delta^o$	Zentrum-, Brücken- und Boundaryzellen
$T_\gamma, \text{Hol}_\gamma$	Charttransport; Holonomie
$\text{Close}_{360/720}$	Closure erster und zweiter Ordnung
$\mathcal{A}(M)$	Adversariale Closure
CRA	Canonical Revision Attractor
$\Delta G, \theta_G$	Marginaler Strukturgewinn; Schwellenwert
$\Lambda_t^{\text{active}}, B_t$	Aktive Facetten; Budget
$R_a^{\text{coh/law/rea/con/act}}$	Realitätsleiter relativ zu Anker a
τ_i, τ_e	Intrinsische und extrinsische Zeit
PASS, HOLD, FAIL, RESIDUE	Kanonische Gate- und Nichtpasszustände
R0–R3	Replayklassen von narrativ bis bitidentisch
C0–C5	Konformitätsklassen von Artefakt bis externe Reproduktion
FC0–FC8	Abgeleiteter Merkmalsdeckungsvektor
M00–M27	Module der Referenzarchitektur
P00–P41	Ports
C1–C11	Compilerpässe
I0–I8	Realisierungsphasen
WP00–WP15	Work Packages
OBL-001–OBL-010	Offene Implementierungsverpflichtungen

Abschlussvermerk

Die vorliegende Fassung ist die kanonische Architekturnormalform der post-symbolischen Kausalsprache und damit der vollständige Übergang zwischen Theorie und Implementierung. Sie bindet Sorten, Syntaxskelett, Zwischenrepräsentation, Compilerpässe, adversarialen Kanonisierungskern, Laufzeitautomaten, Scheduler, Instruktionsmenge, Speicherarchitektur, Gate-, Capability- und Effektsystem, Witness-, Receipt- und Reconciliationmodell, Trace, Replay, Residuen, Fehlerordnung, Recovery, Beobachtung, Konformanz, Zertifizierung, Repository, Build, Plug-in-Modell, Deployment und Realisierungsplan an genau einen Architekturgegenstand.

Es existieren nach Abschluss dieses Werkes keine offenen Architekturfragen außer den sieben explizit markierten, logisch nicht entscheidbaren Implementierungsverpflichtungen OBL-001–OBL-007.

ÜBERGABE

Der nächste normative Gegenstand ist kein weiteres Dokument, sondern das ausführbare Repository. Der erste definitive Erfolg ist ein lokaler Golden Run von versiegeltem Anker über M_{13} -Platzierung, PSK-IR, adversariale Closure, Gate, Sandbox-Effekt, unabhängigen Außenrecord und geschlossene Reconciliation bis zum reproduzierbaren Archiv und Maschinenzertifikat.

Konstitutions-ID	aa10aa5a9ffe5d9205ce7950905f06a81cbce0e66c7d49b77be2529fd09c0f56
Elterntheorie	829df842e54b9ccb51e2bfa30c6b6af0939828eea26752506e6aa66685861583
Architektur-ID	$I_A = H(\text{Can}(\text{architecture}/*))$, berechnet beim Bundlebuild